TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN — ĐHQG-HCM
KHOA MẠNG MÁY TÍNH & TRUYỀN THÔNG

SỔ TAY HƯỚNG DẪN THỰC THI CHO NHÓM (STUDENT MODE V3.1.1 — FINAL TECHNICAL BASELINE)

ĐỒ ÁN MÔN HỌC: NT106 — LẬP TRÌNH MẠNG CĂN BẢN
Dành riêng cho 3 sinh viên năm 2: Dễ hiểu — Dễ làm — Dễ test — Tự tin vấn đáp
Phiên bản: V3.1.1 (Đồng bộ 100% Wire Protocol SSOT & Không sai lệch kỹ thuật)

📌 BẢN ĐỒ SINH TỒN & TRIẾT LÝ STUDENT MODE V3.1.1

• Dành cho ai: 3 sinh viên năm 2 đang học môn Lập trình mạng, chưa từng xây dựng hệ thống phân tán lớn.
• Mục tiêu tối thượng: LÀM ĐƯỢC ĐỒ ÁN, HIỂU RÕ CODE, DEMO TRƠN TRU, BẢO VỆ VẤN ĐÁP TỰ TIN.
• Phương châm: DỄ HIỂU > DỄ LÀM > DỄ CHECK > ĐẸP > QUY TRÌNH PHỨC TẠP.
• Chuẩn mực kỹ thuật: Tuyệt đối tuân thủ Wire Protocol 03_PROTOCOL_SPEC.md và CONTRACT_MATRIX.md.
• Trợ lực AI: Mỗi nhiệm vụ đều có prompt may đo để copy hỏi AI, kèm hộp 'PHẦN BẠN PHẢI TỰ HIỂU' và 3 bước xử lý lỗi cụ thể.

BẮT BUỘC ĐỌC: NẾU BẠN NHỜ AI VIẾT CODE — HÃY LÀM ĐÚNG 5 BƯỚC NÀY

Khuyến cáo học tập: Sinh viên năm 2 hoàn toàn có quyền sử dụng AI (ChatGPT, Claude, Copilot) để hỗ trợ học tập, giải thích code và gỡ lỗi. Tuy nhiên, nếu bạn chỉ copy-paste code mà không hiểu bản chất, bạn sẽ gặp rủi ro rất lớn khi hội đồng hỏi thi: 'Tại sao chỗ này dùng hàm này?' hoặc 'Bỏ dòng code này đi thì hệ thống xảy ra lỗi gì?'.

> **📌 QUY TRÌNH 5 BƯỚC KHI DÙNG AI (NẾU CHƯA XONG BƯỚC 5 = TASK CHƯA HOÀN THÀNH)**
> 1. BƯỚC 1: Đọc kỹ tên Class và tên Function mà AI viết ra. Nhận diện xem nó làm nhiệm vụ gì trong đề cương.
> 2. BƯỚC 2: Hỏi lại AI: 'Input (đầu vào) của hàm này là gì? Output (kết quả trả về) là gì? Dữ liệu đi đâu tiếp theo?'.
> 3. BƯỚC 3: Hỏi AI: 'Tại sao em cần đoạn code này? Nếu em xóa khối lệnh try/catch hoặc using này đi thì hệ thống bị lỗi gì?'.
> 4. BƯỚC 4: Tự chạy code trên máy mình và thử nghiệm tình huống lỗi (ví dụ: rút dây mạng, gửi sai JSON) để tận mắt thấy ngoại lệ văng ra.
> 5. BƯỚC 5: Tự giải thích lại toàn bộ hàm bằng lời của chính mình cho bạn cùng nhóm nghe mà KHÔNG ĐƯỢC NHÌN MÀN HÌNH AI.
> 👉 NẾU CHƯA TỰ GIẢI THÍCH ĐƯỢC BƯỚC 5: Tạm thời giữ trạng thái [Đang làm], chưa vội đánh dấu [Xong]!

PHẦN 1: KIẾN TRÚC LABGUARD NÓI BẰNG TIẾNG NGƯỜI

1.1. Ví dụ đời thường: Đừng nghĩ LabGuard là cái gì đó quá cao siêu. Hãy tưởng tượng toàn bộ hệ thống giống như một Trạm kiểm soát y tế trong trường học:

Ba thành phần trong ví dụ: 1. Máy trạm học sinh (Agent): Giống như bạn học sinh cứ 5 giây tự đo nhiệt độ, nhịp tim của mình một lần rồi gửi phiếu kết quả về phòng y tế.
2. Máy chủ trung tâm (Central Server): Giống như bác sĩ trực phòng y tế, ngồi nhận phiếu của các học sinh gửi về, kiểm tra xem có phiếu nào bị gửi trùng không (Deduplication), lưu vào sổ cái (CSDL) và nếu thấy bạn nào sốt trên 39 độ thì phát loa báo động (Incident Alert).
3. Màn hình quản trị WinForms (Dashboard): Giống như cái bảng điện tử ở cổng trường hiển thị các bóng đèn tương ứng các học sinh: Đèn Xanh là khỏe (ONLINE), Đèn Vàng là cảnh báo (WARNING), Đèn Đỏ là sự cố (CRITICAL) và Đèn Xám là mất liên lạc (OFFLINE).

1.2. Bản chất dòng byte TCP (Tại sao không gửi nhận JSON thẳng qua socket?)

Nhiều bạn thắc mắc: 'Tại sao em dùng socket gửi chuỗi JSON mà bên nhận parse toàn bị lỗi cú pháp?'. Lý do là vì TCP là giao thức Hướng dòng byte (Byte Stream), giống hệt như dòng nước chảy liên tục trong ống. TCP hoàn toàn KHÔNG biết ranh giới bức thư của bạn. Khi bạn gửi 2 bức thư liên tiếp, bên nhận có thể nhận được nửa bức thư đầu (Partial Read) hoặc nhận cả 2 bức thư dính chặt vào nhau (Frame Coalescing).
👉 Giải pháp của LabGuard (FrameCodec): Trước mỗi bức thư, ta dán thêm 4 byte header nhị phân ghi rõ độ dài N của bức thư (Network Byte Order Big-Endian). Bên nhận dùng ReadExactlyBytesAsync đọc đủ đúng 4 byte đầu để biết độ dài N (với 0 < N <= 256 KiB), sau đó gom đủ đúng N byte UTF-8 thì mới đem đi phân tích JSON. Nhờ vậy hệ thống an toàn và không bao giờ giải mã JSON dở dang!

PHẦN 2: KẾ HOẠCH 5 TUẦN & CÁCH CỨU NGUY ĐỒ ÁN TẠI CỔNG M4

2.1. Mục tiêu 5 tuần: Kế hoạch 5 tuần được tóm gọn bằng 5 mục tiêu cụ thể, bám sát năng lực thực tế của nhóm sinh viên năm 2:

Tuần 1: A chốt wire contract và fixtures (LG-A-001); B chuẩn bị SQL Server và skeleton; C dựng WinForms UI shell; một dữ liệu CPU đi thông suốt trên localhost.

Tuần 2: Nhiều Agent cùng kết nối + gửi nhịp tim sống 5s + phát hiện máy quá tải CPU >=90% (Hysteresis chống rung chớp nháy cảnh báo).

Tuần 3: Vòng đời sự cố hoàn chỉnh (OPEN -> ACKNOWLEDGED -> IN_PROGRESS -> RECOVERED -> RESOLVED, trong đó RECOVERED là trạng thái thiết bị đã phục hồi kỹ thuật, làm điều kiện cần trước khi Admin/Operator bấm RESOLVE để đóng sự cố) + 2 Admin cùng xử lý tranh chấp OCC (bẫy lỗi ERR_CONCURRENCY_VIOLATION) + MVP chạy ổn định.

Tuần 4: Ổn định hệ thống, kiểm thử chịu tải mạng LAN; CHỈ LÀM P1 (Collector) NẾU MVP THẬT SỰ ỔN ĐỊNH VÀ ĐỦ THỜI GIAN.

Tuần 5: Cắm dây test 3 máy thật LAN, quay video demo 12 phút dự phòng và luyện 90 câu hỏi vấn đáp cá nhân.

> **📌 CỔNG M4 CUỐI TUẦN 3 — NGUYÊN TẮC BẢO VỆ PHẠM VI ĐỒ ÁN**
> Nếu đến hết Tuần 3 mà nhóm vẫn còn lỗi Sev1, giao thức chưa thông suốt hoặc thành viên chưa hiểu rõ code: KÍCH HOẠT NGAY QUYẾT ĐỊNH 'DEFER P1'!
> • Hoãn toàn bộ 9 task làm Collector trung gian (-47.0 giờ công dự phòng).
> • Dành trọn vẹn Tuần 4 và Tuần 5 để hoàn thiện MVP cốt lõi (164.5 giờ committed), cắm dây test LAN và luyện vấn đáp.
> • Nguyên tắc đánh giá: Hội đồng đánh giá cao một MVP cốt lõi chạy ổn định, có bằng chứng thực nghiệm rõ ràng và thành viên hiểu sâu kiến trúc hơn là một đồ án cố làm thêm tính năng phân tán nâng cao nhưng

> demo chập chờn hoặc sinh viên không nắm được bản chất!

PHẦN 3: SỔ TAY 20 NHIỆM VỤ CỦA SINH VIÊN A (AGENT & MẠNG TCP/TLS)

Sinh viên A phụ trách phần máy trạm Agent, mã hóa TLS 1.3, bộ giải mã khung (FrameCodec), nhịp tim sống, bộ đệm gửi lại khi ngắt kết nối và bắt gói tin Wireshark. Dưới đây là 20 thẻ nhiệm vụ từ LG-A-001 đến LG-A-020:

Thẻ #01: [LG-A-001] — CHỐT WIRE CONTRACT VÀ PROTOCOL FIXTURES (Tuần: W1 | P0)

• **Mục tiêu cụ thể:** Hoàn thành CONTRACT_MATRIX.md & protocol fixtures đáp ứng tiêu chuẩn Outer envelope 6 fields, Big-Endian uint32 length prefix, UTF-8 JSON

• **Trực giác đời thường:** *Giống như chiếc phong bì thư chuẩn gồm 6 mục ghi rõ người gửi, người nhận, loại thư trước khi nhét phần ruột thư vào bên trong.*

• **Điều kiện:** Cần xong trước: None | Phối hợp: None

• **4 Việc cần làm:**
1. Xác định 6 trường bất biến trong Outer Envelope: version (int=1), messageType (string), requestId (UUIDv4), deviceId (string), sentAtUtc (ISO-8601 UTC), payload (object).
2. Tạo thư mục fixtures và viết 16 mẫu JSON chuẩn (HEARTBEAT, METRIC_BATCH, LOGIN, INCIDENT_ACK...).
3. Viết script kiểm tra cú pháp JSON hợp lệ và đảm bảo không có khoảng trắng thừa hay sai tên trường.
4. Họp nhanh với B và C để cả 3 người cùng review và đồng ý với envelope chuẩn.

> 📌 📋 **PROMPT COPY NHANH CHO AI (LG-A-001)**
> Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai A).
> Tôi đang làm nhiệm vụ LG-A-001 của LabGuard: 'Chốt wire contract và protocol fixtures'.
> Mục tiêu cốt lõi: CONTRACT_MATRIX.md & protocol fixtures.
> Ví dụ trực giác: Giống như chiếc phong bì thư chuẩn gồm 6 mục ghi rõ người gửi, người nhận, loại thư trước khi nhét phần ruột thư vào bên trong.
> Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
> 1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
> 2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục Networking.
> 3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
> 4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
> 5. Hướng dẫn tôi cách chạy test 'TEST-A01' để tự mắt thấy kết quả đúng.
> 6. Nhắc tôi lưu tệp minh chứng: CONTRACT_MATRIX.md, fixture JSON files.
> 7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

> 📌 🤓 **PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)**

- Bản chất kỹ thuật của Outer envelope 6 fields, Big-Endian uint32 length prefix, UTF-8 JSON là gì?
- Tại sao cần xử lý tình huống lỗi: Nếu lệch tên trường thì Central hoặc Admin không deserialize được; rollback về CONTRACT_MATRIX freeze?
- Vị trí tệp và dòng code minh chứng cụ thể trong Networking?

• **Cách chạy test:**
Lệnh/Thao tác: Chạy bài test TEST-A01 (Schema validation tool và JSON parser kiểm tra fixture mẫu)
Kết quả ĐÚNG: Tất cả wire catalog có fixture JSON hợp lệ, không sai lệch tên trường

• **Nếu gặp lỗi (3 bước kiểm tra cụ thể):**
1. Dùng Wireshark kiểm tra 4 byte đầu xem đã đúng thứ tự byte mạng Big-Endian chưa. 2. Kiểm tra chuỗi JSON xem có đủ 6 trường root envelope: version, messageType, requestId, deviceId, sentAtUtc, payload. 3. Đảm bảo trường sentAtUtc tuân thủ định dạng ISO-8601 UTC có đuôi 'Z'.

• **Minh chứng & Vấn đáp:**
Tệp minh chứng: CONTRACT_MATRIX.md, fixture JSON files
Câu hỏi thi chính: Framing nhị phân và ranh giới thông điệp TCP? TCP là stream-oriented không bảo toàn ranh giới; cần 4 byte length-prefix để bên nhận biết chính xác kích thước frame

Thẻ #02: [LG-A-002] — FRAMECODEC VÀ XỬ LÝ PARTIAL READS / MESSAGE FRAMING (Tuần: W1 | P0)

• **Mục tiêu cụ thể:** Hoàn thành FrameCodec.cs & Unit tests đáp ứng tiêu chuẩn FrameCodec.ReadExactlyBytesAsync, MaxFrameSize = 262144 bytes

• **Trực giác đời thường:** *Giống như hứng nước từ vòi theo xô dán nhãn: Đọc đúng 4 byte nhãn ghi dung tích N, rồi gom đủ đúng N byte thì mới đem đi dịch JSON, không bao giờ dịch dở dang.*

• **Điều kiện:** Cần xong trước: LG-A-001 | Phối hợp: None

• **4 Việc cần làm:**
1. Hiện thực hàm ReadExactlyBytesAsync(Stream stream, int count) dùng vòng lặp while (read < count) tích lũy byte.
2. Nếu ReadAsync trả về 0 byte giữa chừng, ném ngay EndOfStreamException (xử lý ngắt kết nối an toàn).
3. Đọc 4 byte header, dùng BinaryPrimitives.ReadUInt32BigEndian để giải mã độ dài payload N.
4. Kiểm tra điều kiện bảo mật: nếu N > 262,144 bytes (256 KiB), ném InvalidDataException và đóng kết nối ngay (chống tấn công làm tràn RAM OOM).

📌 📋 PROMPT COPY NHANH CHO AI (LG-A-002)
Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai A).
Tôi đang làm nhiệm vụ LG-A-002 của LabGuard: 'FrameCodec và xử lý partial reads / message framing'.
Mục tiêu cốt lõi: FrameCodec.cs & Unit tests.
Ví dụ trực giác: Giống như hứng nước từ vòi theo xô dán nhãn: Đọc đúng 4 byte nhãn ghi dung tích N, rồi

> gom đủ đúng N byte thì mới đem đi dịch JSON, không bao giờ dịch dở dang.
> Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
> 1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
> 2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục Networking.
> 3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
> 4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
> 5. Hướng dẫn tôi cách chạy test 'TEST-A02' để tự mắt thấy kết quả đúng.
> 6. Nhắc tôi lưu tệp minh chứng: scratch/test_frame_codec/Program.cs, test_frame_codec_run.log.
> 7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

> 📌 🧠 **PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)**
> - Bản chất kỹ thuật của FrameCodec.ReadExactlyBytesAsync, MaxFrameSize = 262144 bytes là gì?
> - Tại sao cần xử lý tình huống lỗi: Đọc thiếu byte dẫn đến parse JSON hỏng; ném EndOfStreamException và đóng kết nối an toàn?
> - Vị trí tệp và dòng code minh chứng cụ thể trong Networking?

• **Cách chạy test:**
Lệnh/Thao tác: Chạy bài test TEST-A02 (Unit test đọc 1-byte chunking, 3 frame liền mạch và frame vượt 256 KiB)
Kết quả ĐÚNG: Pass 5/5 kịch bản unit test; không nuốt ngoại lệ stream; không cấp phát buffer khi frame size quá lớn

• **Nếu gặp lỗi (3 bước kiểm tra cụ thể):**
1. Đặt breakpoint tại ReadExactlyBytesAsync kiểm tra số byte nhận được; nếu stream ngắt sớm thì bẫy EndOfStreamException. 2. Kiểm tra nếu N > 262,144 byte (256 KiB) thì ném lỗi ERR_FRAME_TOO_LARGE và đóng kết nối. 3. Mô phỏng chia nhỏ mảng byte (partial chunk) để xác nhận parser không giải mã JSON dở dang.

• **Minh chứng & Vấn đáp:**
Tệp minh chứng: scratch/test_frame_codec/Program.cs, test_frame_codec_run.log
Câu hỏi thi chính: Hiện tượng Partial Read và cách xử lý trong socket TCP? Stream.ReadAsync có thể trả về ít byte hơn yêu cầu; phải lặp vòng lặp tích lũy byte cho đến khi đủ count

Thẻ #03: [LG-A-003] — AGENT TCP CONNECTION VÀ WRITER QUEUE (Tuần: W1 | P0)

• **Mục tiêu cụ thể:** Hoàn thành AgentClient.cs & WriterLoop đáp ứng tiêu chuẩn TcpClient, NetworkStream, Channel<byte[]> Bounded(100)

• **Trực giác đời thường:** *Giống như cửa xoay 1 người: Mọi luồng trong Agent phải xếp hàng tuần tự qua socket, không để 2 luồng cùng lúc ghi đè làm nát gói tin.*

• **Điều kiện:** Cần xong trước: LG-A-002 | Phối hợp: None

• **4 Việc cần làm:**
1. Tạo một Channel<byte[]> có giới hạn kích thước (BoundedChannelOptions(100)).
2. Viết vòng lặp nền WriterLoopAsync làm nhiệm vụ duy nhất: đọc từng gói tin từ Channel và ghi vào NetworkStream.
3. Khi các luồng khác muốn gửi tin, chỉ cần gọi writerChannel.Writer.WriteAsync(frame).
4. Thiết lập chính sách khi hàng đợi đầy (Backpressure): ghi log cảnh báo và xử lý drop tin không quan trọng nếu mạng bị nghẽn kéo dài.

> 📌 📋 **PROMPT COPY NHANH CHO AI (LG-A-003)**
> Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai A).
> Tôi đang làm nhiệm vụ LG-A-003 của LabGuard: 'Agent TCP connection và writer queue'.
> Mục tiêu cốt lõi: AgentClient.cs & WriterLoop.
> Ví dụ trực giác: Giống như cửa xoay 1 người: Mọi luồng trong Agent phải xếp hàng tuần tự qua socket, không để 2 luồng cùng lúc ghi đè làm nát gói tin.
> Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
> 1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
> 2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục Networking.
> 3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
> 4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
> 5. Hướng dẫn tôi cách chạy test 'TEST-A03' để tự mắt thấy kết quả đúng.
> 6. Nhắc tôi lưu tệp minh chứng: Trace log socket client, Mini Test Server output.
> 7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

> 📌 🧠 **PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)**
> - Bản chất kỹ thuật của TcpClient, NetworkStream, Channel<byte[]> Bounded(100) là gì?
> - Tại sao cần xử lý tình huống lỗi: Nhiều luồng ghi đồng thời vào NetworkStream làm hỏng framing; Channel bảo đảm đúng 1 luồng ghi duy nhất?
> - Vị trí tệp và dòng code minh chứng cụ thể trong Networking?

• **Cách chạy test:**
Lệnh/Thao tác: Chạy bài test TEST-A03 (Chạy song song 10 thread ghi vào writer channel tới Mini Test Server)
Kết quả ĐÚNG: Dữ liệu trên socket được ghi tuần tự, không bị đan xen byte giữa các frame

• **Nếu gặp lỗi (3 bước kiểm tra cụ thể):**
1. Kiểm tra SemaphoreSlim(1,1) hoặc Channel bounded (BoundedChannelOptions(100) [CP2 TO VERIFY]): đảm bảo mọi luồng ghi đều qua hàng đợi tập trung. 2. Bắt IOException / SocketException khi socket bị

đóng phía đầu xa và kích hoạt cờ IsConnected = false. 3. Đảm bảo không gọi song song nhiều lệnh stream.WriteAsync trên cùng một NetworkStream.

• **Minh chứng & Vấn đáp:**
Tệp minh chứng: Trace log socket client, Mini Test Server output
Câu hỏi thi chính: Single Writer pattern trên TCP socket đa luồng? NetworkStream không an toàn đa luồng cho các thao tác ghi đồng thời; Channel làm hàng đợi trung chuyển đảm bảo 1 consumer ghi tuần tự

Thẻ #04: [LG-A-004] — TLS CLIENT VÀ LOCAL IDENTITY ENROLLMENT (Tuần: W1 | P0)

• **Mục tiêu cụ thể:** Hoàn thành TlsClientHandler.cs & IdentityStore đáp ứng tiêu chuẩn SslStream, RemoteCertificateValidationCallback, Local DeviceId store

• **Trực giác đời thường:** *Giống như gọi điện qua đường dây đã khóa mã và kiểm tra đúng người ở đầu bên kia (TLS handshake, chứng chỉ số và định danh máy trạm).*

• **Điều kiện:** Cần xong trước: LG-A-003 | Phối hợp: LG-B-003

• **4 Việc cần làm:**
1. Bọc NetworkStream bên trong SslStream: new SslStream(networkStream, false, ValidateServerCertificate).
2. Gọi AuthenticateAsClientAsync(serverName) để tiến hành bắt tay (Handshake) TLS.
3. Trong hàm kiểm tra chứng chỉ: so khớp Certificate.GetCertHashString() với Thumbprint cấu hình sẵn (Certificate Pinning) để chấp nhận cert tự ký trong lab mà vẫn chặn giả mạo (MITM).
4. Đọc/ghi dữ liệu thông qua SslStream thay vì NetworkStream thô.

> 📌 📋 **PROMPT COPY NHANH CHO AI (LG-A-004)**
> Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai A).
> Tôi đang làm nhiệm vụ LG-A-004 của LabGuard: 'TLS client và local identity enrollment'.
> Mục tiêu cốt lõi: TlsClientHandler.cs & IdentityStore.
> Ví dụ trực giác: Giống như gọi điện qua đường dây đã khóa mã và kiểm tra đúng người ở đầu bên kia (TLS handshake, chứng chỉ số và định danh máy trạm).
> Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
> 1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
> 2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục Security.
> 3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
> 4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
> 5. Hướng dẫn tôi cách chạy test 'TEST-A04' để tự mắt thấy kết quả đúng.
> 6. Nhắc tôi lưu tệp minh chứng: Wireshark packet dump mã hóa TLS, client error log.
> 7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

> 📌 🧠 **PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)**
> - Bản chất kỹ thuật của SslStream, RemoteCertificateValidationCallback, Local DeviceId store là gì?
> - Tại sao cần xử lý tình huống lỗi: MITM attack; từ chối kết nối nếu chứng chỉ không xác minh được tính toàn vẹn?
> - Vị trí tệp và dòng code minh chứng cụ thể trong Security?

• **Cách chạy test:**
Lệnh/Thao tác: Chạy bài test TEST-A04 (Kết nối TLS tới server có cert tự ký; thử nghiệm cert giả mạo để kiểm tra từ chối)
Kết quả ĐÚNG: Bắt tay TLS thành công với cert hợp lệ; ngắt kết nối ngay khi cert không khớp thumbprint

• **Nếu gặp lỗi (3 bước kiểm tra cụ thể):**
1. Dùng lệnh 'openssl s_client -connect localhost:5000' để kiểm tra chứng chỉ máy chủ. 2. Đảm bảo chứng chỉ SSL/TLS có cấu hình SAN (Subject Alternative Name) chứa đúng IP máy chủ LAN. 3. Bắt AuthenticationException để ghi log nguyên nhân từ chối bắt tay TLS.

• **Minh chứng & Vấn đáp:**
Tệp minh chứng: Wireshark packet dump mã hóa TLS, client error log
Câu hỏi thi chính: Bảo mật kênh truyền với SslStream trong .NET? SslStream bọc ngoài NetworkStream, mã hóa payload và bắt tay bảo mật; callback kiểm tra thumbprint ngăn chặn tấn công giả mạo

Thẻ #05: [LG-A-005] — THU THẬP CPU, RAM, NIC, UPTIME HỆ THỐNG (Tuần: W1 | P0)

• **Mục tiêu cụ thể:** Hoàn thành WindowsMetricsCollector.cs đáp ứng tiêu chuẩn IMetricsCollector: ReadCpuPercent(), ReadMemoryBytes(), ReadNicBytes()

• **Trực giác đời thường:** *Giống như cụm đồng hồ xe máy hiển thị tốc độ, xăng và quãng đường: Đo định kỳ CPU %, RAM sử dụng, số byte card mạng gửi/nhận và thời gian máy chạy (Uptime).*

• **Điều kiện:** Cần xong trước: LG-A-001 | Phối hợp: None

• **4 Việc cần làm:**
1. Khởi tạo PerformanceCounter cho CPU ('Processor', '% Processor Time', '_Total').
2. Lấy thông tin RAM vật lý qua API GC.GetGCMemoryInfo() hoặc P/Invoke GlobalMemoryStatusEx.
3. Đọc lưu lượng card mạng qua NetworkInterface.GetIPv4Statistics().
4. Đảm bảo thời gian đọc không vượt quá 100ms; nếu đọc lỗi thì trả về null an toàn kèm cờ lỗi, không crash Agent.

> 📌 📋 **PROMPT COPY NHANH CHO AI (LG-A-005)**
> Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai A).
> Tôi đang làm nhiệm vụ LG-A-005 của LabGuard: 'Thu thập CPU, RAM, NIC, Uptime hệ thống'.

> Mục tiêu cốt lõi: WindowsMetricsCollector.cs.
> Ví dụ trực giác: Giống như cụm đồng hồ xe máy hiển thị tốc độ, xăng và quãng đường: Đo định kỳ CPU %, RAM sử dụng, số byte card mạng gửi/nhận và thời gian máy chạy (Uptime).
> Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
> 1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
> 2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục System.
> 3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
> 4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
> 5. Hướng dẫn tôi cách chạy test 'TEST-A05' để tự mắt thấy kết quả đúng.
> 6. Nhắc tôi lưu tệp minh chứng: Raw metrics CSV log so sánh với Task Manager.
> 7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

> 📌 🧠 **PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)**
> - Bản chất kỹ thuật của IMetricsCollector: ReadCpuPercent(), ReadMemoryBytes(), ReadNicBytes() là gì?
> - Tại sao cần xử lý tình huống lỗi: Lỗi truy cập counter hiệu năng; fallback sang WMI hoặc trả giá trị an toàn kèm cờ lỗi?
> - Vị trí tệp và dòng code minh chứng cụ thể trong System?

• **Cách chạy test:**
Lệnh/Thao tác: Chạy bài test TEST-A05 (Chạy trên Windows 11 thật và máy ảo tải giả lập; đối chiếu số liệu với Task Manager)
Kết quả ĐÚNG: Chỉ số CPU/RAM đo liên tục mỗi 5s sai lệch <= 5% so với Task Manager, thời gian thu thập < 100ms

• **Nếu gặp lỗi (3 bước kiểm tra cụ thể):**
1. Kiểm tra quyền chạy ứng dụng: trên Windows một số counter yêu cầu quyền Administrator hoặc tài khoản tiêu chuẩn. 2. Kiểm tra công thức tính phần trăm CPU giữa 2 lần đo chênh lệch thời gian, tránh chia cho 0. 3. Đảm bảo mảng byte mạng NIC không bị tràn số (dùng kiểu ulong hoặc uint64).

• **Minh chứng & Vấn đáp:**
Tệp minh chứng: Raw metrics CSV log so sánh với Task Manager
Câu hỏi thi chính: Kỹ thuật thu thập chỉ số hiệu năng hệ điều hành Windows? Dùng PerformanceCounter lấy mẫu delta giữa 2 thời điểm; cần chạy trong background thread để tránh giật giao diện

Thẻ #06: [LG-A-006] — HEARTBEAT VÀ BATCH SENDER (Tuần: W1 | P0)

• **Mục tiêu cụ thể:** Hoàn thành BatchScheduler.cs & SenderQueue đáp ứng tiêu chuẩn HEARTBEAT chu kỳ 5s, METRIC_BATCH (batchId, bootId, metrics)

• **Trực giác đời thường:** *Giống như nhân viên trực cứ 5 giây nhắn 'em vẫn ở đây' và định kỳ gửi kiện hàng số liệu về kho (Heartbeat 5s và batch sender).*

• **Điều kiện:** Cần xong trước: LG-A-003, LG-A-005 | Phối hợp: LG-B-005

• **4 Việc cần làm:**

1. Sinh một mã BootId ngẫu nhiên (UUIDv4) mỗi khi tiến trình Agent khởi động (đại diện cho một phiên bật máy).
2. Thiết lập vòng lặp nhịp tim gửi HEARTBEAT với uptimeSeconds đều đặn mỗi 5 giây.
3. Thiết lập vòng lặp telemetry gom dữ liệu gửi METRIC_BATCH với mã batchId tự tăng.
4. Đưa gói tin vào hàng đợi chờ xác nhận; khi nhận BATCH_ACK từ Central Server thì xóa mẻ tin khỏi bộ nhớ.

📌 📋 PROMPT COPY NHANH CHO AI (LG-A-006)

Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai A).
Tôi đang làm nhiệm vụ LG-A-006 của LabGuard: 'Heartbeat và batch sender'.
Mục tiêu cốt lõi: BatchScheduler.cs & SenderQueue.
Ví dụ trực giác: Giống như nhân viên trực cứ 5 giây nhắn 'em vẫn ở đây' và định kỳ gửi kiện hàng số liệu về kho (Heartbeat 5s và batch sender).
ĐẶC BIỆT LƯU Ý: Thời gian chờ BATCH_ACK được cấu hình là ACK_TIMEOUT [CP2 TO VERIFY] (giá trị đề xuất ban đầu 5s [CP2 TO VERIFY]). Nếu timeout thì giữ mẻ tin trong replay queue để retry.
Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục Networking.
3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
5. Hướng dẫn tôi cách chạy test 'TEST-A06' để tự mắt thấy kết quả đúng.
6. Nhắc tôi lưu tệp minh chứng: Trace log trao đổi bản tin, time delta log.
7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

📌 🤔 PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)

- Tại sao cơ chế gửi Telemetry Batch cần cờ isReplay bên trong payload khi gửi lại?
- Thời gian chờ xác nhận ACK_TIMEOUT [CP2 TO VERIFY] ảnh hưởng thế nào đến bộ nhớ hàng đợi replay của Agent?
- Sự khác biệt về thứ tự ưu tiên giữa luồng nhịp tim Heartbeat và luồng Telemetry Batch?

• **Cách chạy test:**

Lệnh/Thao tác: Chạy bài test TEST-A06 (Gửi liên tục 100 batch tới Mini Server; đo đạc độ trễ nhận ACK)
Kết quả ĐÚNG: Heartbeat gửi đúng chu kỳ 5s ± 200ms [CP2 TO VERIFY]; nhận BATCH_ACK trong thời gian chờ ACK_TIMEOUT [CP2 TO VERIFY] (đề xuất 5s) và xóa mẻ khỏi queue bộ nhớ.

• **Nếu gặp lỗi (3 bước kiểm tra cụ thể):**

1. Đặt chu kỳ Timer đúng 5 giây, kiểm tra bản tin HEARTBEAT có chứa đúng bootId và uptimeSeconds. 2. Kiểm tra luồng gửi batch: nếu không nhận được BATCH_ACK sau thời gian chờ ACK_TIMEOUT [CP2 TO

VERIFY] (giá trị đề xuất 5s) thì giữ mẻ tin trong replay queue để retry. 3. Đảm bảo cờ isReplay được đặt bên trong payload của METRIC_BATCH khi gửi lại.

• **Minh chứng & Vấn đáp:**
Tệp minh chứng: Trace log trao đổi bản tin, time delta log
Câu hỏi thi chính: Cơ chế phân tách luồng Heartbeat và Telemetry Batch? Heartbeat là bản tin nhỏ ưu tiên cao để duy trì liveness; telemetry batch là dữ liệu lớn gom mẻ để giảm tải socket

Thẻ #07: [LG-A-007] — LOCAL TCP SERVICE PROBE THEO ALLOWLIST (Tuần: W2 | P0)

• **Mục tiêu cụ thể:** Hoàn thành ServiceProbeWorker.cs đáp ứng tiêu chuẩn SERVICE_STATUS (probes: [{port, serviceName, isUp, responseMs}])

• **Trực giác đời thường:** *Giống như bảo vệ đi gõ cửa từng phòng (cổng 80, 443...) xem bên trong có ai trả lời không (TCP connect probe allowlist <= 5 cổng, timeout 2s [CP2 TO VERIFY]).*

• **Điều kiện:** Cần xong trước: LG-A-006 | Phối hợp: None

• **4 Việc cần làm:**
1. Đọc danh sách cổng cho phép từ file cấu hình (tối đa 5 cổng để không biến Agent thành công cụ quét port quá tải).
2. Duyệt qua từng cổng, dùng TcpClient.ConnectAsync("127.0.0.1", port) với CancellationToken giới hạn đúng 2000ms.
3. Nếu kết nối thành công: isUp = true, đo thời gian phản hồi (ms).
4. Nếu timeout hoặc bị từ chối (Connection Refused): isUp = false, đóng socket sạch sẽ.

> 📌 📋 **PROMPT COPY NHANH CHO AI (LG-A-007)**
> Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai A).
> Tôi đang làm nhiệm vụ LG-A-007 của LabGuard: 'Local TCP service probe theo allowlist'.
> Mục tiêu cốt lõi: ServiceProbeWorker.cs.
> Ví dụ trực giác: Giống như bảo vệ đi gõ cửa từng phòng (cổng 80, 443...) xem bên trong có ai trả lời không (TCP connect probe allowlist <= 5 cổng, timeout 2s [CP2 TO VERIFY]).
> Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
> 1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
> 2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục Networking.
> 3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
> 4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
> 5. Hướng dẫn tôi cách chạy test 'TEST-A07' để tự mắt thấy kết quả đúng.
> 6. Nhắc tôi lưu tệp minh chứng: Service probe status log, timing benchmark.
> 7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

> 📌 🤔 **PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)**
> - Bản chất kỹ thuật của SERVICE_STATUS (probes: [{port, serviceName, isUp, responseMs}]) là gì?
> - Tại sao cần xử lý tình huống lỗi: Socket connect bị treo vô hạn; sử dụng CancellationTokenSource với timeout 2000ms để hủy kết nối?
> - Vị trí tệp và dòng code minh chứng cụ thể trong Networking?

• **Cách chạy test:**
Lệnh/Thao tác: Chạy bài test TEST-A07 (Mô phỏng mở và đóng port 80/8080 cục bộ; ghi nhận trạng thái probe)
Kết quả ĐÚNG: Phát hiện port mở trả isUp=true < 50ms; port đóng/bị lọc trả isUp=false sau đúng timeout 2s

• **Nếu gặp lỗi (3 bước kiểm tra cụ thể):**
1. Dùng lệnh PowerShell 'Test-NetConnection -ComputerName 127.0.0.1 -Port <port>' để kiểm tra cổng probe có đang mở thật không. 2. Đặt CancellationTokenSource với timeout đúng 2000ms [CP2 TO VERIFY] để tránh treo luồng probe. 3. Giới hạn danh sách probe tối đa 5 cổng theo đúng quy định allowlist.

• **Minh chứng & Vấn đáp:**
Tệp minh chứng: Service probe status log, timing benchmark
Câu hỏi thi chính: Thăm dò cổng TCP và xử lý timeout bất đồng bộ? Dùng TcpClient.ConnectAsync kèm CancellationToken hoặc Task.WhenAny với Task.Delay để ép timeout 2s, tránh nghẽn thread

Thẻ #08: [LG-A-008] — AGENT SIMULATOR 10/20/50 SOCKET ĐỘC LẬP (Tuần: W2 | P0)

• **Mục tiêu cụ thể:** Hoàn thành LabGuard.Simulator.exe đáp ứng tiêu chuẩn Giả lập N Agent clients, mỗi client có DeviceId, BootId, IP và metrics profile riêng

• **Trực giác đời thường:** *Giống như thuê 20 diễn viên đóng vai 20 máy tính để thử xem máy chủ có chịu nổi không (Agent Simulator 10/20/50 socket độc lập).*

• **Điều kiện:** Cần xong trước: LG-A-006 | Phối hợp: LG-B-003

• **4 Việc cần làm:**
1. Khởi tạo mảng N đối tượng Agent client độc lập, mỗi client có DeviceId (PC-01 .. PC-20) và BootId riêng.
2. Mỗi simulated agent chạy vòng lặp riêng: gửi bắt tay, gửi heartbeat mỗi 5s và gửi metric mỗi 10s có kèm độ trễ ngẫu nhiên (jitter) để tránh đồng loạt gửi cùng một mili-giây.
3. Hiển thị bảng điều khiển console tóm tắt số kết nối đang sống, số batch đã gửi và số ACK nhận được.
4. Hỗ trợ phím bấm giả lập máy mất kết nối hoặc giả lập máy bị quá tải CPU (>=90%) để kiểm tra Rule Engine của B.

📌 📋 PROMPT COPY NHANH CHO AI (LG-A-008)

Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai A).
Tôi đang làm nhiệm vụ LG-A-008 của LabGuard: 'Agent Simulator 10/20/50 socket độc lập'.
Mục tiêu cốt lõi: LabGuard.Simulator.exe.
Ví dụ trực giác: Giống như thuê 20 diễn viên đóng vai 20 máy tính để thử xem máy chủ có chịu nổi không (Agent Simulator 10/20/50 socket độc lập).
Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục Testing.
3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
5. Hướng dẫn tôi cách chạy test 'TEST-A08' để tự mắt thấy kết quả đúng.
6. Nhắc tôi lưu tệp minh chứng: Simulator process monitor, Central connection count screenshot.
7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

📌 🤔 PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)

- Bản chất kỹ thuật của Giả lập N Agent clients, mỗi client có DeviceId, BootId, IP và metrics profile riêng là gì?
- Tại sao cần xử lý tình huống lỗi: Tràn socket hoặc cạn kiệt port ephemeral; tái sử dụng kết nối và quản lý vòng đời socket cẩn thận?
- Vị trí tệp và dòng code minh chứng cụ thể trong Testing?

• Cách chạy test:

Lệnh/Thao tác: Chạy bài test TEST-A08 (Chạy 20 simulated agent kết nối tới Central Server liên tục 30 phút)
Kết quả ĐÚNG: 20 socket duy trì kết nối độc lập; gửi heartbeat 5s và metric 10s không bị đan xen hay deadlock

• Nếu gặp lỗi (3 bước kiểm tra cụ thể):

1. Mở Resource Monitor theo dõi số lượng socket TCP và luồng của tiến trình Simulator. 2. Kiểm tra cổng TCP nội bộ xem có bị nghẽn cổng tạm thời (Ephemeral Port Exhaustion) khi tạo 50 socket không. 3. Cho mỗi socket ảo một deviceId ngẫu nhiên (LAB01-PC01 đến PC50) để không xung đột phiên trên server.

• Minh chứng & Vấn đáp:

Tệp minh chứng: Simulator process monitor, Central connection count screenshot
Câu hỏi thi chính: Mô hình giả lập tải mạng nhiều node trên môi trường thử nghiệm? Mỗi simulated node là một state machine độc lập chạy trên Task riêng; mô phỏng đúng hành vi delay và jitter thực tế

Thẻ #09: [LG-A-009] — RECONNECT BACKOFF VÀ METRIC REPLAY RETRY (Tuần: W2 | P0)

• **Mục tiêu cụ thể:** Hoàn thành ReplayQueue.cs & BackoffPolicy.cs đáp ứng tiêu chuẩn Backoff: 1s, 2s, 4s, 8s, max 30s + jitter 10%; payload.isReplay = true

• **Trực giác đời thường:** *Giống như gọi điện bị bận: Chờ 1s, 2s, 4s... cộng số giây ngẫu nhiên rồi mới gọi lại, và gửi lại mẻ tin chưa commit khi có mạng (Exponential backoff + jitter, replay queue).*

• **Điều kiện:** Cần xong trước: LG-A-006 | Phối hợp: LG-B-005

• **4 Việc cần làm:**
1. Bắt ngoại lệ mất kết nối (SocketException, IOException).
2. Tính thời gian chờ: delay = Min(30s, baseDelay * 2^attempt) + RandomJitter(-10%, +10%).
3. Trong lúc mất mạng, lưu tạm tối đa 50 mẻ telemetry vào hàng đợi RAM cục bộ (không lưu heartbeat vì heartbeat cũ vô giá trị).
4. Khi kết nối lại thành công: gửi lại các mẻ telemetry cũ với cờ isReplay = true để Central Server biết và không kích hoạt cảnh báo sai mốc thời gian.

> 📌 📋 **PROMPT COPY NHANH CHO AI (LG-A-009)**
> Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai A).
> Tôi đang làm nhiệm vụ LG-A-009 của LabGuard: 'Reconnect backoff và metric replay retry'.
> Mục tiêu cốt lõi: ReplayQueue.cs & BackoffPolicy.cs.
> Ví dụ trực giác: Giống như gọi điện bị bận: Chờ 1s, 2s, 4s... cộng số giây ngẫu nhiên rồi mới gọi lại, và gửi lại mẻ tin chưa commit khi có mạng (Exponential backoff + jitter, replay queue).
> Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
> 1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
> 2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục Networking.
> 3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
> 4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
> 5. Hướng dẫn tôi cách chạy test 'TEST-A09' để tự mắt thấy kết quả đúng.
> 6. Nhắc tôi lưu tệp minh chứng: Replay queue dump log, reconnect timeline capture.
> 7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

> 📌 🤔 **PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)**
> - Bản chất kỹ thuật của Backoff: 1s, 2s, 4s, 8s, max 30s + jitter 10%; payload.isReplay = true là gì?
> - Tại sao cần xử lý tình huống lỗi: Reconnect storm khi server restart; Jitter phân tán thời điểm kết nối giúp giảm xung đột?
> - Vị trí tệp và dòng code minh chứng cụ thể trong Networking?

• **Cách chạy test:**
Lệnh/Thao tác: Chạy bài test TEST-A09 (Ngắt mạng 30s rồi khôi phục; kiểm tra số mẻ replay được gửi lên

Central)
Kết quả ĐÚNG: Tự động kết nối lại theo backoff; replay các mẻ telemetry chưa ACK với isReplay=true; không replay heartbeat

• Nếu gặp lỗi (3 bước kiểm tra cụ thể):
1. Ghi log giá trị thời gian delay sau mỗi lần reconnect thất bại để kiểm chứng công thức 2^attempt * 1s + jitter 10% [CP2 TO VERIFY]. 2. Đảm bảo giới hạn trần backoff tối đa 30s [CP2 TO VERIFY] để Agent không chờ quá lâu khi mạng đã phục hồi. 3. Khi kết nối lại thành công, xả các gói tin trong replay buffer (tối đa 50 batches [CP2 TO VERIFY]) theo đúng thứ tự FIFO.

• Minh chứng & Vấn đáp:
Tệp minh chứng: Replay queue dump log, reconnect timeline capture
Câu hỏi thi chính: Thuật toán Exponential Backoff kèm Jitter và hàng đợi Replay? Nhân đôi thời gian chờ sau mỗi lần thử kèm độ lệch ngẫu nhiên để tránh hiện tượng mọi client đồng loạt kết nối lại cùng lúc

Thẻ #10: [LG-A-010] — FAILURE INJECTION VÀ COUNTERS (Tuần: W2 | P0)

• Mục tiêu cụ thể: Hoàn thành NetworkFailureInjector.cs & DiagnosticsCounters đáp ứng tiêu chuẩn IFailureInjector: InjectDrop(rate), InjectDelay(ms), InjectDisconnect()

• Trực giác đời thường: *Giống như cố tình rút dây điện hoặc cắt mạng trong lúc thử nghiệm để xem hệ thống xử lý sự cố thế nào (Failure injection & counters).*

• Điều kiện: Cần xong trước: LG-A-009 | Phối hợp: None

• 4 Việc cần làm:
1. Thêm cờ cấu hình --inject-faults vào Agent hoặc Simulator.
2. Hỗ trợ các kịch bản: ngắt kết nối đột ngột ngay sau khi gửi 2 byte header, gửi frame vượt 256 KiB, gửi chuỗi JSON hỏng cú pháp.
3. Ghi nhận nhật ký: Agent và Central Server phải xử lý ngoại lệ trơn tru, không crash tiến trình, đóng socket lỗi và giải phóng buffer.
4. Xuất log đo kiểm phục vụ hồ sơ nghiệm thu.

> **PROMPT COPY NHANH CHO AI (LG-A-010)**
> Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai A).
> Tôi đang làm nhiệm vụ LG-A-010 của LabGuard: 'Failure injection và counters'.
> Mục tiêu cốt lõi: NetworkFailureInjector.cs & DiagnosticsCounters.
> Ví dụ trực giác: Giống như cố tình rút dây điện hoặc cắt mạng trong lúc thử nghiệm để xem hệ thống xử lý sự cố thế nào (Failure injection & counters).
> Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
> 1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
> 2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục Testing.
> 3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
> 4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).

5. Hướng dẫn tôi cách chạy test 'TEST-A10' để tự mắt thấy kết quả đúng.
6. Nhắc tôi lưu tệp minh chứng: Failure injection log, counter report.
7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

📌 🧠 PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)
- Bản chất kỹ thuật của IFailureInjector: InjectDrop(rate), InjectDelay(ms), InjectDisconnect() là gì?
- Tại sao cần xử lý tình huống lỗi: Ngoại lệ SocketException không được bắt; bọc try-catch toàn diện tại tầng I/O và ghi nhận counter?
- Vị trí tệp và dòng code minh chứng cụ thể trong Testing?

• **Cách chạy test:**
Lệnh/Thao tác: Chạy bài test TEST-A10 (Inject ngắt kết nối giữa chừng lúc đang gửi frame lớn; theo dõi phản ứng của Agent)
Kết quả ĐÚNG: Agent phát hiện lỗi socket, kích hoạt backoff kết nối lại, đếm đúng số sự kiện ngắt và không crash tiến trình

• **Nếu gặp lỗi (3 bước kiểm tra cụ thể):**
1. Kiểm tra cờ IsFaultInjected trên Agent: xác nhận gói tin bị drop hoặc trễ đúng theo kịch bản thử nghiệm. 2. Ghi nhận số liệu counter vào biến thống kê để so sánh tỷ lệ mất gói giả lập với kết quả trên server. 3. Tắt cờ giả lập lỗi và kiểm tra xem hệ thống có tự động gửi bù số liệu tồn đọng không.

• **Minh chứng & Vấn đáp:**
Tệp minh chứng: Failure injection log, counter report
Câu hỏi thi chính: Kiểm thử khả năng phục hồi lỗi mạng (Chaos / Resilience Testing)? Chủ động tạo lỗi giả định giúp xác minh mã nguồn tự khôi phục đúng thiết kế trước khi đưa vào môi trường thật

Thẻ #11: [LG-A-011] — LOG XOAY VÒNG VÀ EVIDENCE HỖ TRỢ DEBUG (Tuần: W3 | P0)

• **Mục tiêu cụ thể:** Hoàn thành RollingLogger.cs đáp ứng tiêu chuẩn MaxFileSize = 10MB, MaxFiles = 5, Async file append

• **Trực giác đời thường:** *Giống camera hành trình tự xóa video cũ khi bộ nhớ đầy (Rolling file log: tối đa 10 MB/tệp, tối đa 5 tệp, tổng dung lượng log <= 50 MB).*

• **Điều kiện:** Cần xong trước: LG-A-006 | Phối hợp: None

• **4 Việc cần làm:**
1. Cấu hình thư viện logging (như Serilog hoặc viết RollingFileSink đơn giản).
2. Thiết lập kích thước tối đa 10 MB cho mỗi tệp log; số tệp tối đa lưu trữ là 5 (tổng dung lượng <= 50 MB) [CP2 TO VERIFY].

3. Định dạng log chuẩn: [Timestamp UTC] [Level] [DeviceId] Message.
4. Đảm bảo thao tác ghi log không làm chậm luồng mạng chính (dùng background log channel).

📌 📋 PROMPT COPY NHANH CHO AI (LG-A-011)

Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai A).
Tôi đang làm nhiệm vụ LG-A-011 của LabGuard: 'Log xoay vòng và evidence hỗ trợ debug'.
Mục tiêu cốt lõi: RollingLogger.cs.
Ví dụ trực giác: Giống camera hành trình tự xóa video cũ khi bộ nhớ đầy (Rolling file log: tối đa 10 MB/tệp, tối đa 5 tệp, tổng dung lượng log <= 50 MB).
ĐẶC BIỆT LƯU Ý: Cấu hình rolling file log chuẩn: MaxFileSize = 10 MB [CP2 TO VERIFY], MaxFiles = 5 [CP2 TO VERIFY], tổng dung lượng log <= 50 MB [CP2 TO VERIFY].
Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục System.
3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
5. Hướng dẫn tôi cách chạy test 'TEST-A11' để tự mắt thấy kết quả đúng.
6. Nhắc tôi lưu tệp minh chứng: Thư mục log mẫu chứa các file xoay vòng.
7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

📌 🧠 PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)

- Bản chất kỹ thuật của MaxFileSize = 10MB, MaxFiles = 5, Async file append là gì?
- Tại sao cần xử lý tình huống lỗi: Nghẽn I/O đĩa làm chậm luồng mạng; ghi log bất đồng bộ qua hàng đợi riêng biệt?
- Vị trí tệp và dòng code minh chứng cụ thể trong System?

• Cách chạy test:

Lệnh/Thao tác: Chạy bài test TEST-A11 (Bơm liên tục 50MB log kiểm tra xoay vòng file)
Kết quả ĐÚNG: Thư mục log chứa tối đa 5 file log, mỗi file tối đa 10 MB, tổng dung lượng log <= 50 MB [CP2 TO VERIFY].

• Nếu gặp lỗi (3 bước kiểm tra cụ thể):

1. Kiểm tra cấu hình log: MaxFileSize = 10 MB [CP2 TO VERIFY], MaxFiles = 5 [CP2 TO VERIFY], tổng dung lượng thư mục log <= 50 MB [CP2 TO VERIFY]. 2. Dùng tiến trình ghi file liên tục để kích hoạt cơ chế xoay vòng: file cũ nhất phải bị xóa khi vượt 5 files. 3. Đảm bảo StreamWriter dùng cờ FileShare.ReadWrite để không khóa chết file log khi đang chạy.

• Minh chứng & Vấn đáp:

Tệp minh chứng: Thư mục log mẫu chứa các file xoay vòng
Câu hỏi thi chính: Chiến lược lưu log xoay vòng trong ứng dụng giám sát? Giới hạn dung lượng và số file để tránh làm cạn kiệt ổ cứng máy trạm trong các đợt phát sinh lỗi liên tục

Thẻ #12: [LG-A-012] — MVP AGENT END-TO-END TRÊN HAI MÁY THẬT LAN (Tuần: W3 | P0)

• **Mục tiêu cụ thể:** Hoàn thành Script cài đặt & Báo cáo đo kiểm LAN đáp ứng tiêu chuẩn TCP/TLS qua địa chỉ IP LAN thật (192.168.x.x)

• **Trực giác đời thường:** *Giống như thử chạy xe trên đường trường thật thay vì chỉ nổ máy trong gara (MVP Agent end-to-end trên hai máy thật trong mạng LAN).*

• **Điều kiện:** Cần xong trước: LG-A-006, LG-A-007 | Phối hợp: LG-B-005, LG-C-007

• **4 Việc cần làm:**
1. Cài đặt Agent lên 2 máy tính vật lý trong phòng máy (hoặc 2 laptop cá nhân kết nối chung mạng hotspot).
2. Cấu hình địa chỉ IP LAN của máy Central Server (máy sinh viên B).
3. Mở cổng TCP 5000 trên Windows Firewall của máy B: netsh advfirewall firewall add rule ...
4. Chạy Agent liên tục 1 giờ: quan sát cả 2 máy hiển thị trạng thái Online trên Dashboard của sinh viên C.

> 📌 📋 **PROMPT COPY NHANH CHO AI (LG-A-012)**
> Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai A).
> Tôi đang làm nhiệm vụ LG-A-012 của LabGuard: 'MVP Agent end-to-end trên hai máy thật LAN'.
> Mục tiêu cốt lõi: Script cài đặt & Báo cáo đo kiểm LAN.
> Ví dụ trực giác: Giống như thử chạy xe trên đường trường thật thay vì chỉ nổ máy trong gara (MVP Agent end-to-end trên hai máy thật trong mạng LAN).
> Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
> 1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
> 2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục Integration.
> 3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
> 4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
> 5. Hướng dẫn tôi cách chạy test 'TEST-A12' để tự mắt thấy kết quả đúng.
> 6. Nhắc tôi lưu tệp minh chứng: Ảnh chụp màn hình 2 máy thật gửi tin, log kết nối LAN.
> 7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

> 📌 🧠 **PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)**
> - Bản chất kỹ thuật của TCP/TLS qua địa chỉ IP LAN thật (192.168.x.x) là gì?
> - Tại sao cần xử lý tình huống lỗi: Windows Defender Firewall chặn port; thêm rule cho phép inbound/outbound TCP port tương ứng?
> - Vị trí tệp và dòng code minh chứng cụ thể trong Integration?

• **Cách chạy test:**
Lệnh/Thao tác: Chạy bài test TEST-A12 (Chạy đồng thời 2 Agent máy thật liên tục 1 giờ trong giờ thực

hành)
Kết quả ĐÚNG: Cả 2 máy hiển thị trạng thái Online trên Dashboard của sinh viên C; dữ liệu metric cập nhật đều đặn

• **Nếu gặp lỗi (3 bước kiểm tra cụ thể):**
1. Dùng lệnh 'ping <Server_IP>' từ máy Agent sang máy Server để chắc chắn thông mạng LAN vật lý. 2. Kiểm tra Windows Firewall trên máy Server đã mở cổng 5000 (TCP) chưa bằng lệnh 'netstat -an | findstr 5000'. 3. Đổi cấu hình ServerIp trong appsettings.json của Agent trỏ đúng địa chỉ IP máy Server thay vì localhost.

• **Minh chứng & Vấn đáp:**
Tệp minh chứng: Ảnh chụp màn hình 2 máy thật gửi tin, log kết nối LAN
Câu hỏi thi chính: Thực tế kiểm thử mạng LAN phòng máy so với chạy localhost? Localhost bỏ qua tường lửa, trễ mạng và phân mảnh gói tin thực tế; test LAN kiểm chứng đúng giao thức mạng

Thẻ #13: [LG-A-013] — GATEWAY LEAST-CONNECTIONS ADMISSION (Tuần: W4 | P1)

• **Mục tiêu cụ thể:** Hoàn thành LabGuard.Gateway.exe & LoadBalancerCore.cs đáp ứng tiêu chuẩn TCP proxy L4, thuật toán Least-Connections trên danh sách backend Collector

• **Trực giác đời thường:** *Giống như trạm phân luồng: Xe đến trạm thì trạm chuyển hướng sang làn có ít xe đang chờ nhất (Gateway least-connections admission [P1]).*

• **Điều kiện:** Cần xong trước: M4 Gate GO, LG-A-012 | Phối hợp: LG-C-013

• **4 Việc cần làm:**
1. Gateway lắng nghe kết nối client trên cổng công khai 5000.
2. Duy trì bảng theo dõi trạng thái các node Collector: {collectorId, address, port, activeConnections, isHealthy}.
3. Khi có kết nối mới: chọn node Collector có isHealthy = true và activeConnections nhỏ nhất.
4. Mở kết nối nội bộ tới Collector đó và tăng bộ đếm active connections.

> 📌 📋 **PROMPT COPY NHANH CHO AI (LG-A-013)**
> Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai A).
> Tôi đang làm nhiệm vụ LG-A-013 của LabGuard: 'Gateway least-connections admission'.
> Mục tiêu cốt lõi: LabGuard.Gateway.exe & LoadBalancerCore.cs.
> Ví dụ trực giác: Giống như trạm phân luồng: Xe đến trạm thì trạm chuyển hướng sang làn có ít xe đang chờ nhất (Gateway least-connections admission [P1]).
> Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
> 1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
> 2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục Networking.
> 3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
> 4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).

5. Hướng dẫn tôi cách chạy test 'TEST-A13' để tự mắt thấy kết quả đúng.
6. Nhắc tôi lưu tệp minh chứng: Gateway connection metrics log, bảng phân bổ tải.
7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

📌 🧠 PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)
- Bản chất kỹ thuật của TCP proxy L4, thuật toán Least-Connections trên danh sách backend Collector là gì?
- Tại sao cần xử lý tình huống lỗi: Collector bị đầy kết nối; từ chối kết nối hoặc giữ trong hàng đợi chờ với timeout quy định?
- Vị trí tệp và dòng code minh chứng cụ thể trong Networking?

• **Cách chạy test:**
Lệnh/Thao tác: Chạy bài test TEST-A13 (Mở lần lượt 10 kết nối mới qua Gateway; kiểm tra số kết nối phân bổ về 2 Collector)
Kết quả ĐÚNG: Phân bổ đều 5-5 giữa 2 Collector; chuyển kết nối mới về node ít tải hơn khi có sự chênh lệch

• **Nếu gặp lỗi (3 bước kiểm tra cụ thể):**
1. [P1] Kiểm tra bảng đếm số kết nối hoạt động của từng Collector tại Gateway: thuật toán least-connections phải chọn Collector có tải thấp nhất. 2. Kiểm tra timeout kết nối Gateway -> Collector: nếu Collector không phản hồi trong 3s thì chuyển sang Collector kế tiếp. 3. Bắt SocketException tại Gateway và trả mã lỗi thích hợp về Agent nếu tất cả Collector đều sập.

• **Minh chứng & Vấn đáp:**
Tệp minh chứng: Gateway connection metrics log, bảng phân bổ tải
Câu hỏi thi chính: Thuật toán cân bằng tải Least-Connections tầng giao vận (L4)? Theo dõi số lượng kết nối đang hoạt động trên từng backend để định tuyến kết nối mới vào node rảnh nhất

Thẻ #14: [LG-A-014] — GATEWAY DUPLEX RELAY VÀ HEALTH POLLING (Tuần: W4 | P1)

• **Mục tiêu cụ thể:** Hoàn thành DuplexRelay.cs & HealthPoller.cs đáp ứng tiêu chuẩn Stream.CopyToAsync hai chiều, Health Probe TCP chu kỳ 2s (3 fail / 2 success)

• **Trực giác đời thường:** *Giống như hai người nói chuyện qua người phiên dịch chuyển tiếp hai chiều và liên tục hỏi 'bạn có khỏe không' (Gateway duplex relay và health polling [P1]).*

• **Điều kiện:** Cần xong trước: LG-A-013 | Phối hợp: LG-C-015

• **4 Việc cần làm:**
1. Sử dụng 2 Task chạy song song: clientStream.CopyToAsync(collectorStream) và collectorStream.CopyToAsync(clientStream).

2. Sử dụng Task.WhenAny để khi 1 trong 2 đầu ngắt kết nối thì đóng đầu còn lại an toàn.
3. Chạy luồng nền định kỳ theo HealthProbeInterval [CP2 TO VERIFY] (đề xuất 5s) gửi bản tin nội bộ HEALTH_PROBE tới từng Collector; nếu không trả lời quá HealthFailThreshold [CP2 TO VERIFY] (đề xuất 3 chu kỳ) thì đánh dấu node bị Unhealthy/NotReady.
4. Cập nhật số kết nối active khi phiên kết thúc.

📌 📋 PROMPT COPY NHANH CHO AI (LG-A-014)

Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai A).
Tôi đang làm nhiệm vụ LG-A-014 của LabGuard: 'Gateway duplex relay và health polling'.
Mục tiêu cốt lõi: DuplexRelay.cs & HealthPoller.cs.
Ví dụ trực giác: Giống như hai người nói chuyện qua người phiên dịch chuyển tiếp hai chiều và liên tục hỏi 'bạn có khỏe không' (Gateway duplex relay và health polling [P1]).
ĐẶC BIỆT LƯU Ý: Health Probe sử dụng các cấu hình HealthProbeInterval [CP2 TO VERIFY] (đề xuất 5s), HealthFailThreshold [CP2 TO VERIFY] (đề xuất 3 chu kỳ liên tiếp), HealthRecoveryThreshold [CP2 TO VERIFY] (đề xuất 2 chu kỳ).
Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục Networking.
3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
5. Hướng dẫn tôi cách chạy test 'TEST-A14' để tự mắt thấy kết quả đúng.
6. Nhắc tôi lưu tệp minh chứng: Duplex relay throughput log, health probe timeline.
7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

📌 🤔 PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)

- Tại sao cần bẫy Task.WhenAny cho 2 luồng CopyToAsync hai chiều để xử lý half-closed socket?
- Ý nghĩa của cấu hình HealthProbeInterval và HealthFailThreshold [CP2 TO VERIFY] trong việc phòng ngừa flapping?
- Khi Gateway đánh dấu một Collector là NotReady, luồng tiếp nhận kết nối Agent mới sẽ chuyển hướng ra sao?

• Cách chạy test:
Lệnh/Thao tác: Chạy bài test TEST-A14 (Tắt đột ngột 1 Collector; đo thời gian Gateway phát hiện và dừng chuyển tiếp)
Kết quả ĐÚNG: Phát hiện Collector chết sau HealthFailThreshold [CP2 TO VERIFY] lần probe lỗi liên tiếp (đề xuất 3 chu kỳ liên tiếp [CP2 TO VERIFY]); đóng socket kết nối phía Agent để kích hoạt reconnect.

• Nếu gặp lỗi (3 bước kiểm tra cụ thể):
1. [P1] Dùng Wireshark kiểm tra bản tin FORWARD_DATA xem đã bọc đúng payload của Agent chưa. 2. Kiểm tra định kỳ gửi HEALTH_PROBE từ Gateway tới Collector theo chu kỳ HealthProbeInterval [CP2 TO VERIFY] (đề xuất 5s) và nhận lại HEALTH_RESULT. 3. Nếu Collector không phản hồi quá

HealthFailThreshold [CP2 TO VERIFY] (đề xuất 3 chu kỳ probe liên tiếp), Gateway lập tức đánh dấu Collector là NotReady/Unhealthy.

• **Minh chứng & Vấn đáp:**
Tệp minh chứng: Duplex relay throughput log, health probe timeline
Câu hỏi thi chính: Chuyển tiếp byte hai chiều bất đồng bộ và xử lý half-closed socket? Dùng 2 Task CopyToAsync đồng thời; khi 1 Task hoàn thành (do ngắt kết nối), lập tức hủy Task còn lại và đóng socket

Thẻ #15: [LG-A-015] — LB INTEGRATION VÀ COLLECTOR KILL FAILOVER PROOF (Tuần: W5 | P1)

• **Mục tiêu cụ thể:** Hoàn thành Báo cáo failover & Raw metrics capture đáp ứng tiêu chuẩn Gateway + 2 Collector instances + Central Server

• **Trực giác đời thường:** *Giống như cắt cầu đột ngột để kiểm tra xe có tự động rẽ sang đường tránh mà không bị tắc nghẽn không (LB integration và collector kill failover proof [P1]).*

• **Điều kiện:** Cần xong trước: LG-A-014 | Phối hợp: LG-C-015, LG-B-015

• **4 Việc cần làm:**
1. Khởi động Gateway và 2 instance Collector (Node 1 port 5001, Node 2 port 5002) cùng kết nối về Central Server.
2. Bật 10 simulated agent qua Gateway; xác nhận các kết nối được chia đều cho cả 2 node.
3. Dùng lệnh taskkill /F cưỡng bức dừng tiến trình Collector Node 1.
4. Ghi nhận nhật ký: Gateway phát hiện probe fail, đánh dấu Node 1 chết; các Agent bị ngắt kích hoạt Reconnect Backoff và được Gateway chuyển tiếp sang Node 2; hệ thống tiếp tục hoạt động bình thường.

> 📌 📋 **PROMPT COPY NHANH CHO AI (LG-A-015)**
> Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai A).
> Tôi đang làm nhiệm vụ LG-A-015 của LabGuard: 'LB integration và collector kill failover proof'.
> Mục tiêu cốt lõi: Báo cáo failover & Raw metrics capture.
> Ví dụ trực giác: Giống như cắt cầu đột ngột để kiểm tra xe có tự động rẽ sang đường tránh mà không bị tắc nghẽn không (LB integration và collector kill failover proof [P1]).
> Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
> 1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
> 2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục Testing.
> 3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
> 4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
> 5. Hướng dẫn tôi cách chạy test 'TEST-A15' để tự mắt thấy kết quả đúng.
> 6. Nhắc tôi lưu tệp minh chứng: Failover latency histogram, video quay lại quá trình kill tiến trình.
> 7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

> 📌 🧠 **PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)**
> - Bản chất kỹ thuật của Gateway + 2 Collector instances + Central Server là gì?
> - Tại sao cần xử lý tình huống lỗi: Mất dữ liệu in-flight; Agent replay lại mẻ tin chưa có ACK, Central khử trùng lặp qua ProcessedBatches?
> - Vị trí tệp và dòng code minh chứng cụ thể trong Testing?

• Cách chạy test:
Lệnh/Thao tác: Chạy bài test TEST-A15 (Kill tiến trình Collector 01 khi có 20 agent đang kết nối; đo thời gian tái kết nối p95)
Kết quả ĐÚNG: Thời gian tái kết nối p95 <= 45s; Agent chuyển sang Collector 02; không thất thoát dữ liệu do có replay queue

• Nếu gặp lỗi (3 bước kiểm tra cụ thể):
1. [P1] Dùng Task Manager cưỡng bức tắt tiến trình Collector 1 để quan sát hành vi chuyển vùng (failover). 2. Kiểm tra Gateway có tự động chuyển hướng các Agent mới sang Collector 2 không. 3. Kiểm tra Agent có tự động kết nối lại qua Gateway mà không bị crash ứng dụng không.

• Minh chứng & Vấn đáp:
Tệp minh chứng: Failover latency histogram, video quay lại quá trình kill tiến trình
Câu hỏi thi chính: Hiện tượng Failover trong hệ thống phân tán và bảo toàn dữ liệu? Hệ thống chấp nhận gián đoạn tạm thời nhưng bảo toàn dữ liệu nhờ kết hợp hàng đợi replay phía client và dedup phía server

Thẻ #16: [LG-A-016] — INTERNET DEPLOYMENT CHUẨN VÀ FALLBACK (Tuần: W5 | P0/Release)

• Mục tiêu cụ thể: Hoàn thành Topology.md & Runbook triển khai Internet đáp ứng tiêu chuẩn TLS connection qua Public Domain / DNS / DDNS / Tunnel

• Trực giác đời thường: *Giống như chạy xe trên đường liên tỉnh và có sẵn đường tránh dự phòng khi sạt lở (Internet deployment chuẩn và fallback Tailscale/Cloudflare tunnel).*

• Điều kiện: Cần xong trước: LG-A-012 | Phối hợp: LG-B-003

• 4 Việc cần làm:
1. Thiết lập đường hầm TCP cho cổng 5000 bằng công cụ tunneling hoặc cấu hình mở cổng router nếu có IP tĩnh.
2. Cấu hình Agent kết nối qua domain/port công khai của đường hầm.
3. Kiểm tra bắt tay TLS 1.3 qua Internet.
4. Chuẩn bị sẵn kịch bản Fallback: nếu Wi-Fi phòng lab trường UIT chặn cổng hoặc mất mạng Internet, lập tức bật mạng phát Wi-Fi từ điện thoại di động (Hotspot) để chuyển về LAN cục bộ trong 2 phút.

> 📌 📋 **PROMPT COPY NHANH CHO AI (LG-A-016)**

> Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai A).
> Tôi đang làm nhiệm vụ LG-A-016 của LabGuard: 'Internet deployment chuẩn và fallback'.
> Mục tiêu cốt lõi: Topology.md & Runbook triển khai Internet.
> Ví dụ trực giác: Giống như chạy xe trên đường liên tỉnh và có sẵn đường tránh dự phòng khi sạt lở (Internet deployment chuẩn và fallback Tailscale/Cloudflare tunnel).
> Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
> 1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
> 2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục System.
> 3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
> 4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
> 5. Hướng dẫn tôi cách chạy test 'TEST-A16' để tự mắt thấy kết quả đúng.
> 6. Nhắc tôi lưu tệp minh chứng: Tracert log, sơ đồ topology mạng, ảnh chụp kết nối 4G.
> 7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

> 📌 🧠 **PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)**
> - Bản chất kỹ thuật của TLS connection qua Public Domain / DNS / DDNS / Tunnel là gì?
> - Tại sao cần xử lý tình huống lỗi: CGNAT nhà mạng chặn Port Forwarding; fallback sang giải pháp Tunnel bảo mật?
> - Vị trí tệp và dòng code minh chứng cụ thể trong System?

• **Cách chạy test:**
Lệnh/Thao tác: Chạy bài test TEST-A16 (Kết nối Agent từ mạng 4G điện thoại vào Central Server đặt tại mạng nhà/trường)
Kết quả ĐÚNG: Bắt tay TLS thành công qua Internet; heartbeat duy trì ổn định không rớt gói bất thường

• **Nếu gặp lỗi (3 bước kiểm tra cụ thể):**
1. Kiểm tra cấu hình Tailscale hoặc Cloudflare Tunnel: lệnh 'tailscale status' phải hiển thị máy chủ đang online. 2. Kiểm tra độ trễ mạng qua Internet (ping RTT): nếu RTT > 150ms thì tăng timeout socket lên 10s. 3. Kiểm tra chứng chỉ TLS khi chạy qua Internet để tránh lỗi RemoteCertificateNameMismatch.

• **Minh chứng & Vấn đáp:**
Tệp minh chứng: Tracert log, sơ đồ topology mạng, ảnh chụp kết nối 4G
Câu hỏi thi chính: Kỹ thuật định tuyến kết nối từ Internet qua NAT và Firewall? Trình bày sự khác biệt giữa Port Forwarding router thông thường và Tunnel bảo mật khi gặp hạ tầng CGNAT

Thẻ #17: [LG-A-017] — LOAD TEST VÀ RECONNECT STORM (Tuần: W5 | P0/Release)

• **Mục tiêu cụ thể:** Hoàn thành LoadStormReport.md & System counters đáp ứng tiêu chuẩn 50 concurrent TCP sockets, heartbeat 5s, telemetry 10s

• **Trực giác đời thường:** *Giống như hàng trăm người cùng ùa vào cổng soát vé sau cơn mưa để xem cổng có bị kẹt cứng không (Load test và reconnect storm).*

• **Điều kiện:** Cần xong trước: LG-A-008 | Phối hợp: LG-B-010

• **4 Việc cần làm:**

1. Cấu hình Simulator tạo 50 kết nối cùng lúc trong vòng 5 giây.
2. Quan sát biểu đồ tài nguyên của Central Server: CPU không bị treo 100%, bộ nhớ RAM không tăng đột biến không giới hạn.
3. Kiểm tra hàng đợi Ingress Channel của Central Server có kích hoạt Backpressure an toàn hay không.
4. Ghi nhận thời gian để toàn bộ 50 máy ổn định trạng thái Online trên Dashboard.

📌 📋 PROMPT COPY NHANH CHO AI (LG-A-017)

Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai A).
Tôi đang làm nhiệm vụ LG-A-017 của LabGuard: 'Load test và reconnect storm'.
Mục tiêu cốt lõi: LoadStormReport.md & System counters.
Ví dụ trực giác: Giống như hàng trăm người cùng ùa vào cổng soát vé sau cơn mưa để xem cổng có bị kẹt cứng không (Load test và reconnect storm).
Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục Testing.
3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
5. Hướng dẫn tôi cách chạy test 'TEST-A17' để tự mắt thấy kết quả đúng.
6. Nhắc tôi lưu tệp minh chứng: Biểu đồ tải CPU/RAM server, reconnect storm log.
7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

📌 🧠 PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)

- Bản chất kỹ thuật của 50 concurrent TCP sockets, heartbeat 5s, telemetry 10s là gì?
- Tại sao cần xử lý tình huống lỗi: Bão kết nối làm cạn ThreadPool; ThreadPool tự điều hòa nhờ cấu trúc Bounded Queue và async I/O?
- Vị trí tệp và dòng code minh chứng cụ thể trong Testing?

• **Cách chạy test:**

Lệnh/Thao tác: Chạy bài test TEST-A17 (Khởi động lại Central trong khi 50 agent đang chạy; quan sát biểu đồ CPU và RAM)
Kết quả ĐÚNG: Toàn bộ 50 agent kết nối lại thành công trong vòng 60s; server không bị tràn bộ nhớ hay treo CPU

• **Nếu gặp lỗi (3 bước kiểm tra cụ thể):**

1. Chạy kịch bản ngắt mạng 50 Agent ảo đồng thời và khôi phục sau 10s: quan sát CPU của máy chủ. 2. Kiểm tra xem các Agent có gửi dồn dập cùng 1 giây không: nếu có, kiểm tra lại thành phần ngẫu nhiên

jitter trong backoff. 3. Đảm bảo server không bị tràn bộ đệm Channel và hàng đợi BoundedChannel giữ đúng ngưỡng an toàn.

• **Minh chứng & Vấn đáp:**
Tệp minh chứng: Biểu đồ tải CPU/RAM server, reconnect storm log
Câu hỏi thi chính: Hiện tượng Reconnect Storm và cách phòng ngừa? Khi hàng loạt client mất kết nối và cùng thử lại đồng thời sẽ gây quá tải máy chủ; giải pháp là Exponential Backoff kèm Jitter ngẫu nhiên

Thẻ #18: [LG-A-018] — THU PROOF LAN INTERNET CHO RELEASE CANDIDATE (Tuần: W5 | P0/Release)

• **Mục tiêu cụ thể:** Hoàn thành Bộ minh chứng thực nghiệm & Capture file (.pcapng) đáp ứng tiêu chuẩn Toàn bộ hệ thống LABGUARD chạy thực tế

• **Trực giác đời thường:** *Giống như quay phim chụp ảnh hiện trường đầy đủ góc máy làm bằng chứng nghiệm thu (Thu proof LAN Internet cho release candidate).*

• **Điều kiện:** Cần xong trước: LG-A-012, LG-A-016 | Phối hợp: None

• **4 Việc cần làm:**
1. Bật Wireshark bắt gói tin trên card mạng: chứng minh gói tin được bọc TLS mã hóa, không thấy plaintext payload.
2. Chụp ảnh màn hình 2 máy thật trong mạng LAN đang gửi heartbeat và metric đều đặn.
3. Lưu tệp log của Agent thể hiện các pha kết nối, ngắt mạng, backoff và reconnect thành công.
4. Đóng gói bằng chứng và đối soát với cột Evidence trong WBS.

> 📌 📋 **PROMPT COPY NHANH CHO AI (LG-A-018)**
> Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai A).
> Tôi đang làm nhiệm vụ LG-A-018 của LabGuard: 'Thu proof LAN Internet cho release candidate'.
> Mục tiêu cốt lõi: Bộ minh chứng thực nghiệm & Capture file (.pcapng).
> Ví dụ trực giác: Giống như quay phim chụp ảnh hiện trường đầy đủ góc máy làm bằng chứng nghiệm thu (Thu proof LAN Internet cho release candidate).
> Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
> 1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
> 2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục Integration.
> 3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
> 4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
> 5. Hướng dẫn tôi cách chạy test 'TEST-A18' để tự mắt thấy kết quả đúng.
> 6. Nhắc tôi lưu tệp minh chứng: File .pcapng, ảnh chụp màn hình Wireshark follow stream.
> 7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

> 📌 🧐 **PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)**

> - Bản chất kỹ thuật của Toàn bộ hệ thống LABGUARD chạy thực tế là gì?
> - Tại sao cần xử lý tình huống lỗi: File pcap thiếu thông tin; bắt gói tin trực tiếp trên card mạng vật lý có lọc IP rõ ràng?
> - Vị trí tệp và dòng code minh chứng cụ thể trong Integration?

• Cách chạy test:
Lệnh/Thao tác: Chạy bài test TEST-A18 (Đối soát chéo dữ liệu packet capture với log hệ thống của Central Server)
Kết quả ĐÚNG: Bằng chứng rõ ràng, thể hiện đúng mốc thời gian, IP thật, gói tin TLS mã hóa và kịch bản chuyển đổi trạng thái

• Nếu gặp lỗi (3 bước kiểm tra cụ thể):
1. Mở Wireshark lưu file capture với bộ lọc 'tcp.port == 5000' thành tệp 'evidence_tls_lan.pcapng'. 2. Kiểm tra file pcap: xác nhận gói tin được mã hóa TLSv1.3 và không chứa chuỗi văn bản JSON lộ thiên. 3. Chụp ảnh màn hình các mốc thời gian bắt gói tin và lưu kèm checksum SHA-256 vào thư mục minh chứng.

• Minh chứng & Vấn đáp:
Tệp minh chứng: File .pcapng, ảnh chụp màn hình Wireshark follow stream
Câu hỏi thi chính: Phương pháp thu thập và phân tích gói tin mạng bằng Wireshark? Đặt bộ lọc capture filter và display filter phù hợp để ghi lại quá trình bắt tay 3 bước TCP, TLS handshake và frame dữ liệu

Thẻ #19: [LG-A-019] — RELEASE SCRIPTS VÀ PACKAGE CHECKSUM (Tuần: W5 | P0/Release)

• Mục tiêu cụ thể: Hoàn thành PackageRelease.ps1 & SHA256SUMS.txt đáp ứng tiêu chuẩn .NET SDK CLI publish -c Release, SHA-256 hash algorithm

• Trực giác đời thường: *Giống như đóng gói sản phẩm vào thùng carton, dán niêm phong mã vạch kiểm định trước khi giao hàng (Release scripts và package checksum SHA-256).*

• Điều kiện: Cần xong trước: LG-A-018 | Phối hợp: LG-B-019, LG-C-019

• 4 Việc cần làm:
1. Viết script PowerShell biên dịch toàn bộ solution với cấu hình Release tối ưu: dotnet publish -c Release -r win-x64 --self-contained false.
2. Thu thập các tệp thực thi (.exe), cấu hình (.json), tài liệu (.pdf) vào một thư mục staging sạch.
3. Tạo tệp MANIFEST_SHA256.txt chứa mã băm SHA-256 và kích thước của từng file.
4. Nén thành tệp ZIP duy nhất; kiểm tra giải nén trên một máy tính sạch khác để đảm bảo chạy được ngay.

> 📌 📋 **PROMPT COPY NHANH CHO AI (LG-A-019)**
> Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai A).

> Tôi đang làm nhiệm vụ LG-A-019 của LabGuard: 'Release scripts và package checksum'.
> Mục tiêu cốt lõi: PackageRelease.ps1 & SHA256SUMS.txt.
> Ví dụ trực giác: Giống như đóng gói sản phẩm vào thùng carton, dán niêm phong mã vạch kiểm định trước khi giao hàng (Release scripts và package checksum SHA-256).
> Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
> 1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
> 2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục System.
> 3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
> 4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
> 5. Hướng dẫn tôi cách chạy test 'TEST-A19' để tự mắt thấy kết quả đúng.
> 6. Nhắc tôi lưu tệp minh chứng: PackageRelease.ps1, MANIFEST_SHA256.txt.
> 7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

> 📌 🤔 **PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)**
> - Bản chất kỹ thuật của .NET SDK CLI publish -c Release, SHA-256 hash algorithm là gì?
> - Tại sao cần xử lý tình huống lỗi: Thiếu runtime dependency; xuất bản ở chế độ self-contained hoặc quy định rõ .NET Runtime yêu cầu?
> - Vị trí tệp và dòng code minh chứng cụ thể trong System?

• **Cách chạy test:**
Lệnh/Thao tác: Chạy bài test TEST-A19 (Chạy script trên máy tính môi trường sạch; giải nén và đối chiếu checksum)
Kết quả ĐÚNG: Tạo gói ZIP hoàn chỉnh chạy được ngay; 100% file có mã băm SHA-256 khớp tuyệt đối

• **Nếu gặp lỗi (3 bước kiểm tra cụ thể):**
1. Chạy script 'build_release.ps1' trong PowerShell: kiểm tra mã thoát ($LASTEXITCODE == 0). 2. Kiểm tra file zip bàn giao: phải chứa đầy đủ binary, file cấu hình mẫu và tài liệu hướng dẫn README. 3. Chạy lệnh 'Get-FileHash -Algorithm SHA256 <file_zip>' và ghi lại mã hash vào file manifest.

• **Minh chứng & Vấn đáp:**
Tệp minh chứng: PackageRelease.ps1, MANIFEST_SHA256.txt
Câu hỏi thi chính: Quy trình đóng gói phần mềm và kiểm tra tính toàn vẹn (Integrity Check)? Dùng mã băm mật mã học SHA-256 để đảm bảo gói cài đặt không bị hỏng hóc hoặc can thiệp trên đường truyền

Thẻ #20: [LG-A-020] — VIVA A VÀ BÀN GIAO BACKUP (Tuần: W5 | P0/Viva)

• **Mục tiêu cụ thể:** Hoàn thành HANDBOOK_A.md & Viva rehearsal log đáp ứng tiêu chuẩn 30 câu hỏi viva chuyên sâu phần Agent, Networking & Gateway

• **Trực giác đời thường:** *Giống như diễn tập bảo vệ đồ án: Nắm chắc cơ chế socket TCP, byte stream và ranh giới bản tin để tự tin trả lời vấn đáp.*

• **Điều kiện:** Cần xong trước: LG-A-019 | Phối hợp: None

• **4 Việc cần làm:**

1. Tự luyện tập trả lời 30 câu hỏi vấn đáp trong handbook; tự chấm điểm và ghi nhận điểm yếu.
2. Tổ chức buổi bảo vệ thử (Mock Viva): thành viên B và C đóng vai giảng viên phản biện, đặt câu hỏi bất kỳ và yêu cầu chỉ code.
3. Rèn luyện phản xạ: nêu định nghĩa -> giải thích cơ chế -> giải thích tại sao LabGuard lại làm vậy -> mở đúng file code minh họa.
4. Đóng gói tài liệu dự phòng (USB, in sẵn đề cương) sẵn sàng cho ngày bảo vệ chính thức.

> 📌 📋 **PROMPT COPY NHANH CHO AI (LG-A-020)**
> Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai A).
> Tôi đang làm nhiệm vụ LG-A-020 của LabGuard: 'Viva A và bàn giao backup'.
> Mục tiêu cốt lõi: HANDBOOK_A.md & Viva rehearsal log.
> Ví dụ trực giác: Giống như diễn tập bảo vệ đồ án: Nắm chắc cơ chế socket TCP, byte stream và ranh giới bản tin để tự tin trả lời vấn đáp.
> Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
> 1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
> 2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục Viva.
> 3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
> 4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
> 5. Hướng dẫn tôi cách chạy test 'TEST-A20' để tự mắt thấy kết quả đúng.
> 6. Nhắc tôi lưu tệp minh chứng: HANDBOOK_A.md, biên bản diễn tập vấn đáp nội bộ.
> 7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

> 📌 🧠 **PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)**
> - Bản chất kỹ thuật của 30 câu hỏi viva chuyên sâu phần Agent, Networking & Gateway là gì?
> - Tại sao cần xử lý tình huống lỗi: Lúng túng khi bị hỏi sâu về socket; ôn tập kỹ bảng thuật ngữ và các kịch bản kiểm thử trong handbook?
> - Vị trí tệp và dòng code minh chứng cụ thể trong Viva?

• **Cách chạy test:**

Lệnh/Thao tác: Chạy bài test TEST-A20 (Vấn đáp chéo giả định với thành viên B và C chấm điểm độc lập)
Kết quả ĐÚNG: Giải thích lưu loát các cơ chế mạng, vẽ được sơ đồ luồng dữ liệu và chỉ ra đúng dòng code trong repo

• **Nếu gặp lỗi (3 bước kiểm tra cụ thể):**

1. Ôn lại sơ đồ đóng gói Length-Prefix: giải thích rõ tại sao 4 byte độ dài giải quyết được vấn đề TCP stream. 2. Mở file FrameCodec.cs chỉ đúng vị trí đọc 4 byte header và vị trí đọc N byte payload. 3. Tự trả lời câu hỏi: 'Nếu bên gửi chỉ gửi 2 byte rồi mất mạng thì hàm đọc xử lý thế nào?'.

- **Minh chứng & Vấn đáp:**

Tệp minh chứng: HANDBOOK_A.md, biên bản diễn tập vấn đáp nội bộ

Câu hỏi thi chính: Tổng quan kiến trúc phân tầng mạng và ranh giới trách nhiệm của Lane A? Trình bày mạch lạc vai trò của Agent thu thập dữ liệu, cơ chế framing, đảm bảo an toàn kết nối và phân tải P1

PHẦN 4: SỔ TAY 20 NHIỆM VỤ CỦA SINH VIÊN B (SERVER & CƠ SỞ DỮ LIỆU)

Sinh viên B phụ trách máy chủ Central Server, hàng đợi có chặn BoundedChannel (1000 items [CP2 TO VERIFY]), khử trùng lặp lô số liệu (ProcessedBatches), cơ sở dữ liệu và khung di trú (khóa chính thức tại CP2), khóa lạc quan OCC (rowversion) và bảo mật PBKDF2. Dưới đây là 20 thẻ nhiệm vụ từ LG-B-001 đến LG-B-020:

Thẻ #01: [LG-B-001] — CHUẨN BỊ SQL SERVER + KẾT NỐI + MIGRATION SKELETON [CHỜ CP2 TẠO BẢNG] (Tuần: W1 | P0)

> 📌 ⛔ **CHỜ CHECKPOINT 2 TRƯỚC KHI TẠO SCHEMA THẬT**
> TẠM THỜI CHƯA CODE FULL DATABASE SCHEMA. Trước Checkpoint 2: cài đặt SQL Server; kiểm tra chuỗi kết nối từ C#; tạo database LabGuard_Dev rỗng; nắm vững khái niệm PK, FK, rowversion; chuẩn bị khung migration skeleton. Danh sách bảng chính thức sẽ được chốt và đóng băng tại Checkpoint 2.

• **Mục tiêu cụ thể:** Trước CP2: cài SQL Server, kiểm tra connection từ C#, tạo LabGuard_Dev rỗng, học PK/FK/rowversion, chuẩn bị migration skeleton. Schema chính thức chốt tại Checkpoint 2.

• **Trực giác đời thường:** *Giống như chuẩn bị mặt bằng và bản vẽ nền móng trước khi đổ bê tông thật ở Checkpoint 2 (Cài SQL Server, kiểm tra kết nối, chuẩn bị migration skeleton; CHỜ CP2 TẠO BẢNG THẬT).*

• **Điều kiện:** Cần xong trước: LG-A-001 | Phối hợp: None

• **4 Việc cần làm:**
1. Cài đặt SQL Server 2022 Developer / Express và SSMS trên máy tính cá nhân.
2. Mở SSMS tạo cơ sở dữ liệu rỗng mang tên LabGuard_Dev và kiểm tra quyền đăng nhập.
3. Viết đoạn code C# nhỏ kiểm tra kết nối (SqlConnection.OpenAsync()) thông suốt.
4. Tạo cấu trúc thư mục Migrations/ và tìm hiểu cú pháp PK, FK, kiểu rowversion trong SQL Server.

> 📌 📋 **PROMPT COPY NHANH CHO AI (LG-B-001)**
> Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai B).
> Tôi đang làm nhiệm vụ LG-B-001 của LabGuard: 'Schema migrations và seed an toàn'.
> Mục tiêu cốt lõi: V1__Initial_Schema.sql & Seed.sql.
> Ví dụ trực giác: Giống như chuẩn bị mặt bằng và bản vẽ nền móng trước khi đổ bê tông thật ở Checkpoint 2 (Cài SQL Server, kiểm tra kết nối, chuẩn bị migration skeleton; CHỜ CP2 TẠO BẢNG THẬT).
> ĐẶC BIỆT LƯU Ý: Đang ở Checkpoint 1. TẠM THỜI CHƯA TẠO FULL DATABASE SCHEMA (danh sách bảng sẽ được khóa ở Checkpoint 2). Chỉ hướng dẫn tôi: cài đặt SQL Server, kiểm tra kết nối chuỗi connection string, tạo database LabGuard_Dev rỗng, giải thích PK, FK, rowversion và chuẩn bị khung migration skeleton.
> Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
> 1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.

2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục Storage.
3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
5. Hướng dẫn tôi cách chạy test 'TEST-B01' để tự mắt thấy kết quả đúng.
6. Nhắc tôi lưu tệp minh chứng: Script SQL migration, sơ đồ quan hệ thực thể ERD.
7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

📌 🧠 **PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)**
- Tại sao không nên tạo bảng vội vã trước khi toàn bộ nhóm chốt xong hợp đồng giao thức ở Checkpoint 2?
- Kiểu dữ liệu rowversion trong SQL Server có ý nghĩa gì khác với DateTime thông thường?
- Cách kiểm tra chuỗi connection string bằng lệnh PowerShell hoặc mã C# cơ bản?

• **Cách chạy test:**
Lệnh/Thao tác: Chạy bài test TEST-B01 (Chạy script migration trên instance SQL Server sạch)
Kết quả ĐÚNG: Cài đặt thành công SQL Server, kết nối thành công từ .NET, tạo được cơ sở dữ liệu LabGuard_Dev, chuẩn bị sẵn cấu trúc thư mục migrations [Chờ CP2 tạo bảng thật].

• **Nếu gặp lỗi (3 bước kiểm tra cụ thể):**
1. Mở SQL Server Configuration Manager kiểm tra dịch vụ MSSQLSERVER / SQLEXPRESS đang Running. 2. Kiểm tra kết nối qua PowerShell: 'Test-NetConnection -ComputerName localhost -Port 1433'. 3. Ghi nhớ cảnh báo: KHÔNG tạo 8 bảng trước Checkpoint 2; chỉ tạo database rỗng LabGuard_Dev và migration skeleton.

• **Minh chứng & Vấn đáp:**
Tệp minh chứng: Script SQL migration, sơ đồ quan hệ thực thể ERD
Câu hỏi thi chính: Kiểu dữ liệu rowversion trong SQL Server và cơ chế OCC? rowversion là kiểu dữ liệu đặc thù của SQL Server tự động tăng khi dòng thay đổi; client coi là token mờ 8-byte kiểm tra tranh chấp

Thẻ #02: [LG-B-002] — SQL REPOSITORIES VÀ TRANSACTION BOUNDARY (Tuần: W1 | P0)

• **Mục tiêu cụ thể:** Hoàn thành IncidentRepository.cs & MetricRepository.cs đáp ứng tiêu chuẩn IDbTransaction, atomic incident creation with audit history

• **Trực giác đời thường:** *Giống như chuyển khoản ngân hàng: Trừ tiền tài khoản A và cộng tiền tài khoản B phải cùng thành công (Transaction), nếu một bước lỗi thì hoàn tiền lại nguyên vẹn (Rollback).*

• **Điều kiện:** Cần xong trước: LG-B-001 | Phối hợp: None

• **4 Việc cần làm:**
1. Viết interface IIncidentRepository và IMetricRepository với các phương thức Async.

2. Hiện thực hàm CreateIncidentWithEventAsync bọc trong SqlConnection.BeginTransactionAsync().
3. Sử dụng tham số hóa hoàn toàn (SqlParameter) để triệt tiêu lỗ hổng SQL Injection.
4. Đảm bảo khối try/catch thực hiện transaction.RollbackAsync() nếu xảy ra bất kỳ ngoại lệ nào.

📌 📋 PROMPT COPY NHANH CHO AI (LG-B-002)

Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai B).
Tôi đang làm nhiệm vụ LG-B-002 của LabGuard: 'SQL repositories và transaction boundary'.
Mục tiêu cốt lõi: IncidentRepository.cs & MetricRepository.cs.
Ví dụ trực giác: Giống như chuyển khoản ngân hàng: Trừ tiền tài khoản A và cộng tiền tài khoản B phải cùng thành công (Transaction), nếu một bước lỗi thì hoàn tiền lại nguyên vẹn (Rollback).
Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục Storage.
3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
5. Hướng dẫn tôi cách chạy test 'TEST-B02' để tự mắt thấy kết quả đúng.
6. Nhắc tôi lưu tệp minh chứng: Test log kiểm chứng rollback thành công.
7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

📌 🧠 PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)

- Bản chất kỹ thuật của IDbTransaction, atomic incident creation with audit history là gì?
- Tại sao cần xử lý tình huống lỗi: Deadlock cơ sở dữ liệu; sắp xếp thứ tự cập nhật bảng nhất quán và dùng command timeout phù hợp?
- Vị trí tệp và dòng code minh chứng cụ thể trong Storage?

• Cách chạy test:

Lệnh/Thao tác: Chạy bài test TEST-B02 (Unit test cố tình throw exception giữa giao dịch; kiểm tra rollback CSDL)
Kết quả ĐÚNG: Giao dịch rollback hoàn toàn, không để lại dữ liệu rác (orphan records)

• Nếu gặp lỗi (3 bước kiểm tra cụ thể):

1. Mở kết nối SqlConnection trong khối 'using' để bảo đảm giải phóng connection pool khi xong việc. 2. Bọc các thao tác ghi dữ liệu phụ thuộc lẫn nhau trong SqlTransaction: gọi transaction.Commit() ở cuối và Rollback() khi bắt Exception. 3. Đảm bảo không để kết nối mở quá lâu trong lúc xử lý tác vụ mạng I/O.

• Minh chứng & Vấn đáp:

Tệp minh chứng: Test log kiểm chứng rollback thành công
Câu hỏi thi chính: Tính nguyên tử (Atomicity) và ranh giới giao dịch trong CSDL? Mọi thao tác ghi liên quan (mở sự cố, ghi lịch sử, cập nhật thiết bị) phải nằm trong 1 Transaction để đảm bảo tính nhất quán

Thẻ #03: [LG-B-003] — CENTRAL ACCEPT TLS VÀ BOUNDED INGRESS (Tuần: W1 | P0)

• **Mục tiêu cụ thể:** Hoàn thành CentralServerDaemon.cs & IngressChannel.cs đáp ứng tiêu chuẩn TcpListener, SslStream (Server cert), BoundedChannelOptions(1000)

• **Trực giác đời thường:** *Giống như cửa soát vé sân bay: Tiếp nhận hàng trăm hành khách nhưng có hàng rào zíc-zắc chặn lại không cho chen lấn (BoundedChannel 1000 items [CP2 TO VERIFY] để backpressure).*

• **Điều kiện:** Cần xong trước: LG-A-002 | Phối hợp: LG-A-003

• **4 Việc cần làm:**
1. Khởi tạo TcpListener trên IPAddress.Any và Port chỉ định, bắt đầu vòng lặp AcceptTcpClientAsync.
2. Thiết lập SslStream xác thực chứng chỉ máy chủ (Server Authentication) an toàn.
3. Khởi tạo System.Threading.Channels.Channel.CreateBounded<WireEnvelope>(options) với FullMode = Wait hoặc DropOldest có ghi log.
4. Với mỗi kết nối client được chấp nhận, bàn giao luồng đọc FrameCodec cho Task độc lập đẩy vào Ingress Channel.

📌 📋 PROMPT COPY NHANH CHO AI (LG-B-003)

Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai B).
Tôi đang làm nhiệm vụ LG-B-003 của LabGuard: 'Central accept TLS và bounded ingress'.
Mục tiêu cốt lõi: CentralServerDaemon.cs & IngressChannel.cs.
Ví dụ trực giác: Giống như cửa soát vé sân bay: Tiếp nhận hàng trăm hành khách nhưng có hàng rào zíc-zắc chặn lại không cho chen lấn (BoundedChannel 1000 items [CP2 TO VERIFY] để backpressure).
Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục Networking.
3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
5. Hướng dẫn tôi cách chạy test 'TEST-B03' để tự mắt thấy kết quả đúng.
6. Nhắc tôi lưu tệp minh chứng: Server socket listen log, ThreadPool performance counters.
7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

📌 🧠 PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)

- Bản chất kỹ thuật của TcpListener, SslStream (Server cert), BoundedChannelOptions(1000) là gì?
- Tại sao cần xử lý tình huống lỗi: Tràn bộ nhớ do quá nhiều kết nối; giới hạn số kết nối tối đa và áp dụng Bounded Channel?
- Vị trí tệp và dòng code minh chứng cụ thể trong Networking?

• **Cách chạy test:**
Lệnh/Thao tác: Chạy bài test TEST-B03 (Bơm đồng thời 50 kết nối TLS thử nghiệm kiểm tra phân bổ luồng

ThreadPool)
Kết quả ĐÚNG: Chấp nhận kết nối bất đồng bộ không tạo 1 thread riêng cho mỗi socket; kênh ingress kích hoạt backpressure khi đầy

• **Nếu gặp lỗi (3 bước kiểm tra cụ thể):**
1. Kiểm tra cấu hình BoundedChannelOptions: Capacity = 1000 [CP2 TO VERIFY], FullMode = BoundedChannelFullMode.Wait để áp dụng backpressure. 2. Dùng netstat kiểm tra cổng 5000: 'netstat -ano | findstr 5000' để xác nhận server đang lắng nghe. 3. Bắt AuthenticationException khi client gửi chứng chỉ không hợp lệ và đóng socket an toàn.

• **Minh chứng & Vấn đáp:**
Tệp minh chứng: Server socket listen log, ThreadPool performance counters
Câu hỏi thi chính: Mô hình I/O bất đồng bộ (Non-blocking I/O) trên máy chủ mạng? Dùng async/await tận dụng I/O Completion Ports của hệ điều hành, giải phóng thread về pool khi chờ dữ liệu socket

Thẻ #04: [LG-B-004] — SESSION REGISTRY, SERVER AUTH PBKDF2 VÀ RBAC (Tuần: W1 | P0)

• **Mục tiêu cụ thể:** Hoàn thành AuthService.cs & SessionStore.cs đáp ứng tiêu chuẩn PBKDF2-HMAC-SHA256 (600k iter, 16B salt, 32B key, FixedTimeEquals), RBAC: Admin, Operator, Viewer

• **Trực giác đời thường:** *Giống như cửa bảo mật quét vân tay: Không lưu mật khẩu thô mà băm nhỏ 600,000 lần, và phân loại thẻ ra vào (Admin toàn quyền, Operator trực ban, Viewer chỉ xem).*

• **Điều kiện:** Cần xong trước: LG-B-001, LG-B-003 | Phối hợp: LG-C-003

• **4 Việc cần làm:**
1. Hiện thực hàm HashPassword(string password): tạo ngẫu nhiên salt 16 byte (RandomNumberGenerator), băm PBKDF2 600,000 vòng với HMAC-SHA256 cho ra subkey 32 byte.
2. Hiện thực VerifyPassword bằng CryptographicOperations.FixedTimeEquals để ngăn chặn kẻ tấn công dò độ dài chuỗi qua thời gian phản hồi.
3. Lưu trữ chuỗi phiên token ngẫu nhiên (UUIDv4) kèm vai trò tương ứng trong ConcurrentDictionary.
4. Kiểm tra phân quyền chặt chẽ: Viewer chỉ đọc; Operator được phép thao tác sự cố (ACK, START, RESOLVE); Admin toàn quyền quản lý người dùng và cấu hình.

> 📌 📋 **PROMPT COPY NHANH CHO AI (LG-B-004)**
> Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai B).
> Tôi đang làm nhiệm vụ LG-B-004 của LabGuard: 'Session registry, server auth PBKDF2 và RBAC'.
> Mục tiêu cốt lõi: AuthService.cs & SessionStore.cs.
> Ví dụ trực giác: Giống như cửa bảo mật quét vân tay: Không lưu mật khẩu thô mà băm nhỏ 600,000 lần, và phân loại thẻ ra vào (Admin toàn quyền, Operator trực ban, Viewer chỉ xem).
> Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
> 1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
> 2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục Security.

3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
5. Hướng dẫn tôi cách chạy test 'TEST-B04' để tự mắt thấy kết quả đúng.
6. Nhắc tôi lưu tệp minh chứng: Bảng dữ liệu Users đã băm mật khẩu, auth security audit log.
7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

📌 🧠 PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)
- Bản chất kỹ thuật của PBKDF2-HMAC-SHA256 (600k iter, 16B salt, 32B key, FixedTimeEquals), RBAC: Admin, Operator, Viewer là gì?
- Tại sao cần xử lý tình huống lỗi: Timing attack khi so sánh chuỗi hash; sử dụng CryptographicOperations.FixedTimeEquals?
- Vị trí tệp và dòng code minh chứng cụ thể trong Security?

• **Cách chạy test:**
Lệnh/Thao tác: Chạy bài test TEST-B04 (Kiểm tra đăng nhập đúng/sai pass; thử nghiệm bypass quyền từ client gửi trực tiếp command)
Kết quả ĐÚNG: Mật khẩu băm an toàn kèm salt; server chặn đứng mọi lệnh không đủ quyền dù client có cố tình gửi

• **Nếu gặp lỗi (3 bước kiểm tra cụ thể):**
1. Kiểm tra tham số băm PBKDF2: 600,000 vòng lặp, salt 16 byte ngẫu nhiên, derived key 32 byte. 2. So sánh mật khẩu bằng CryptographicOperations.FixedTimeEquals để chống tấn công đo thời gian (Timing Attack). 3. Kiểm tra phân quyền RBAC: từ chối lệnh của Viewer với lỗi ERR_UNAUTHORIZED (403).

• **Minh chứng & Vấn đáp:**
Tệp minh chứng: Bảng dữ liệu Users đã băm mật khẩu, auth security audit log
Câu hỏi thi chính: Thuật toán băm mật khẩu PBKDF2 và kiểm soát quyền bắt buộc phía server? PBKDF2 làm chậm quá trình dò vét brute-force; kiểm tra quyền phía server là chốt chặn an ninh bắt buộc vì UI client có thể bị can thiệp

Thẻ #05: [LG-B-005] — METRIC INGESTION, DEDUP PROCESSEDBATCHES, COMMIT ACK (Tuần: W2 | P0)

• **Mục tiêu cụ thể:** Hoàn thành BatchIngestProcessor.cs đáp ứng tiêu chuẩn Table ProcessedBatches (DeviceId, BootId, BatchId, ReceivedUtc), BATCH_ACK

• **Trực giác đời thường:** *Giống như nhận bưu phẩm: Nếu người giao hàng giao lại món đồ đã ký nhận hôm qua thì ký xác nhận lại để họ yên tâm, nhưng không lưu món hàng thừa vào kho (ProcessedBatches TTL 24h, BATCH_ACK).*

• **Điều kiện:** Cần xong trước: LG-B-002, LG-B-003 | Phối hợp: LG-A-006

• 4 Việc cần làm:

1. Kiểm tra sự tồn tại của bộ khóa (DeviceId, BootId, BatchId) trong bảng ProcessedBatches.
2. Nếu đã tồn tại: Bỏ qua bước ghi dữ liệu, gửi ngay BATCH_ACK xác nhận thành công về cho Agent (khử trùng lặp an toàn).
3. Nếu chưa tồn tại: Mở Transaction, ghi nhận lô số liệu vào MetricSnapshots và Processes, đồng thời chèn bản ghi vào ProcessedBatches.
4. Commit Transaction và trả về gói tin BATCH_ACK mang đúng batchId.

📌 📋 PROMPT COPY NHANH CHO AI (LG-B-005)

Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai B).
Tôi đang làm nhiệm vụ LG-B-005 của LabGuard: 'Metric ingestion, dedup ProcessedBatches, commit ACK'.
Mục tiêu cốt lõi: BatchIngestProcessor.cs.
Ví dụ trực giác: Giống như nhận bưu phẩm: Nếu người giao hàng giao lại món đồ đã ký nhận hôm qua thì ký xác nhận lại để họ yên tâm, nhưng không lưu món hàng thừa vào kho (ProcessedBatches TTL 24h, BATCH_ACK).
Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục Storage.
3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
5. Hướng dẫn tôi cách chạy test 'TEST-B05' để tự mắt thấy kết quả đúng.
6. Nhắc tôi lưu tệp minh chứng: Bảng ProcessedBatches SQL Server, log xử lý mẻ tin.
7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

📌 🧠 PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)

- Bản chất kỹ thuật của Table ProcessedBatches (DeviceId, BootId, BatchId, ReceivedUtc), BATCH_ACK là gì?
- Tại sao cần xử lý tình huống lỗi: Client gửi lặp gây sai lệch thống kê; khóa chính bảng ProcessedBatches ngăn chặn triệt để?
- Vị trí tệp và dòng code minh chứng cụ thể trong Storage?

• Cách chạy test:

Lệnh/Thao tác: Chạy bài test TEST-B05 (Gửi 10 mẻ tin trùng lặp liên tiếp; kiểm tra số dòng ghi vào bảng telemetry)
Kết quả ĐÚNG: Chỉ ghi vào bảng telemetry 1 lần duy nhất; luôn gửi lại BATCH_ACK xác nhận cho client

• Nếu gặp lỗi (3 bước kiểm tra cụ thể):

1. Kiểm tra khóa chính bảng ProcessedBatches (DeviceId, BootId, BatchId): nếu đã tồn tại thì bỏ qua ghi mới. 2. BẮT BUỘC gửi lại BATCH_ACK về cho Agent ngay cả khi phát hiện batch trùng lặp để Agent giải phóng replay queue. 3. Ghi log cảnh báo khi nhận mẻ tin trùng lặp để phục vụ đối soát kỹ thuật.

• Minh chứng & Vấn đáp:

Tệp minh chứng: Bảng ProcessedBatches SQL Server, log xử lý mẻ tin

Câu hỏi thi chính: Cơ chế khử trùng lặp bền vững (Deduplication) và Idempotency? Dùng bảng cơ sở dữ liệu có khóa chính tự nhiên để chặn trùng lặp qua các đợt restart, không phụ thuộc vào bộ nhớ RAM tạm

Thẻ #06: [LG-B-006] — LIVENESS REGISTRY VÀ MONOTONIC DEADLINE EVALUATOR (Tuần: W2 | P0)

• **Mục tiêu cụ thể:** Hoàn thành LivenessRegistry.cs đáp ứng tiêu chuẩn Liveness threshold: Missed 3 intervals (15s) + 5s grace = 20s offline deadline

• **Trực giác đời thường:** *Giống như đồng hồ bấm giờ độc lập: Quá 20 giây không nghe tiếng chuông điểm danh từ Agent thì máy chủ bấm còi báo vắng mặt, không phụ thuộc vào giờ đồng hồ của máy trạm (Stopwatch monotonic clock).*

• **Điều kiện:** Cần xong trước: LG-B-003 | Phối hợp: LG-A-006

• **4 Việc cần làm:**
1. Duy trì bảng ánh xạ ConcurrentDictionary<string, DateTime> ghi nhận LastSeenUtc của từng máy trạm.
2. Khi nhận HEARTBEAT, cập nhật LastSeenUtc và chuyển trạng thái máy thành ONLINE nếu trước đó đang OFFLINE.
3. Khởi tạo vòng lặp kiểm tra định kỳ mỗi 5 giây (sử dụng PeriodicTimer).
4. Áp dụng công thức hạn chót: Nếu thời gian trôi qua vượt quá 3 khoảng nhịp tim (15 giây) + 5 giây dung sai (grace period) = 20 giây -> đánh dấu OFFLINE.

📌 📋 PROMPT COPY NHANH CHO AI (LG-B-006)

Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai B).
Tôi đang làm nhiệm vụ LG-B-006 của LabGuard: 'Liveness registry và monotonic deadline evaluator'.
Mục tiêu cốt lõi: LivenessRegistry.cs.
Ví dụ trực giác: Giống như đồng hồ bấm giờ độc lập: Quá 20 giây không nghe tiếng chuông điểm danh từ Agent thì máy chủ bấm còi báo vắng mặt, không phụ thuộc vào giờ đồng hồ của máy trạm (Stopwatch monotonic clock).
Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục Networking.
3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
5. Hướng dẫn tôi cách chạy test 'TEST-B06' để tự mắt thấy kết quả đúng.
6. Nhắc tôi lưu tệp minh chứng: Liveness deadline event log, timeline đối chiếu.
7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

📌 🤔 PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)

- Bản chất kỹ thuật của Liveness threshold: Missed 3 intervals (15s) + 5s grace = 20s offline deadline là gì?
- Tại sao cần xử lý tình huống lỗi: Đồng hồ máy trạm bị sai lệch giờ; server chỉ căn cứ vào thời điểm nhận

> trên đồng hồ Stopwatch nội bộ?
> - Vị trí tệp và dòng code minh chứng cụ thể trong Networking?

• **Cách chạy test:**
Lệnh/Thao tác: Chạy bài test TEST-B06 (Ngắt heartbeat của một Agent thử nghiệm; đo thời gian server công bố offline)
Kết quả ĐÚNG: Chuyển trạng thái máy sang Offline sau đúng 20s ± 500ms; phát sự kiện DEVICE_UPDATE tới Admin

• **Nếu gặp lỗi (3 bước kiểm tra cụ thể):**
1. Sử dụng Stopwatch.GetTimestamp() trên server để tính khoảng thời gian trôi qua, không dùng DateTime của máy trạm. 2. Cài đặt ngưỡng timeout liveness đúng 20 giây (> 20s không nhận heartbeat hợp lệ). 3. Khi phát hiện thiết bị mất liên lạc, cập nhật trạng thái DEVICE_OFFLINE và phát sóng DEVICE_UPDATE.

• **Minh chứng & Vấn đáp:**
Tệp minh chứng: Liveness deadline event log, timeline đối chiếu
Câu hỏi thi chính: Đánh giá trạng thái hoạt động (Liveness) bằng đồng hồ đơn điệu máy chủ? Không tin tưởng mốc thời gian của client gửi lên; dùng thời gian thực trôi qua trên server để tránh lỗi lệch giờ (Clock Drift)

Thẻ #07: [LG-B-007] — RULE EVALUATOR: HYSTERESIS VÀ COUNTERS (Tuần: W2 | P0)

• **Mục tiêu cụ thể:** Hoàn thành RuleEvaluator.cs đáp ứng tiêu chuẩn Trigger: CPU >= 90% (streak 3), Recovery: CPU <= 75% (streak 3); SERVICE_DOWN 2 fail / 2 success

• **Trực giác đời thường:** *Giống như công tắc máy lạnh thông minh: Nhiệt độ nóng trên 30 độ đủ 3 phút mới bật máy lạnh, mát dưới 24 độ đủ 2 phút mới tắt, tránh rơ-le nhảy tanh tách liên tục (Hysteresis chống báo động giả [CP2 TO VERIFY]).*

• **Điều kiện:** Cần xong trước: LG-B-005 | Phối hợp: None

• **4 Việc cần làm:**
1. Định nghĩa quy tắc CPU: Kích hoạt khi CPU >= 90% liên tiếp 3 mẫu đo; Hồi phục khi CPU <= 75% liên tiếp 3 mẫu đo.
2. Định nghĩa quy tắc SERVICE_DOWN: Kích hoạt khi dịch vụ giám sát dừng 2 lần liên tiếp; Hồi phục khi dịch vụ chạy lại 2 lần liên tiếp.
3. Duy trì bộ đếm streak trong bộ nhớ theo cặp (DeviceId, RuleCode).
4. Khi thỏa điều kiện kích hoạt, gọi IncidentManager để tạo sự cố mới nếu chưa có sự cố nào đang mở.

> 📌 📋 **PROMPT COPY NHANH CHO AI (LG-B-007)**

Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai B).
Tôi đang làm nhiệm vụ LG-B-007 của LabGuard: 'Rule evaluator: hysteresis và counters'.
Mục tiêu cốt lõi: RuleEvaluator.cs.
Ví dụ trực giác: Giống như công tắc máy lạnh thông minh: Nhiệt độ nóng trên 30 độ đủ 3 phút mới bật máy lạnh, mát dưới 24 độ đủ 2 phút mới tắt, tránh rơ-le nhảy tanh tách liên tục (Hysteresis chống báo động giả [CP2 TO VERIFY]).
Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục RuleEngine.
3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
5. Hướng dẫn tôi cách chạy test 'TEST-B07' để tự mắt thấy kết quả đúng.
6. Nhắc tôi lưu tệp minh chứng: Test log kịch bản streak biến thiên, biểu đồ trạng thái.
7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

📌 🤔 PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)

- Bản chất kỹ thuật của Trigger: CPU >= 90% (streak 3), Recovery: CPU <= 75% (streak 3); SERVICE_DOWN 2 fail / 2 success là gì?
- Tại sao cần xử lý tình huống lỗi: Cảnh báo chớp nháy liên tục (flapping/alert fatigue); thuật toán hysteresis giải quyết triệt để?
- Vị trí tệp và dòng code minh chứng cụ thể trong RuleEngine?

• Cách chạy test:

Lệnh/Thao tác: Chạy bài test TEST-B07 (Bơm chuỗi telemetry biến thiên qua test harness (89%, 91%, 92%, 90%, 80%, 74%, 73%, 72%))
Kết quả ĐÚNG: Bật sự cố sau đúng 3 mẫu vượt ngưỡng; không tắt cảnh báo ở mức 80%; chỉ phục hồi sau đủ 3 mẫu <= 75%

• Nếu gặp lỗi (3 bước kiểm tra cụ thể):

1. Kiểm tra bộ đếm hysteresis: chỉ mở Incident khi CPU >= 90% liên tục trong 3 chu kỳ lấy mẫu [CP2 TO VERIFY]. 2. Chỉ chuyển ConditionState sang RECOVERED khi CPU < 80% trong 2 chu kỳ liên tiếp [CP2 TO VERIFY]. 3. Đảm bảo không tạo sự cố mới nếu sự cố trước đó trên cùng thiết bị vẫn đang ở trạng thái ACTIVE.

• Minh chứng & Vấn đáp:

Tệp minh chứng: Test log kịch bản streak biến thiên, biểu đồ trạng thái
Câu hỏi thi chính: Cơ chế trễ (Hysteresis) và đếm chuỗi liên tiếp (Streak) trong giám sát? Tạo khoảng cách giữa ngưỡng kích hoạt và ngưỡng phục hồi kèm điều kiện duy trì mẫu để loại bỏ các đỉnh tải tức thời

Thẻ #08: [LG-B-008] — INCIDENT PERSISTENCE VÀ UNIQUE ACTIVE CONSTRAINT (Tuần: W2 | P0)

• **Mục tiêu cụ thể:** Hoàn thành IncidentManager.cs đáp ứng tiêu chuẩn Filtered Unique Index: WHERE WorkflowState != 'RESOLVED'; ConditionState (ACTIVE/RECOVERED)

• **Trực giác đời thường:** *Giống như hồ sơ bệnh án: Một bệnh nhân không thể mở 2 hồ sơ điều trị cùng lúc cho cùng một triệu chứng sốt cao, tránh phân tán nhân lực (Unique active incident constraint).*

• **Điều kiện:** Cần xong trước: LG-B-002, LG-B-007 | Phối hợp: None

• **4 Việc cần làm:**
1. Kiểm tra xem máy trạm đã có sự cố cùng RuleCode ở trạng thái khác RESOLVED chưa.
2. Tận dụng ràng buộc Filtered Unique Index ở mức CSDL để bảo vệ chống trùng lặp ngay cả khi có 2 luồng cùng ghi.
3. Khi tạo sự cố, ghi nhận trạng thái ban đầu WorkflowState = OPEN và ConditionState = ACTIVE.
4. Thêm bản ghi tương ứng vào bảng IncidentEvents ghi rõ thời điểm và nguyên nhân kích hoạt.

📌 📋 PROMPT COPY NHANH CHO AI (LG-B-008)

Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai B).
Tôi đang làm nhiệm vụ LG-B-008 của LabGuard: 'Incident persistence và unique active constraint'.
Mục tiêu cốt lõi: IncidentManager.cs.
Ví dụ trực giác: Giống như hồ sơ bệnh án: Một bệnh nhân không thể mở 2 hồ sơ điều trị cùng lúc cho cùng một triệu chứng sốt cao, tránh phân tán nhân lực (Unique active incident constraint).
ĐẶC BIỆT LƯU Ý: Vòng đời sự cố hoàn chỉnh gồm 5 trạng thái WorkflowState: OPEN -> ACKNOWLEDGED -> IN_PROGRESS -> RECOVERED -> RESOLVED (trong đó RECOVERED là trạng thái hệ thống đã phục hồi kỹ thuật, làm điều kiện cần trước khi Admin/Operator bấm RESOLVE để đóng sự cố). Tuân thủ nghiêm ngặt 5 trạng thái chuẩn của hệ thống. Mã lỗi xung đột OCC là ERR_CONCURRENCY_VIOLATION.
Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục Storage.
3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
5. Hướng dẫn tôi cách chạy test 'TEST-B08' để tự mắt thấy kết quả đúng.
6. Nhắc tôi lưu tệp minh chứng: Script định nghĩa Filtered Index, log test chống duplicate ticket.
7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

📌 🧠 PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)

- Bản chất kỹ thuật của Filtered Unique Index: WHERE WorkflowState != 'RESOLVED'; ConditionState (ACTIVE/RECOVERED) là gì?
- Tại sao cần xử lý tình huống lỗi: Trùng lặp sự cố làm phân tán nhân lực xử lý; ràng buộc CSDL bảo đảm toàn vẹn dữ liệu ở mức cứng?
- Vị trí tệp và dòng code minh chứng cụ thể trong Storage?

• **Cách chạy test:**
Lệnh/Thao tác: Chạy bài test TEST-B08 (Cố tình kích hoạt mở sự cố 2 lần liên tiếp trên cùng thiết bị và rule)
Kết quả ĐÚNG: Chỉ tạo đúng 1 dòng sự cố; lần kích hoạt sau chỉ cập nhật lịch sử và giữ nguyên ticket đang mở

• **Nếu gặp lỗi (3 bước kiểm tra cụ thể):**
1. Kiểm tra chỉ mục duy nhất Unique Filtered Index trên bảng Incidents: ngăn chặn 2 sự cố ACTIVE cùng thiết bị. 2. Trạng thái WorkflowState khởi tạo ban đầu phải là OPEN (theo đúng chuẩn WorkflowState). 3. Phát sóng sự cố mới qua bản tin INCIDENT_OPEN tới toàn bộ Admin Client đang đăng ký nhận sự kiện.

• **Minh chứng & Vấn đáp:**
Tệp minh chứng: Script định nghĩa Filtered Index, log test chống duplicate ticket
Câu hỏi thi chính: Phân tách ConditionState vs WorkflowState và chỉ mục duy nhất có điều kiện?
ConditionState phản ánh tình trạng kỹ thuật thực tế của máy, WorkflowState phản ánh tiến trình xử lý của con người

Thẻ #09: [LG-B-009] — COMMANDS ACK/START/RESOLVE + ROWVERSION OCC (Tuần: W3 | P0)

• **Mục tiêu cụ thể:** Hoàn thành IncidentCommandHandler.cs đáp ứng tiêu chuẩn Optimistic Concurrency Control (OCC) via rowversion, Table ProcessedCommands

• **Trực giác đời thường:** *Giống như đặt vé xem phim: Hai người cùng bấm chọn 1 ghế, người xác nhận trước mua được, người thứ hai nhận thông báo xung đột phiên bản (ERR_CONCURRENCY_VIOLATION) và được tải lại sơ đồ ghế mới nhất.*

• **Điều kiện:** Cần xong trước: LG-B-008 | Phối hợp: LG-C-008

• **4 Việc cần làm:**
1. Trích xuất incidentId và expectedRowVersion từ payload của bản tin gửi lên.
2. Thực thi câu lệnh UPDATE Incidents SET WorkflowState = @newState, AssignedTo = @user WHERE IncidentId = @id AND RowVersion = @expectedRowVersion.
3. Kiểm tra số dòng bị ảnh hưởng (rows affected): nếu bằng 0, chứng minh sự cố đã bị sửa bởi người khác giữa chừng.
4. Nếu bằng 0: Trả về bản tin lỗi có mã ERR_CONCURRENCY_VIOLATION kèm trạng thái mới nhất để UI của client tự động tải lại.

> 📌 📋 **PROMPT COPY NHANH CHO AI (LG-B-009)**
> Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai B).
> Tôi đang làm nhiệm vụ LG-B-009 của LabGuard: 'Commands ACK/START/RESOLVE + rowversion OCC'.
> Mục tiêu cốt lõi: IncidentCommandHandler.cs.

Ví dụ trực giác: Giống như đặt vé xem phim: Hai người cùng bấm chọn 1 ghế, người xác nhận trước mua được, người thứ hai nhận thông báo xung đột phiên bản (ERR_CONCURRENCY_VIOLATION) và được tải lại sơ đồ ghế mới nhất.
ĐẶC BIỆT LƯU Ý: Vòng đời sự cố hoàn chỉnh gồm 5 trạng thái WorkflowState: OPEN -> ACKNOWLEDGED -> IN_PROGRESS -> RECOVERED -> RESOLVED (trong đó RECOVERED là trạng thái hệ thống đã phục hồi kỹ thuật, làm điều kiện cần trước khi Admin/Operator bấm RESOLVE để đóng sự cố). Tuân thủ nghiêm ngặt 5 trạng thái chuẩn của hệ thống. Mã lỗi xung đột OCC là ERR_CONCURRENCY_VIOLATION.
Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục Storage.
3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
5. Hướng dẫn tôi cách chạy test 'TEST-B09' để tự mắt thấy kết quả đúng.
6. Nhắc tôi lưu tệp minh chứng: Command handler log, test report xử lý tranh chấp 2 Admin.
7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

📌 🧠 PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)

- Bản chất kỹ thuật của Optimistic Concurrency Control (OCC) via rowversion, Table ProcessedCommands là gì?
- Tại sao cần xử lý tình huống lỗi: Ghi đè mất dữ liệu (Lost Update); OCC bằng rowversion phát hiện ngay lập tức mà không cần lock bảng?
- Vị trí tệp và dòng code minh chứng cụ thể trong Storage?

• Cách chạy test:

Lệnh/Thao tác: Chạy bài test TEST-B09 (Mô phỏng 2 Admin cùng gửi lệnh ACK trên cùng 1 sự cố tại cùng 1 mốc thời gian)
Kết quả ĐÚNG: 1 Admin cập nhật thành công, 1 Admin nhận mã lỗi xung đột 409 Conflict kèm dữ liệu mới nhất

• Nếu gặp lỗi (3 bước kiểm tra cụ thể):

1. Kiểm tra câu lệnh UPDATE: 'WHERE IncidentId = @Id AND RowVersion = @ExpectedRowVersion'; nếu @@ROWCOUNT == 0 thì trả lỗi ERR_CONCURRENCY_VIOLATION. 2. Kiểm tra bảng ProcessedCommands (UserId + CommandId): nếu cùng CommandId nhưng khác RequestDigest thì trả ERR_IDEMPOTENCY_KEY_REUSE. 3. Trả về đối tượng sự cố mới nhất trong phản hồi lỗi để client tự làm mới giao diện.

• Minh chứng & Vấn đáp:

Tệp minh chứng: Command handler log, test report xử lý tranh chấp 2 Admin
Câu hỏi thi chính: Kiểm soát tương tranh lạc quan (OCC) bằng kiểu rowversion? Đính kèm rowversion khi gửi lệnh cập nhật; câu lệnh SQL kiểm tra WHERE rowversion = @expected, nếu số dòng ảnh hưởng = 0 nghĩa là đã bị sửa đổi

Thẻ #10: [LG-B-010] — RESTART/DB FAULT HANDLING VÀ CENTRAL HEALTH (Tuần: W3 | P0)

• **Mục tiêu cụ thể:** Hoàn thành HealthMonitor.cs & DbFaultRecovery.cs đáp ứng tiêu chuẩn HEALTH_UPDATE (activeConnections, dbConnected, queueLag, uptime)

• **Trực giác đời thường:** *Giống như máy đo nhịp tim bệnh viện: Luôn hiển thị đèn xanh báo máy chủ khỏe mạnh, nếu đứt cáp CSDL tạm thời thì tự động kết nối lại và chuyển sang trạng thái chờ (DbFaultRecovery).*

• **Điều kiện:** Cần xong trước: LG-B-005 | Phối hợp: None

• **4 Việc cần làm:**
1. Xây dựng dịch vụ nền định kỳ 10 giây đo lường: activeConnections, dbConnected, queueLag, memoryUsed, uptime.
2. Phát sóng bản tin HEALTH_UPDATE đến các Dashboard quản trị để người trực phòng máy nắm được tình trạng máy chủ.
3. Khi câu lệnh DB ném ngoại lệ SqlException do mất mạng, không để crash tiến trình mà bật cờ DbDegraded.
4. Thử kết nối lại CSDL với chính sách gia tăng thời gian chờ (Exponential Backoff: 1s, 2s, 4s, tối đa 30s).

📌 📋 PROMPT COPY NHANH CHO AI (LG-B-010)

Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai B).
Tôi đang làm nhiệm vụ LG-B-010 của LabGuard: 'Restart/DB fault handling và Central health'.
Mục tiêu cốt lõi: HealthMonitor.cs & DbFaultRecovery.cs.
Ví dụ trực giác: Giống như máy đo nhịp tim bệnh viện: Luôn hiển thị đèn xanh báo máy chủ khỏe mạnh, nếu đứt cáp CSDL tạm thời thì tự động kết nối lại và chuyển sang trạng thái chờ (DbFaultRecovery).
Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục Testing.
3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
5. Hướng dẫn tôi cách chạy test 'TEST-B10' để tự mắt thấy kết quả đúng.
6. Nhắc tôi lưu tệp minh chứng: DB reconnect test trace, Health update broadcast log.
7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

📌 🧐 PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)

- Bản chất kỹ thuật của HEALTH_UPDATE (activeConnections, dbConnected, queueLag, uptime) là gì?
- Tại sao cần xử lý tình huống lỗi: Ngoại lệ mất kết nối DB làm chết server mạng; bọc retry logic có giới hạn và thông báo trạng thái NotReady?
- Vị trí tệp và dòng code minh chứng cụ thể trong Testing?

• **Cách chạy test:**
Lệnh/Thao tác: Chạy bài test TEST-B10 (Tạm dừng dịch vụ SQL Server 10s trong lúc client đang kết nối; sau đó bật lại)
Kết quả ĐÚNG: Central Server tự động kết nối lại CSDL thành công; không bị crash tiến trình; khôi phục ghi dữ liệu bình thường

• **Nếu gặp lỗi (3 bước kiểm tra cụ thể):**
1. Bọc truy vấn CSDL trong khối Polly hoặc vòng lặp retry có số lần tối đa (3 lần, delay 1s, 2s, 4s). 2. Nếu CSDL tạm thời mất kết nối, Central Server chuyển cờ IsReady = false và phản hồi trạng thái chờ. 3. Xử lý bản tin GET_SERVER_HEALTH và trả về SERVER_HEALTH gồm CPU server, RAM, số socket mở và trạng thái DB.

• **Minh chứng & Vấn đáp:**
Tệp minh chứng: DB reconnect test trace, Health update broadcast log
Câu hỏi thi chính: Xử lý lỗi đứt gãy kết nối CSDL trong ứng dụng máy chủ thường trực? Dùng Connection Resiliency / Retry Policy; hàng đợi tạm thời lưu trữ tin nhắn trong RAM ngắn hạn chờ DB phục hồi

Thẻ #11: [LG-B-011] — RETENTION, QUERY/REPORT VÀ RESTORE PLAN (Tuần: W3 | P0)

• **Mục tiêu cụ thể:** Hoàn thành RetentionWorker.cs & QueryService.cs đáp ứng tiêu chuẩn Purge query: ReceivedUtc < 48h ago, GET_HISTORY, GET_METRICS

• **Trực giác đời thường:** *Giống như dọn rác văn phòng theo từng thùng nhỏ định kỳ để không làm nghẽn lối đi, và lưu bản sao sổ cái lưu kho sang két sắt (Retention batch 5000 rows [CP2 TO VERIFY], DB backup).*

• **Điều kiện:** Cần xong trước: LG-B-002 | Phối hợp: LG-C-010

• **4 Việc cần làm:**
1. Viết thủ tục lưu trữ hoặc câu lệnh DELETE có giới hạn theo lô (Batch Delete 1000 dòng) để tránh khóa bảng quá lâu.
2. Thiết lập RetentionWorker chạy vào ban đêm (hoặc mỗi giờ một lần) xóa các bản ghi trong MetricSnapshots có ReceivedUtc < 48 giờ trước.
3. Hiện thực hàm xử lý bản tin GET_HISTORY hỗ trợ lọc theo thời gian, thiết bị, trạng thái và phân trang.
4. Hiện thực hàm GET_METRICS trích xuất biểu đồ biến thiên tài nguyên của máy trạm phục vụ báo cáo.

> 📌 📋 **PROMPT COPY NHANH CHO AI (LG-B-011)**
> Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai B).
> Tôi đang làm nhiệm vụ LG-B-011 của LabGuard: 'Retention, query/report và restore plan'.
> Mục tiêu cốt lõi: RetentionWorker.cs & QueryService.cs.
> Ví dụ trực giác: Giống như dọn rác văn phòng theo từng thùng nhỏ định kỳ để không làm nghẽn lối đi, và lưu bản sao sổ cái lưu kho sang két sắt (Retention batch 5000 rows [CP2 TO VERIFY], DB backup).
> Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
> 1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.

2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục Storage.
3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
5. Hướng dẫn tôi cách chạy test 'TEST-B11' để tự mắt thấy kết quả đúng.
6. Nhắc tôi lưu tệp minh chứng: Retention execution log, số lượng record trước và sau dọn dẹp.
7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

📌 🧠 **PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)**
- Bản chất kỹ thuật của Purge query: ReceivedUtc < 48h ago, GET_HISTORY, GET_METRICS là gì?
- Tại sao cần xử lý tình huống lỗi: Lệnh DELETE lớn gây khóa bảng (Table Lock); xóa theo từng batch nhỏ 5000 dòng để không ảnh hưởng ghi mới?
- Vị trí tệp và dòng code minh chứng cụ thể trong Storage?

• **Cách chạy test:**
Lệnh/Thao tác: Chạy bài test TEST-B11 (Chèn dữ liệu giả lập 3 ngày trước; chạy job retention; kiểm tra số dòng còn lại)
Kết quả ĐÚNG: Xóa sạch dữ liệu thô quá 48h; giữ nguyên 100% dữ liệu trong 48h và toàn bộ lịch sử Incident

• **Nếu gặp lỗi (3 bước kiểm tra cụ thể):**
1. Kiểm tra tiến trình dọn dẹp (RetentionWorker): xóa số liệu telemetry cũ hơn 48h theo từng mẻ 5000 dòng [CP2 TO VERIFY] để tránh khóa bảng. 2. Xử lý truy vấn GET_HISTORY phân trang và trả về HISTORY_PAGE đúng định dạng. 3. Tạo script backup CSDL định kỳ: 'BACKUP DATABASE LabGuard_Dev TO DISK = ...' và lưu vào thư mục dự phòng.

• **Minh chứng & Vấn đáp:**
Tệp minh chứng: Retention execution log, số lượng record trước và sau dọn dẹp
Câu hỏi thi chính: Chiến lược lưu trữ phân tầng và dọn dẹp dữ liệu (Data Retention)? Dữ liệu thô phát sinh liên tục làm phình to CSDL nhanh chóng; cần dọn dẹp định kỳ và chỉ lưu trữ dài hạn sự cố tóm tắt

Thẻ #12: [LG-B-012] — MVP CONCURRENCY VÀ INCIDENT ACCEPTANCE (Tuần: W3 | P0)

• **Mục tiêu cụ thể:** Hoàn thành ConcurrencyAcceptanceReport.md đáp ứng tiêu chuẩn Hệ thống MVP tích hợp đầy đủ 3 khối

• **Trực giác đời thường:** *Giống như kiểm tra chịu tải cầu đường: Cho 50 xe tải chạy qua cầu cùng lúc để kiểm tra nhịp cầu có bị võng hay nứt không trước khi cắt băng khánh thành (Concurrency acceptance test).*

• **Điều kiện:** Cần xong trước: LG-B-009, LG-B-010 | Phối hợp: LG-C-012

• 4 Việc cần làm:

1. Phối hợp cùng A và C dựng kịch bản thử nghiệm tích hợp 3 khối trên môi trường mạng nội bộ.
2. Chạy kịch bản 10 máy trạm gửi đồng thời nhịp tim và số liệu, kiểm tra hàng đợi IngressChannel không bị tràn.
3. Kích hoạt kịch bản 2 Admin cùng tranh chấp xử lý 1 sự cố để kiểm chứng phản hồi ERR_CONCURRENCY_VIOLATION.
4. Kiểm tra CSDL không xuất hiện bản ghi mồ côi hoặc vi phạm ràng buộc toàn vẹn.

> 📌 📋 **PROMPT COPY NHANH CHO AI (LG-B-012)**
> Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai B).
> Tôi đang làm nhiệm vụ LG-B-012 của LabGuard: 'MVP concurrency và Incident acceptance'.
> Mục tiêu cốt lõi: ConcurrencyAcceptanceReport.md.
> Ví dụ trực giác: Giống như kiểm tra chịu tải cầu đường: Cho 50 xe tải chạy qua cầu cùng lúc để kiểm tra nhịp cầu có bị võng hay nứt không trước khi cắt băng khánh thành (Concurrency acceptance test).
> Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
> 1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
> 2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục Integration.
> 3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
> 4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
> 5. Hướng dẫn tôi cách chạy test 'TEST-B12' để tự mắt thấy kết quả đúng.
> 6. Nhắc tôi lưu tệp minh chứng: ConcurrencyAcceptanceReport.md, trace đối soát dữ liệu.
> 7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

> 📌 🧠 **PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)**
> - Bản chất kỹ thuật của Hệ thống MVP tích hợp đầy đủ 3 khối là gì?
> - Tại sao cần xử lý tình huống lỗi: Phát hiện lỗi race condition; cô lập module lỗi và sửa dứt điểm trước khi quyết định mở rộng P1?
> - Vị trí tệp và dòng code minh chứng cụ thể trong Integration?

• Cách chạy test:

Lệnh/Thao tác: Chạy bài test TEST-B12 (Chạy kịch bản 33 test scenarios từ Master Plan; kiểm tra log lỗi CSDL và mạng)
Kết quả ĐÚNG: Không có lỗi nghiêm trọng Sev1/Sev2; số liệu hiển thị trên Admin nhất quán 100% với CSDL

• Nếu gặp lỗi (3 bước kiểm tra cụ thể):

1. Chạy bài kiểm thử đồng thời 10 client gửi lệnh INCIDENT_ACK trên cùng một phiếu sự cố. 2. Xác nhận đúng 1 client nhận kết quả SUCCESS và 9 client còn lại nhận lỗi ERR_CONCURRENCY_VIOLATION. 3. Lưu báo cáo kết quả kiểm thử tương tranh vào tệp minh chứng để phục vụ nghiệm thu.

• Minh chứng & Vấn đáp:

Tệp minh chứng: ConcurrencyAcceptanceReport.md, trace đối soát dữ liệu

Câu hỏi thi chính: Đảm bảo tính nhất quán dữ liệu trong môi trường nhiều tác nhân đồng thời? Kết hợp phân tầng: hàng đợi mạng tuần tự, transaction CSDL, kiểm tra khóa chính dedup và OCC trên đối tượng nghiệp vụ

Thẻ #13: [LG-B-013] — INTERNAL COLLECTOR LISTENER VÀ DISPATCH (Tuần: W4 | P1)

• **Mục tiêu cụ thể:** Hoàn thành InternalCollectorDispatcher.cs đáp ứng tiêu chuẩn HELLO_COLLECTOR/AUTH_COLLECTOR, FORWARD_DATA (multiplexed stream qua 1 socket)

• **Trực giác đời thường:** *Giống như mở cổng phụ nội bộ có bảo vệ kiểm tra mật mã dành riêng cho xe chở hàng gom từ các chi nhánh (Internal Collector listener & dispatch [P1]).*

• **Điều kiện:** Cần xong trước: M4 Gate GO, LG-B-012 | Phối hợp: LG-C-014

• **4 Việc cần làm:**
1. Mở thêm một cổng lắng nghe chuyên biệt dành riêng cho các Collector nội bộ.
2. Xử lý bản tin xác thực HELLO_COLLECTOR/AUTH_COLLECTOR để đăng ký danh tính và dải phòng máy mà Collector đại diện.
3. Hiện thực bộ giải mã bóc tách luồng dữ liệu gộp (Multiplexed Frame) thành từng WireEnvelope riêng biệt.
4. Đưa các thông điệp đã bóc tách vào IngressChannel chung của hệ thống với cờ nhận diện nguồn từ Collector.

> 📌 📋 **PROMPT COPY NHANH CHO AI (LG-B-013)**
> Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai B).
> Tôi đang làm nhiệm vụ LG-B-013 của LabGuard: 'Internal Collector listener và dispatch'.
> Mục tiêu cốt lõi: InternalCollectorDispatcher.cs.
> Ví dụ trực giác: Giống như mở cổng phụ nội bộ có bảo vệ kiểm tra mật mã dành riêng cho xe chở hàng gom từ các chi nhánh (Internal Collector listener & dispatch [P1]).
> Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
> 1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
> 2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục Networking.
> 3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
> 4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
> 5. Hướng dẫn tôi cách chạy test 'TEST-B13' để tự mắt thấy kết quả đúng.
> 6. Nhắc tôi lưu tệp minh chứng: Internal listener connection log, dispatch throughput counters.
> 7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

> 📌 🤔 **PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)**
> - Bản chất kỹ thuật của HELLO_COLLECTOR/AUTH_COLLECTOR, FORWARD_DATA (multiplexed stream qua 1 socket) là gì?

> - Tại sao cần xử lý tình huống lỗi: Collector giả mạo kết nối vào cổng nội bộ; xác thực token bí mật chia sẻ (Pre-shared Secret) khi bắt tay?
> - Vị trí tệp và dòng code minh chứng cụ thể trong Networking?

• Cách chạy test:
Lệnh/Thao tác: Chạy bài test TEST-B13 (Kết nối đồng thời 2 Collector vào Central; đẩy dữ liệu từ 20 agent qua Collector)
Kết quả ĐÚNG: Central nhận diện đúng danh tính từng Collector; giải mã và xử lý trơn tru luồng tin ghép

• Nếu gặp lỗi (3 bước kiểm tra cụ thể):
1. [P1] Mở cổng lắng nghe nội bộ 5001 dành riêng cho Collector, kiểm tra xác thực AUTH_COLLECTOR bằng khóa bí mật. 2. Từ chối ngay lập tức các kết nối không gửi đúng secret token nội bộ. 3. Điều phối các bản tin forward từ Collector vào cùng bộ đệm IngestChannel như các Agent kết nối trực tiếp.

• Minh chứng & Vấn đáp:
Tệp minh chứng: Internal listener connection log, dispatch throughput counters
Câu hỏi thi chính: Mô hình dồn luồng (Multiplexing) dữ liệu từ Collector về Central? Nhiều Agent kết nối tới Collector, Collector gộp dữ liệu đẩy qua một kết nối socket duy nhất về Central để tối ưu tài nguyên

Thẻ #14: [LG-B-014] — SESSION GENERATION QUA COLLECTOR (Tuần: W4 | P1)

• Mục tiêu cụ thể: Hoàn thành SessionGenerationManager.cs đáp ứng tiêu chuẩn Header sessionGeneration (monotonically increasing integer per device)

• Trực giác đời thường: *Giống như đóng dấu giáp lai phiên làm việc: Gói tin nào mang dấu cũ từ phiên kết nối trước lập tức bị loại bỏ để tránh lệch trạng thái (Session generation tracking [P1]).*

• Điều kiện: Cần xong trước: LG-B-013 | Phối hợp: None

• 4 Việc cần làm:
1. Bổ sung trường sessionGeneration kiểu số nguyên tăng dần vào bảng theo dõi phiên máy trạm.
2. Khi Collector đăng ký lại sau sự cố, Central cấp phát một sessionGeneration mới tăng 1 đơn vị.
3. Trong mọi gói tin gửi qua Collector, kiểm tra số thế hệ: nếu nhỏ hơn thế hệ hiện tại của máy trạm thì loại bỏ ngay.
4. Ghi log cảnh báo khi phát hiện gói tin đến trễ hoặc rớt từ phiên làm việc trước.

> 📌 📋 **PROMPT COPY NHANH CHO AI (LG-B-014)**
> Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai B).
> Tôi đang làm nhiệm vụ LG-B-014 của LabGuard: 'Session generation qua Collector'.
> Mục tiêu cốt lõi: SessionGenerationManager.cs.
> Ví dụ trực giác: Giống như đóng dấu giáp lai phiên làm việc: Gói tin nào mang dấu cũ từ phiên kết nối trước lập tức bị loại bỏ để tránh lệch trạng thái (Session generation tracking [P1]).

> Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
> 1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
> 2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục Architecture.
> 3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
> 4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
> 5. Hướng dẫn tôi cách chạy test 'TEST-B14' để tự mắt thấy kết quả đúng.
> 6. Nhắc tôi lưu tệp minh chứng: Session fencing log, test report loại bỏ bản tin cũ.
> 7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

> 📌 🧐 **PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)**
> - Bản chất kỹ thuật của Header sessionGeneration (monotonically increasing integer per device) là gì?
> - Tại sao cần xử lý tình huống lỗi: Gói tin cũ bị trễ mạng đến sau làm sai lệch trạng thái mới (Out-of-order packets); session generation ngăn chặn triệt để?
> - Vị trí tệp và dòng code minh chứng cụ thể trong Architecture?

• **Cách chạy test:**
Lệnh/Thao tác: Chạy bài test TEST-B14 (Mô phỏng gửi xen kẽ các bản tin thế hệ cũ (gen 1) sau khi đã thiết lập thế hệ mới (gen 2))
Kết quả ĐÚNG: Central tự động hủy bỏ các bản tin có thế hệ thấp hơn thế hệ hiện tại mà không gây xáo trộn trạng thái

• **Nếu gặp lỗi (3 bước kiểm tra cụ thể):**
1. [P1] Quản lý số hiệu thế hệ phiên (Session Generation): mỗi lần Collector kết nối lại thì tăng thế hệ phiên lên 1. 2. Loại bỏ ngay các bản tin mang số hiệu thế hệ cũ hơn thế hệ hiện tại để tránh trễ mạng gây sai lệch dữ liệu. 3. Xử lý lệnh CLOSE_SESSION từ Admin khi cần cưỡng bức đóng một phiên làm việc bất thường.

• **Minh chứng & Vấn đáp:**
Tệp minh chứng: Session fencing log, test report loại bỏ bản tin cũ
Câu hỏi thi chính: Kỹ thuật Session Generation / Fencing Token trong hệ thống mạng phân tán? Gán số thứ tự thế hệ tăng dần; mọi thao tác từ thế hệ cũ hơn đều bị từ chối để chống lỗi ghi đè dữ liệu quá hạn

Thẻ #15: [LG-B-015] — COLLECTOR LOSS RECOVERY VÀ OFFLINE SUPPRESSION (Tuần: W4 | P1)

• **Mục tiêu cụ thể:** Hoàn thành CollectorFailoverHandler.cs đáp ứng tiêu chuẩn Grace period 45s cho các Agent thuộc node Collector vừa mất kết nối

• **Trực giác đời thường:** *Giống như cúp điện tạm thời ở trạm biến áp: Đợi 45 giây xem cầu dao tự động có đóng lại không rồi mới phát loa báo sự cố cho toàn trường, tránh hoảng loạn không cần thiết (Collector suppression 45s [CP2 TO VERIFY]).*

• **Điều kiện:** Cần xong trước: LG-B-014 | Phối hợp: LG-A-015

• **4 Việc cần làm:**
1. Khi phát hiện kết nối Collector bị đứt, ghi nhận thời điểm đứt và gán trạng thái UNCERTAIN cho các máy trạm thuộc Collector đó.
2. Thiết lập bộ đếm thời gian ân hạn 45 giây, tạm dừng kích hoạt sự cố DEVICE_OFFLINE cho từng máy riêng lẻ.
3. Nếu Collector kết nối lại trong vòng 45 giây: Đồng bộ lại trạng thái và hủy bỏ lệnh báo sập hàng loạt.
4. Nếu quá 45 giây vẫn không thấy Collector: Báo động sự cố COLLECTOR_DOWN cấp độ cao và đánh dấu các máy trạm là OFFLINE.

📌 📋 PROMPT COPY NHANH CHO AI (LG-B-015)

Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai B).
Tôi đang làm nhiệm vụ LG-B-015 của LabGuard: 'Collector loss recovery và offline suppression'.
Mục tiêu cốt lõi: CollectorFailoverHandler.cs.
Ví dụ trực giác: Giống như cúp điện tạm thời ở trạm biến áp: Đợi 45 giây xem cầu dao tự động có đóng lại không rồi mới phát loa báo sự cố cho toàn trường, tránh hoảng loạn không cần thiết (Collector suppression 45s [CP2 TO VERIFY]).
Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục RuleEngine.
3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
5. Hướng dẫn tôi cách chạy test 'TEST-B15' để tự mắt thấy kết quả đúng.
6. Nhắc tôi lưu tệp minh chứng: Collector loss trace log, suppression timeline capture.
7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

📌 🤔 PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)

- Bản chất kỹ thuật của Grace period 45s cho các Agent thuộc node Collector vừa mất kết nối là gì?
- Tại sao cần xử lý tình huống lỗi: Bão cảnh báo giả làm tê liệt đội ngũ hỗ trợ; trì hoãn thông minh (Suppression) giúp hệ thống ổn định?
- Vị trí tệp và dòng code minh chứng cụ thể trong RuleEngine?

• **Cách chạy test:**
Lệnh/Thao tác: Chạy bài test TEST-B15 (Ngắt kết nối Collector 01; quan sát danh sách sự cố trên Admin trong 45s đầu)
Kết quả ĐÚNG: Không kích hoạt bão cảnh báo Offline hàng loạt; chỉ báo trạng thái chuyển vùng; nếu quá 45s chưa kết nối lại mới báo Offline

• **Nếu gặp lỗi (3 bước kiểm tra cụ thể):**
1. [P1] Khi mất kết nối với một Collector, bật cơ chế trì hoãn cảnh báo (Offline Suppression) trong 45 giây [CP2 TO VERIFY]. 2. Không vội vàng đánh dấu hàng loạt máy trạm thuộc Collector đó là OFFLINE trong thời gian ân hạn 45s. 3. Nếu sau 45s mà Collector không khôi phục, lúc này mới kích hoạt cảnh báo OFFLINE cho các máy trạm liên quan.

• **Minh chứng & Vấn đáp:**
Tệp minh chứng: Collector loss trace log, suppression timeline capture
Câu hỏi thi chính: Kỹ thuật hoãn cảnh báo (Alert Suppression) khi xảy ra sự cố hạ tầng tập trung? Khi một node trung chuyển gặp sự cố, nguyên nhân là ở hạ tầng mạng chứ không phải toàn bộ máy con bị hỏng; cần hoãn báo lỗi máy trạm

Thẻ #16: [LG-B-016] — TUNE SQL INDEX VÀ INGEST BUDGETS (Tuần: W4 | P0/Stability)

• **Mục tiêu cụ thể:** Hoàn thành DbPerformanceAudit.md & OptimizedIndexes.sql đáp ứng tiêu chuẩn Ingest budget: latency < 50ms cho batch 50 records

• **Trực giác đời thường:** *Giống như đánh số mục lục và dán nhãn sách tra cứu nhanh: Lật đúng trang tìm số liệu trong chớp mắt mà không làm chậm người đang xếp sách vào kệ (Tune SQL index & ingest budget < 50ms [CP2 TO VERIFY]).*

• **Điều kiện:** Cần xong trước: LG-B-005 | Phối hợp: None

• **4 Việc cần làm:**
1. Phân tích kế hoạch thực thi của các câu truy vấn trọng yếu: Ingest, Liveness Check, Incident Query.
2. Thêm chỉ mục bao phủ (Covering Index) chứa cột Include để loại bỏ thao tác Key Lookup tốn kém.
3. Kiểm tra ngân sách: Thao tác ghi nhận một lô 50 bản ghi phải hoàn thành trong dưới 50ms.
4. Tinh chỉnh cấu hình kích thước cấp phát tệp dữ liệu MDF và nhật ký LDF để tránh phân mảnh đĩa.

> 📌 📋 **PROMPT COPY NHANH CHO AI (LG-B-016)**
> Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai B).
> Tôi đang làm nhiệm vụ LG-B-016 của LabGuard: 'Tune SQL index và ingest budgets'.
> Mục tiêu cốt lõi: DbPerformanceAudit.md & OptimizedIndexes.sql.
> Ví dụ trực giác: Giống như đánh số mục lục và dán nhãn sách tra cứu nhanh: Lật đúng trang tìm số liệu trong chớp mắt mà không làm chậm người đang xếp sách vào kệ (Tune SQL index & ingest budget < 50ms [CP2 TO VERIFY]).
> Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
> 1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
> 2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục Testing.
> 3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
> 4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
> 5. Hướng dẫn tôi cách chạy test 'TEST-B16' để tự mắt thấy kết quả đúng.
> 6. Nhắc tôi lưu tệp minh chứng: SQL Server Execution Plan, báo cáo đo đạc hiệu năng.

> 7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

> 📌 🤔 **PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)**
> - Bản chất kỹ thuật của Ingest budget: latency < 50ms cho batch 50 records là gì?
> - Tại sao cần xử lý tình huống lỗi: Chỉ mục quá nhiều làm chậm thao tác ghi; cân bằng giữa tốc độ ghi của Ingest và tốc độ đọc của Admin?
> - Vị trí tệp và dòng code minh chứng cụ thể trong Testing?

• **Cách chạy test:**
Lệnh/Thao tác: Chạy bài test TEST-B16 (Đo đạc execution plan và thời gian insert liên tục 10,000 dòng telemetry)
Kết quả ĐÚNG: Thời gian ghi mẻ đạt p95 < 30ms; không xảy ra hiện tượng thiếu bộ đệm hay lock escalation

• **Nếu gặp lỗi (3 bước kiểm tra cụ thể):**
1. Mở SQL Server Profiler hoặc Extended Events kiểm tra thời gian thực thi của câu lệnh chèn telemetry. 2. Tinh chỉnh chỉ mục (Indexes): bảo đảm các truy vấn theo DeviceId và Timestamp có chỉ mục hỗ trợ phù hợp. 3. Kiểm tra ngân sách thời gian xử lý: đảm bảo thời gian ghi đĩa trung bình < 50ms cho mỗi mẻ tin [CP2 TO VERIFY].

• **Minh chứng & Vấn đáp:**
Tệp minh chứng: SQL Server Execution Plan, báo cáo đo đạc hiệu năng
Câu hỏi thi chính: Chiến lược tối ưu hóa chỉ mục CSDL cho hệ thống chuyên ghi (Write-heavy)? Tránh đánh index trên các cột có độ biến thiên cao không cần thiết; dùng Clustered Index phù hợp và duy trì bảo trì chỉ mục

Thẻ #17: [LG-B-017] — PARSER/AUTH/INCIDENT REGRESSION (Tuần: W4 | P0/Stability)

• **Mục tiêu cụ thể:** Hoàn thành RegressionSuiteResults.md đáp ứng tiêu chuẩn Test suite tự động bao phủ các kịch bản quan trọng

• **Trực giác đời thường:** *Giống như kiểm tra sức khỏe tổng quát định kỳ: Tự động chạy lại toàn bộ xét nghiệm để bảo đảm việc uống thuốc mới không gây tác dụng phụ (Automated regression suite).*

• **Điều kiện:** Cần xong trước: LG-B-009, LG-B-016 | Phối hợp: None

• **4 Việc cần làm:**
1. Gom toàn bộ các bài kiểm thử đơn vị và tích hợp vào một kịch bản chạy tự động (dotnet test).
2. Kiểm tra lại việc bảo mật mật khẩu PBKDF2 với các trường hợp mật khẩu đúng, sai, chuỗi rỗng.

3. Kiểm tra lại tính nguyên tử của giao dịch sự cố khi giả lập đứt kết nối giữa chừng.
4. Chạy lại bài kiểm tra khử trùng lặp lô số liệu với 100 gói tin lặp liên tiếp.

📌 📋 PROMPT COPY NHANH CHO AI (LG-B-017)

Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai B).
Tôi đang làm nhiệm vụ LG-B-017 của LabGuard: 'Parser/auth/incident regression'.
Mục tiêu cốt lõi: RegressionSuiteResults.md.
Ví dụ trực giác: Giống như kiểm tra sức khỏe tổng quát định kỳ: Tự động chạy lại toàn bộ xét nghiệm để bảo đảm việc uống thuốc mới không gây tác dụng phụ (Automated regression suite).
Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục Testing.
3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
5. Hướng dẫn tôi cách chạy test 'TEST-B17' để tự mắt thấy kết quả đúng.
6. Nhắc tôi lưu tệp minh chứng: Báo cáo kết quả kiểm thử hồi quy, test run console log.
7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

📌 🧠 PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)

- Bản chất kỹ thuật của Test suite tự động bao phủ các kịch bản quan trọng là gì?
- Tại sao cần xử lý tình huống lỗi: Sửa chức năng này làm hỏng chức năng khác; bộ test tự động phát hiện sớm sai sót?
- Vị trí tệp và dòng code minh chứng cụ thể trong Testing?

• Cách chạy test:

Lệnh/Thao tác: Chạy bài test TEST-B17 (Thực thi toàn bộ kịch bản kiểm thử tích hợp)
Kết quả ĐÚNG: 100% kịch bản kiểm thử pass; không phát sinh lỗi hồi quy sau các chỉnh sửa hiệu năng

• Nếu gặp lỗi (3 bước kiểm tra cụ thể):

1. Chạy toàn bộ bộ test hồi quy tự động: 'dotnet test --filter Category=Regression'. 2. Kiểm tra tính toàn vẹn của bộ phân tích cú pháp JSON, logic xác thực mật khẩu PBKDF2 và quy trình sự cố. 3. Đảm bảo 100% test case trong bộ hồi quy vượt qua mà không có lỗi hồi quy phát sinh.

• Minh chứng & Vấn đáp:

Tệp minh chứng: Báo cáo kết quả kiểm thử hồi quy, test run console log
Câu hỏi thi chính: Tầm quan trọng của kiểm thử hồi quy (Regression Testing) trước khi đóng băng mã nguồn? Đảm bảo các thay đổi tối ưu mã hoặc sửa lỗi không vô tình phá vỡ các hợp đồng giao tiếp và tính toàn vẹn đã được kiểm chứng

Thẻ #18: [LG-B-018] — DB RESTORE VÀ CRASH CONSISTENCY PROOF (Tuần: W5 | P0/Release)

• **Mục tiêu cụ thể:** Hoàn thành DbRestoreProof.md & Backup.bak đáp ứng tiêu chuẩn SQL Server Full Backup & Restore, DBCC CHECKDB

• **Trực giác đời thường:** *Giống như diễn tập phục hồi sau hỏa hoạn: Lấy cuốn sổ lưu trong két sắt ra kiểm tra xem các trang ghi chép có nguyên vẹn và đọc lại được 100% không (DB restore & crash consistency).*

• **Điều kiện:** Cần xong trước: LG-B-011 | Phối hợp: None

• **4 Việc cần làm:**
1. Soạn thảo lệnh BACKUP DATABASE LabGuard TO DISK = '...' WITH INIT, COMPRESSION.
2. Tiến hành giả lập sự cố: tắt nóng tiến trình SQL Server hoặc ngắt ứng dụng giữa lúc đang ghi số liệu.
3. Khởi động lại dịch vụ và thực thi câu lệnh DBCC CHECKDB ('LabGuard') WITH NO_INFOMSGS kiểm tra tính toàn vẹn trang dữ liệu.
4. Thực hiện kịch bản phục hồi cơ sở dữ liệu từ tệp Backup.bak sang một CSDL tạm thời để kiểm tra dữ liệu.

📌 📋 PROMPT COPY NHANH CHO AI (LG-B-018)

Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai B).
Tôi đang làm nhiệm vụ LG-B-018 của LabGuard: 'DB restore và crash consistency proof'.
Mục tiêu cốt lõi: DbRestoreProof.md & Backup.bak.
Ví dụ trực giác: Giống như diễn tập phục hồi sau hỏa hoạn: Lấy cuốn sổ lưu trong két sắt ra kiểm tra xem các trang ghi chép có nguyên vẹn và đọc lại được 100% không (DB restore & crash consistency).
Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục Storage.
3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
5. Hướng dẫn tôi cách chạy test 'TEST-B18' để tự mắt thấy kết quả đúng.
6. Nhắc tôi lưu tệp minh chứng: File backup CSDL .bak, output lệnh DBCC CHECKDB.
7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

📌 🧠 PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)

- Bản chất kỹ thuật của SQL Server Full Backup & Restore, DBCC CHECKDB là gì?
- Tại sao cần xử lý tình huống lỗi: File cơ sở dữ liệu bị hỏng vật lý; cấu hình chế độ ghi đồng bộ và kiểm tra trang lưu trữ?
- Vị trí tệp và dòng code minh chứng cụ thể trong Storage?

• **Cách chạy test:**
Lệnh/Thao tác: Chạy bài test TEST-B18 (Backup CSDL; kill tiến trình SQL khi đang ghi; restore từ bản

backup và kiểm tra toàn vẹn)
Kết quả ĐÚNG: DBCC CHECKDB trả về 0 lỗi; dữ liệu sự cố và người dùng được phục hồi nguyên vẹn

• Nếu gặp lỗi (3 bước kiểm tra cụ thể):
1. Thực hiện kịch bản khôi phục CSDL: xóa database hiện tại và chạy lệnh RESTORE DATABASE từ bản backup gần nhất. 2. Kiểm tra tính nhất quán dữ liệu bằng lệnh 'DBCC CHECKDB(LabGuard_Dev)' xem có thông báo lỗi trang không. 3. Chụp ảnh màn hình kết quả kiểm tra nhất quán CSDL và lưu vào báo cáo nghiệm thu.

• Minh chứng & Vấn đáp:
Tệp minh chứng: File backup CSDL .bak, output lệnh DBCC CHECKDB
Câu hỏi thi chính: Cơ chế Write-Ahead Logging (WAL) và tính bền vững (Durability) trong CSDL? Mọi thay đổi phải được ghi vào Transaction Log trên đĩa trước khi ghi vào file dữ liệu chính để bảo đảm phục hồi khi mất điện

Thẻ #19: [LG-B-019] — AUDIT TÀI LIỆU ARCHITECTURE/PROTOCOL/DATABASE (Tuần: W5 | P0/Docs)

• Mục tiêu cụ thể: Hoàn thành AuditChecklist_B.md & Đồng bộ hồ sơ thiết kế đáp ứng tiêu chuẩn Toàn bộ tài liệu thiết kế và mã nguồn Central/CSDL

• Trực giác đời thường: *Giống như đối chiếu sổ sách kế toán với hàng hóa thực tế trong kho xem có khớp từng món và từng con số không (Technical documentation audit).*

• Điều kiện: Cần xong trước: LG-B-018 | Phối hợp: LG-A-019

• 4 Việc cần làm:
1. Đối chiếu danh sách cột của 8 bảng CSDL trong code DTO với file schema V1__Initial_Schema.sql.
2. Kiểm tra danh mục 69 bản tin xem phía Server đã xử lý đầy đủ các bản tin thuộc phạm vi P0 cốt lõi chưa.
3. Kiểm tra các mã lỗi trả về (ERROR, ERR_CONCURRENCY_VIOLATION, ERR_UNAUTHENTICATED) có đúng chuẩn chuỗi đã cam kết không.
4. Rà soát lại các giá trị cấu hình mặc định (cổng 5000, dung sai 20s liveness, chu kỳ dọn dẹp 48h).

> **PROMPT COPY NHANH CHO AI (LG-B-019)**
> Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai B).
> Tôi đang làm nhiệm vụ LG-B-019 của LabGuard: 'Audit tài liệu architecture/protocol/database'.
> Mục tiêu cốt lõi: AuditChecklist_B.md & Đồng bộ hồ sơ thiết kế.
> Ví dụ trực giác: Giống như đối chiếu sổ sách kế toán với hàng hóa thực tế trong kho xem có khớp từng món và từng con số không (Technical documentation audit).
> Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
> 1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
> 2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục Integration.
> 3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.

4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
5. Hướng dẫn tôi cách chạy test 'TEST-B19' để tự mắt thấy kết quả đúng.
6. Nhắc tôi lưu tệp minh chứng: Bảng đối soát kiến trúc hoàn chỉnh, biên bản review.
7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

📌 🤔 **PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)**
- Bản chất kỹ thuật của Toàn bộ tài liệu thiết kế và mã nguồn Central/CSDL là gì?
- Tại sao cần xử lý tình huống lỗi: Tài liệu không đồng bộ với code thực tế; cập nhật trực tiếp tài liệu theo đúng hiện trạng mã nguồn chuẩn?
- Vị trí tệp và dòng code minh chứng cụ thể trong Integration?

• **Cách chạy test:**
Lệnh/Thao tác: Chạy bài test TEST-B19 (Rà soát từng bảng CSDL, từng trường DTO và so khớp với file CONTRACT_MATRIX.md)
Kết quả ĐÚNG: 100% tên trường, kiểu dữ liệu và mô tả kỹ thuật thống nhất tuyệt đối giữa tài liệu và mã nguồn

• **Nếu gặp lỗi (3 bước kiểm tra cụ thể):**
1. Rà soát file 03_PROTOCOL_SPEC.md và CONTRACT_MATRIX.md: đối chiếu từng trường trong DTO với code C#. 2. Sửa ngay các sai lệch về tên bản tin hoặc mã lỗi trong tài liệu cho khớp chính xác với mã nguồn thực tế. 3. Đảm bảo toàn bộ 69 thông điệp wire protocol được mô tả đúng danh mục chuẩn.

• **Minh chứng & Vấn đáp:**
Tệp minh chứng: Bảng đối soát kiến trúc hoàn chỉnh, biên bản review
Câu hỏi thi chính: Tầm quan trọng của tính đồng bộ giữa tài liệu kiến trúc và mã nguồn? Tài liệu chuẩn xác giúp bảo trì hệ thống hiệu quả và là căn cứ pháp lý kỹ thuật khi nghiệm thu đồ án với hội đồng

Thẻ #20: [LG-B-020] — VIVA B VÀ BÀN GIAO BACKUP (Tuần: W5 | P0/Viva)

• **Mục tiêu cụ thể:** Hoàn thành HANDBOOK_B.md & Viva rehearsal log đáp ứng tiêu chuẩn 30 câu hỏi viva chuyên sâu phần Server, Database & Rule Engine

• **Trực giác đời thường:** *Giống như diễn tập bảo vệ đồ án: Nắm vững bản chất khóa lạc quan OCC, thuật toán băm PBKDF2 và cơ chế chống trùng lặp để tự tin giải trình trước hội đồng.*

• **Điều kiện:** Cần xong trước: LG-B-019 | Phối hợp: None

• **4 Việc cần làm:**
1. Ôn tập kỹ nguyên lý khóa lạc quan (OCC) và giải thích tại sao dùng rowversion thay vì khóa bi quan (SELECT FOR UPDATE).
2. Nắm chắc thuật toán băm PBKDF2, giải thích ý nghĩa của salt, số vòng lặp và hàm FixedTimeEquals.
3. Chuẩn bị sẵn đoạn code minh chứng cho cơ chế khử trùng lặp (Idempotent Ingestion) qua bảng

ProcessedBatches.
4. Diễn tập trả lời các câu hỏi tình huống: 'Điều gì xảy ra nếu mạng chập chờn làm gửi lại gói tin?', 'Tại sao cần Filtered Index?'.

📌 📋 PROMPT COPY NHANH CHO AI (LG-B-020)

Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai B).
Tôi đang làm nhiệm vụ LG-B-020 của LabGuard: 'Viva B và bàn giao backup'.
Mục tiêu cốt lõi: HANDBOOK_B.md & Viva rehearsal log.
Ví dụ trực giác: Giống như diễn tập bảo vệ đồ án: Nắm vững bản chất khóa lạc quan OCC, thuật toán băm PBKDF2 và cơ chế chống trùng lặp để tự tin giải trình trước hội đồng.
Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục Viva.
3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
5. Hướng dẫn tôi cách chạy test 'TEST-B20' để tự mắt thấy kết quả đúng.
6. Nhắc tôi lưu tệp minh chứng: HANDBOOK_B.md, biên bản diễn tập vấn đáp nội bộ.
7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

📌 🧠 PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)

- Bản chất kỹ thuật của 30 câu hỏi viva chuyên sâu phần Server, Database & Rule Engine là gì?
- Tại sao cần xử lý tình huống lỗi: Nhầm lẫn giữa cơ chế khóa CSDL và khóa trong bộ nhớ; làm rõ ranh giới trong handbook?
- Vị trí tệp và dòng code minh chứng cụ thể trong Viva?

• Cách chạy test:
Lệnh/Thao tác: Chạy bài test TEST-B20 (Vấn đáp chéo giả định với thành viên A và C chấm điểm độc lập)
Kết quả ĐÚNG: Giải thích xuất sắc các khái niệm giao dịch, khóa, bất đồng bộ và tự tin thao tác SQL trực tiếp

• Nếu gặp lỗi (3 bước kiểm tra cụ thể):
1. Ôn tập kỹ giải thích cơ chế Optimistic Concurrency Control (OCC) bằng rowversion trong SQL Server. 2. Chuẩn bị giải thích tại sao chọn PBKDF2-HMAC-SHA256 600k rounds và cách phòng chống timing attack. 3. Mở sẵn file IncidentCommandHandler.cs và AuthService.cs để sẵn sàng giải trình code trước giảng viên.

• Minh chứng & Vấn đáp:
Tệp minh chứng: HANDBOOK_B.md, biên bản diễn tập vấn đáp nội bộ
Câu hỏi thi chính: Tổng quan kiến trúc máy chủ xử lý dữ liệu tập trung của Lane B? Trình bày toàn diện luồng nhận dữ liệu, xác thực an ninh, xử lý quy tắc sự cố và bảo toàn dữ liệu bằng CSDL quan hệ

PHẦN 5: SỔ TAY 20 NHIỆM VỤ CỦA SINH VIÊN C (DASHBOARD & UI WINFORMS)

Sinh viên C phụ trách giao diện WinForms quản trị, kết nối RPC client qua TLS, chuyển luồng an toàn bằng Control.Invoke / BeginInvoke, lưới phòng máy High-DPI và kịch bản demo 12 phút. Dưới đây là 20 thẻ nhiệm vụ từ LG-C-001 đến LG-C-020:

Thẻ #01: [LG-C-001] — WINFORMS UI SHELL WIREFRAME VÀ MOCK DTO (Tuần: W1 | P0)

• **Mục tiêu cụ thể:** Hoàn thành MainForm.cs & NavigationFrame đáp ứng tiêu chuẩn WinForms native controls, UI layout standards, Mock view models

• **Trực giác đời thường:** *Giống như dựng mô hình sa bàn bằng bìa các-tông trước khi bắt tay đúc bê tông xây nhà thật (WinForms UI shell wireframe & mock DTO).*

• **Điều kiện:** Cần xong trước: LG-A-001 | Phối hợp: None

• **4 Việc cần làm:**
1. Thiết lập dự án WinForms trên .NET 9, cấu hình High-DPI PerMonitorV2 trong Program.cs.
2. Tạo MainForm với bố cục thanh điều hướng (Dashboard, Incidents, Reports, Settings) và vùng hiển thị chính.
3. Viết các lớp Mock DTO (MockDevice, MockIncident) tạo dữ liệu giả lập 20 máy trạm trong phòng máy.
4. Đảm bảo giao diện co giãn hợp lý khi thay đổi kích thước cửa sổ (Anchor, Dock).

> 📌 📋 **PROMPT COPY NHANH CHO AI (LG-C-001)**
> Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai C).
> Tôi đang làm nhiệm vụ LG-C-001 của LabGuard: 'WinForms UI shell wireframe và mock DTO'.
> Mục tiêu cốt lõi: MainForm.cs & NavigationFrame.
> Ví dụ trực giác: Giống như dựng mô hình sa bàn bằng bìa các-tông trước khi bắt tay đúc bê tông xây nhà thật (WinForms UI shell wireframe & mock DTO).
> Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
> 1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
> 2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục UI.
> 3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
> 4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
> 5. Hướng dẫn tôi cách chạy test 'TEST-C01' để tự mắt thấy kết quả đúng.
> 6. Nhắc tôi lưu tệp minh chứng: Ảnh chụp màn hình các form giao diện, file mã nguồn layout.
> 7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

> 📌 🤔 **PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)**
> - Bản chất kỹ thuật của WinForms native controls, UI layout standards, Mock view models là gì?

> - Tại sao cần xử lý tình huống lỗi: Giao diện bị giật (flickering); bật thuộc tính DoubleBuffered trên các UserControl?
> - Vị trí tệp và dòng code minh chứng cụ thể trong UI?

• Cách chạy test:
Lệnh/Thao tác: Chạy bài test TEST-C01 (Chạy ứng dụng WinForms, kiểm tra điều hướng giữa các màn hình)
Kết quả ĐÚNG: Giao diện hiển thị rõ ràng, không vỡ layout khi co giãn cửa sổ, các menu chuyển đổi mượt mà

• Nếu gặp lỗi (3 bước kiểm tra cụ thể):
1. Mở Form designer kiểm tra thuộc tính DoubleBuffered = true trên MainForm và các UserControl để chống giật hình. 2. Kiểm tra layout neo (Anchor / Dock) để đảm bảo khi phóng to cửa sổ giao diện co giãn hợp lý, không bị vỡ. 3. Tạo các đối tượng DTO mẫu (mock DTO) để đổ dữ liệu thử nghiệm lên giao diện trước khi nối mạng.

• Minh chứng & Vấn đáp:
Tệp minh chứng: Ảnh chụp màn hình các form giao diện, file mã nguồn layout
Câu hỏi thi chính: Thiết kế giao diện WinForms chuẩn mực và trải nghiệm người dùng? Sử dụng tông màu sáng chuyên nghiệp, bố cục phân cấp rõ ràng và hỗ trợ đầy đủ phím tắt điều hướng

Thẻ #02: [LG-C-002] — ADMIN TCP/TLS RPC CLIENT CONNECTION (Tuần: W1 | P0)

• Mục tiêu cụ thể: Hoàn thành AdminRpcClient.cs đáp ứng tiêu chuẩn TcpClient, SslStream, FrameCodec, Duplex channel

• Trực giác đời thường: *Giống như cắm dây điện thoại bảo mật nối từ bàn trực chỉ huy trực tiếp về phòng máy chủ trung tâm (Admin TCP/TLS RPC client connection).*

• Điều kiện: Cần xong trước: LG-A-002 | Phối hợp: LG-B-003

• 4 Việc cần làm:
1. Sử dụng TcpClient.ConnectAsync để thiết lập kết nối socket tới Central Server.
2. Khởi tạo SslStream bọc lấy NetworkStream, cấu hình kiểm tra chứng chỉ máy chủ an toàn.
3. Tích hợp FrameCodec để đóng gói độ dài khung 4 byte Big-Endian và giải mã dòng byte nhận được.
4. Duy trì vòng lặp đọc ngầm liên tục để đón các gói tin sự kiện phát sóng từ máy chủ.

> 📌 📋 **PROMPT COPY NHANH CHO AI (LG-C-002)**
> Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai C).
> Tôi đang làm nhiệm vụ LG-C-002 của LabGuard: 'Admin TCP/TLS RPC client connection'.
> Mục tiêu cốt lõi: AdminRpcClient.cs.

> Ví dụ trực giác: Giống như cắm dây điện thoại bảo mật nối từ bàn trực chỉ huy trực tiếp về phòng máy chủ trung tâm (Admin TCP/TLS RPC client connection).
> Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
> 1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
> 2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục Networking.
> 3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
> 4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
> 5. Hướng dẫn tôi cách chạy test 'TEST-C02' để tự mắt thấy kết quả đúng.
> 6. Nhắc tôi lưu tệp minh chứng: Network communication trace log, Wireshark dump.
> 7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

> 📌 🤔 **PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)**
> - Bản chất kỹ thuật của TcpClient, SslStream, FrameCodec, Duplex channel là gì?
> - Tại sao cần xử lý tình huống lỗi: Đứt cáp mạng không nhận được thông báo; gửi định kỳ heartbeat ping tầng ứng dụng để dò kênh truyền?
> - Vị trí tệp và dòng code minh chứng cụ thể trong Networking?

• **Cách chạy test:**
Lệnh/Thao tác: Chạy bài test TEST-C02 (Kết nối tới Central Server, gửi bản tin echo và kiểm tra nhận phản hồi)
Kết quả ĐÚNG: Bắt tay TLS thành công; duy trì kết nối duplex ổn định; tự động phát hiện khi kết nối đứt

• **Nếu gặp lỗi (3 bước kiểm tra cụ thể):**
1. Kiểm tra quy trình bắt tay TLS: nạp chứng chỉ CA hoặc thiết lập cấu hình tin cậy chứng chỉ phòng lab. 2. Gửi bản tin HELLO_ADMIN ngay sau khi kết nối socket thành công và chờ nhận HELLO_ACK từ server. 3. Bắt SocketException khi server chưa bật và hiển thị thông báo thân thiện 'Không thể kết nối đến máy chủ'.

• **Minh chứng & Vấn đáp:**
Tệp minh chứng: Network communication trace log, Wireshark dump
Câu hỏi thi chính: Kênh truyền hai chiều (Duplex Connection) trên nền tảng Socket TCP? Một kết nối socket duy nhất được sử dụng đồng thời cho cả việc gửi request và nhận các event bất đồng bộ do server chủ động đẩy về

Thẻ #03: [LG-C-003] — LOGIN/SIGNUP WINFORMS UI & TOKEN STORAGE CLIENT (Tuần: W1 | P0)

• **Mục tiêu cụ thể:** Hoàn thành LoginForm.cs & SessionStorage.cs đáp ứng tiêu chuẩn LOGIN (username, password), LOGIN_RESULT (token, role), Client RAM token store

• **Trực giác đời thường:** *Giống như thẻ nhân viên quẹt cửa: Đăng nhập đúng tên và mật khẩu thì cửa mở và cấp thẻ đeo tạm thời có ghi rõ quyền hạn Admin/Operator/Viewer (LoginForm & token storage).*

• **Điều kiện:** Cần xong trước: LG-C-001, LG-C-002 | Phối hợp: LG-B-004

• **4 Việc cần làm:**

1. Thiết kế biểu mẫu LoginForm với ô nhập Username, Password (ẩn ký tự), nút Đăng nhập và Đăng ký (nếu có mã mời).
2. Gửi bản tin LOGIN (username, password) qua AdminRpcClient và chờ phản hồi LOGIN_RESULT.
3. Khi đăng nhập thành công, lưu chuỗi token và thông tin vai trò (Role: Admin/Operator/Viewer) vào SessionStorage.
4. Đóng LoginForm và mở màn hình chính MainForm với quyền hạn tương ứng.

📌 📋 PROMPT COPY NHANH CHO AI (LG-C-003)

Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai C).
Tôi đang làm nhiệm vụ LG-C-003 của LabGuard: 'Login/Signup WinForms UI & token storage client'.
Mục tiêu cốt lõi: LoginForm.cs & SessionStorage.cs.
Ví dụ trực giác: Giống như thẻ nhân viên quẹt cửa: Đăng nhập đúng tên và mật khẩu thì cửa mở và cấp thẻ đeo tạm thời có ghi rõ quyền hạn Admin/Operator/Viewer (LoginForm & token storage).
Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục UI.
3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
5. Hướng dẫn tôi cách chạy test 'TEST-C03' để tự mắt thấy kết quả đúng.
6. Nhắc tôi lưu tệp minh chứng: Ảnh chụp màn hình form đăng nhập, code review session store.
7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

📌 🧠 PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)

- Bản chất kỹ thuật của LOGIN (username, password), LOGIN_RESULT (token, role), Client RAM token store là gì?
- Tại sao cần xử lý tình huống lỗi: Lưu mật khẩu plain text trên client; chỉ truyền mật khẩu qua TLS và không lưu mật khẩu vào bộ nhớ sau khi đăng nhập?
- Vị trí tệp và dòng code minh chứng cụ thể trong UI?

• **Cách chạy test:**

Lệnh/Thao tác: Chạy bài test TEST-C03 (Thực hiện đăng nhập thành công và đăng nhập sai mật khẩu; kiểm tra lưu token)
Kết quả ĐÚNG: Đăng nhập đúng mở màn hình chính; đăng nhập sai báo lỗi thân thiện; token được giữ trong RAM, không lưu đĩa dạng plain text

• **Nếu gặp lỗi (3 bước kiểm tra cụ thể):**

1. Kiểm tra form đăng nhập: bắt buộc nhập username và password, xóa mật khẩu khỏi bộ nhớ sau khi gửi.
2. [P0 OPTIONAL] Kiểm tra form đăng ký: bắt buộc người dùng nhập inviteCode hợp lệ; gửi bản tin

SIGN_UP. 3. Lưu sessionToken nhận được từ LOGIN_RESULT vào biến bảo mật trong bộ nhớ client, không lưu plain text vào ổ đĩa.

• **Minh chứng & Vấn đáp:**
Tệp minh chứng: Ảnh chụp màn hình form đăng nhập, code review session store
Câu hỏi thi chính: Quản lý vòng đời phiên làm việc (Session Lifecycle) phía máy khách? Lưu session token trong bộ nhớ tạm (RAM) của ứng dụng; tự động hủy bỏ khi người dùng nhấn Logout hoặc tắt ứng dụng

Thẻ #04: [LG-C-004] — KHỚP CẶP REQUEST/RESPONSE CORRELATION VÀ TIMEOUT (Tuần: W1 | P0)

• **Mục tiêu cụ thể:** Hoàn thành RpcCorrelationManager.cs đáp ứng tiêu chuẩn ConcurrentDictionary<Guid, TaskCompletionSource<WireEnvelope>>, timeout 5000ms

• **Trực giác đời thường:** *Giống như gửi thư có đính kèm phong bì hồi đáp ghi sẵn mã số: Khi thư phản hồi về tới nơi là biết ngay của câu hỏi nào (TaskCompletionSource dictionary, requestId UUIDv4).*

• **Điều kiện:** Cần xong trước: LG-C-002 | Phối hợp: None

• **4 Việc cần làm:**
1. Với mỗi yêu cầu gửi đi, tạo một đối tượng TaskCompletionSource<WireEnvelope> và lưu vào ConcurrentDictionary với khóa là requestId.
2. Thiết lập bộ đếm thời gian quá hạn 5000ms; nếu sau 5 giây không có hồi đáp, hủy Task và ném TimeoutException.
3. Khi luồng đọc mạng nhận được gói tin trả lời, trích xuất requestId trong Envelope, tìm kiếm trong Dictionary và kích hoạt tcs.SetResult(envelope).
4. Xóa đối tượng ra khỏi Dictionary ngay sau khi khớp cặp thành công để tránh rò rỉ bộ nhớ (Memory Leak).

> 📌 📋 **PROMPT COPY NHANH CHO AI (LG-C-004)**
> Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai C).
> Tôi đang làm nhiệm vụ LG-C-004 của LabGuard: 'Khớp cặp Request/Response correlation và timeout'.
> Mục tiêu cốt lõi: RpcCorrelationManager.cs.
> Ví dụ trực giác: Giống như gửi thư có đính kèm phong bì hồi đáp ghi sẵn mã số: Khi thư phản hồi về tới nơi là biết ngay của câu hỏi nào (TaskCompletionSource dictionary, requestId UUIDv4).
> Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
> 1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
> 2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục Testing.
> 3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
> 4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
> 5. Hướng dẫn tôi cách chạy test 'TEST-C04' để tự mắt thấy kết quả đúng.
> 6. Nhắc tôi lưu tệp minh chứng: RPC correlation benchmark log, unit test suite.
> 7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

> 📌 🧠 **PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)**
> - Bản chất kỹ thuật của ConcurrentDictionary<Guid, TaskCompletionSource<WireEnvelope>>, timeout 5000ms là gì?
> - Tại sao cần xử lý tình huống lỗi: Rò rỉ bộ nhớ do các TaskCompletionSource không bao giờ hoàn thành; dùng Timer hoặc CancellationToken để tự dọn dẹp?
> - Vị trí tệp và dòng code minh chứng cụ thể trong Testing?

• **Cách chạy test:**
Lệnh/Thao tác: Chạy bài test TEST-C04 (Gửi đồng thời 20 request qua kênh RPC tới mock server; kiểm tra khớp đúng kết quả)
Kết quả ĐÚNG: 100% response trả về đúng luồng gọi tương ứng; tự động hủy và ném TimeoutException sau 5s nếu server không phản hồi

• **Nếu gặp lỗi (3 bước kiểm tra cụ thể):**
1. Khởi tạo requestId bằng UUIDv4 chuẩn: Guid.NewGuid().ToString() cho mỗi request gửi đi. 2. Lưu TaskCompletionSource vào ConcurrentDictionary với khóa là requestId để ghép cặp response tương ứng. 3. Đặt thời gian chờ timeout (mặc định 10s): nếu quá hạn thì hủy Task và xóa khỏi dictionary để tránh rò rỉ RAM.

• **Minh chứng & Vấn đáp:**
Tệp minh chứng: RPC correlation benchmark log, unit test suite
Câu hỏi thi chính: Mô hình Request-Response trên kết nối Socket TCP hai chiều (RPC Correlation)? Mỗi request được gán một requestId (UUID); client lưu TaskCompletionSource vào từ điển; khi nhận response sẽ lấy ra hoàn thành Task

Thẻ #05: [LG-C-005] — BỘ NHẬN SNAPSHOT VÀ KHỞI TẠO CACHE ADMIN (Tuần: W2 | P0)

• **Mục tiêu cụ thể:** Hoàn thành SnapshotReceiver.cs & LocalCacheStore.cs đáp ứng tiêu chuẩn SNAPSHOT_BEGIN, SNAPSHOT_PAGE, SNAPSHOT_END protocol flow

• **Trực giác đời thường:** *Giống như mở cuốn album ảnh chụp toàn cảnh phòng máy lúc vừa bước vào cửa để nắm bắt đầy đủ hiện trạng trước khi theo dõi sự kiện mới (Snapshot receiver & local cache store).*

• **Điều kiện:** Cần xong trước: LG-C-004 | Phối hợp: LG-B-004

• **4 Việc cần làm:**
1. Bắt bản tin SNAPSHOT_BEGIN để chuẩn bị vùng nhớ đệm tạm thời và xóa bỏ trạng thái cũ.
2. Đón nhận lần lượt các gói tin SNAPSHOT_PAGE, giải mã danh sách máy trạm, trạng thái số liệu và sự cố đang mở.
3. Khi nhận bản tin SNAPSHOT_END, xác nhận toàn bộ ảnh chụp đã hoàn tất và nạp vào LocalCacheStore

chính thức.
4. Đảm bảo hiển thị thanh tiến trình (Progress Bar) thông báo cho người dùng biết trạng thái đang tải dữ liệu.

> 📌 📋 **PROMPT COPY NHANH CHO AI (LG-C-005)**
> Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai C).
> Tôi đang làm nhiệm vụ LG-C-005 của LabGuard: 'Bộ nhận Snapshot và khởi tạo cache Admin'.
> Mục tiêu cốt lõi: SnapshotReceiver.cs & LocalCacheStore.cs.
> Ví dụ trực giác: Giống như mở cuốn album ảnh chụp toàn cảnh phòng máy lúc vừa bước vào cửa để nắm bắt đầy đủ hiện trạng trước khi theo dõi sự kiện mới (Snapshot receiver & local cache store).
> Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
> 1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
> 2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục Storage.
> 3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
> 4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
> 5. Hướng dẫn tôi cách chạy test 'TEST-C05' để tự mắt thấy kết quả đúng.
> 6. Nhắc tôi lưu tệp minh chứng: Snapshot receipt timeline log, cache dump.
> 7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

> 📌 🧠 **PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)**
> - Bản chất kỹ thuật của SNAPSHOT_BEGIN, SNAPSHOT_PAGE, SNAPSHOT_END protocol flow là gì?
> - Tại sao cần xử lý tình huống lỗi: Hiển thị dữ liệu chắp vá khi đang nhận dở snapshot; cơ chế rào cản (Snapshot Barrier) ngăn chặn hiển thị trước khi hoàn tất?
> - Vị trí tệp và dòng code minh chứng cụ thể trong Storage?

• Cách chạy test:
Lệnh/Thao tác: Chạy bài test TEST-C05 (Mô phỏng nhận snapshot 50 thiết bị chia làm 5 trang; kiểm tra dữ liệu trong cache)
Kết quả ĐÚNG: Nạp đủ 50 thiết bị vào cache; chỉ kích hoạt hiển thị dữ liệu sau khi nhận được bản tin SNAPSHOT_END

• Nếu gặp lỗi (3 bước kiểm tra cụ thể):
1. Sau khi gửi SUBSCRIBE_DEVICES, chờ nhận bản tin SNAPSHOT_BEGIN để thiết lập rào cản nhận snapshot. 2. Nạp từng trang dữ liệu từ các bản tin SNAPSHOT_PAGE vào bộ nhớ đệm cache cục bộ trên client. 3. Khi nhận bản tin SNAPSHOT_END, mở khóa giao diện và bắt đầu áp dụng các delta event nhận được trong lúc tải.

• Minh chứng & Vấn đáp:
Tệp minh chứng: Snapshot receipt timeline log, cache dump
Câu hỏi thi chính: Cơ chế khởi tạo dữ liệu bằng Snapshot phân trang (Snapshot Barrier)? Chia nhỏ dữ liệu lớn thành nhiều trang để không gây nghẽn kết nối mạng; khóa hiển thị cho tới khi nhận đủ toàn bộ các trang

Thẻ #06: [LG-C-006] — ADMIN CACHE MERGE, RECONNECT VÀ UI-THREAD MARSHAL (Tuần: W2 | P0)

• **Mục tiêu cụ thể:** Hoàn thành CacheSynchronizer.cs & UiDispatcher.cs đáp ứng tiêu chuẩn Control.Invoke, Control.BeginInvoke, Control.InvokeAsync, eventSeq ordering

• **Trực giác đời thường:** *Giống như người đưa thư mang tin mới về không được tự tiện vẽ lên bảng điều khiển mà phải nhờ thư ký trực ban (Control.Invoke) vẽ hộ để tránh xung đột màn hình (UI-thread marshal & delta merge).*

• **Điều kiện:** Cần xong trước: LG-C-005 | Phối hợp: None

• **4 Việc cần làm:**
1. Viết phương thức tiện ích RunOnUi(Action action) tự động kiểm tra control.InvokeRequired.
2. Nếu InvokeRequired là true, gọi control.BeginInvoke(action) để chuyển tác vụ cập nhật về luồng giao diện chính.
3. Kiểm tra số thứ tự sự kiện (eventSeq): chỉ áp dụng các bản tin có eventSeq lớn hơn số hiện tại để loại bỏ các gói tin đến muộn.
4. Xử lý kịch bản mất mạng và kết nối lại: tự động dọn cache và yêu cầu Server gửi lại Snapshot mới.

📌 📋 PROMPT COPY NHANH CHO AI (LG-C-006)

Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai C).
Tôi đang làm nhiệm vụ LG-C-006 của LabGuard: 'Admin cache merge, reconnect và UI-thread marshal'.
Mục tiêu cốt lõi: CacheSynchronizer.cs & UiDispatcher.cs.
Ví dụ trực giác: Giống như người đưa thư mang tin mới về không được tự tiện vẽ lên bảng điều khiển mà phải nhờ thư ký trực ban (Control.Invoke) vẽ hộ để tránh xung đột màn hình (UI-thread marshal & delta merge).
Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục UI.
3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
5. Hướng dẫn tôi cách chạy test 'TEST-C06' để tự mắt thấy kết quả đúng.
6. Nhắc tôi lưu tệp minh chứng: UI thread safety test log, code review dispatcher.
7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

📌 🧠 PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)

- Bản chất kỹ thuật của Control.Invoke, Control.BeginInvoke, Control.InvokeAsync, eventSeq ordering là gì?
- Tại sao cần xử lý tình huống lỗi: Truy cập trực tiếp control từ luồng mạng gây crash ứng dụng; kiểm tra InvokeRequired và chuyển về UI thread?
- Vị trí tệp và dòng code minh chứng cụ thể trong UI?

• **Cách chạy test:**
Lệnh/Thao tác: Chạy bài test TEST-C06 (Bơm liên tục 100 event cập nhật từ background thread mạng vào giao diện)
Kết quả ĐÚNG: Giao diện cập nhật mượt mà; 0 ngoại lệ InvalidOperationException (Cross-thread operation not valid)

• **Nếu gặp lỗi (3 bước kiểm tra cụ thể):**
1. Khi nhận delta event từ luồng mạng, luôn kiểm tra thuộc tính InvokeRequired trên control giao diện. 2. Sử dụng Control.BeginInvoke để chuyển việc cập nhật giao diện về UI thread bất đồng bộ, tránh deadlock. 3. Kiểm tra biến cờ IsDisposed của Form trước khi gọi Invoke để tránh ném ObjectDisposedException khi đóng ứng dụng.

• **Minh chứng & Vấn đáp:**
Tệp minh chứng: UI thread safety test log, code review dispatcher
Câu hỏi thi chính: Điều phối luồng trong Windows Forms (Thread Marshaling)? Control trong WinForms gắn liền với UI thread (STA); mọi cập nhật giao diện từ luồng mạng phải được chuyển qua Invoke hoặc BeginInvoke

Thẻ #07: [LG-C-007] — DASHBOARD, LAB VIEW, DEVICE DETAIL (Tuần: W2 | P0)

• **Mục tiêu cụ thể:** Hoàn thành DashboardForm.cs, LabViewGridControl.cs, DeviceDetailDialog.cs đáp ứng tiêu chuẩn Grid layout phòng máy 20-40 máy trạm, trạng thái: Online (Xanh), Warning (Vàng), Offline (Xám)

• **Trực giác đời thường:** *Giống như bản đồ phòng điều khiển bay: 40 ô vuông đại diện cho 40 máy tính, đổi màu Xanh, Vàng, Đỏ thời gian thực kèm biểu tượng dễ nhận biết (Dashboard, Lab View, Device Detail).*

• **Điều kiện:** Cần xong trước: LG-C-006 | Phối hợp: None

• **4 Việc cần làm:**
1. Thiết kế lưới phòng máy gồm các ô đại diện cho từng máy trạm (LAB-01, LAB-02...).
2. Áp dụng quy tắc màu sắc chuẩn: Xanh lá (ONLINE bình thường), Vàng (WARNING - CPU/RAM cao), Đỏ/Xám (OFFLINE / Sự cố nghiêm trọng).
3. Hiển thị thanh tiến trình mini đo phần trăm CPU/RAM trực tiếp trên từng thẻ máy trạm.
4. Bắt sự kiện Double-Click vào một máy trạm để mở DeviceDetailDialog hiển thị chi tiết tiến trình, địa chỉ IP, MAC và lịch sử cảnh báo.

> 📌 📋 **PROMPT COPY NHANH CHO AI (LG-C-007)**
> Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai C).
> Tôi đang làm nhiệm vụ LG-C-007 của LabGuard: 'Dashboard, Lab View, Device Detail'.
> Mục tiêu cốt lõi: DashboardForm.cs, LabViewGridControl.cs, DeviceDetailDialog.cs.

> Ví dụ trực giác: Giống như bản đồ phòng điều khiển bay: 40 ô vuông đại diện cho 40 máy tính, đổi màu Xanh, Vàng, Đỏ thời gian thực kèm biểu tượng dễ nhận biết (Dashboard, Lab View, Device Detail).
> Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
> 1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
> 2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục UI.
> 3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
> 4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
> 5. Hướng dẫn tôi cách chạy test 'TEST-C07' để tự mắt thấy kết quả đúng.
> 6. Nhắc tôi lưu tệp minh chứng: Ảnh chụp màn hình Lab View 20 máy trạm ở các trạng thái khác nhau.
> 7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

> 📌 🧠 **PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)**
> - Bản chất kỹ thuật của Grid layout phòng máy 20-40 máy trạm, trạng thái: Online (Xanh), Warning (Vàng), Offline (Xám) là gì?
> - Tại sao cần xử lý tình huống lỗi: Người dùng không phân biệt được trạng thái do mù màu; kết hợp đồng thời màu nền, icon cảnh báo và văn bản trạng thái?
> - Vị trí tệp và dòng code minh chứng cụ thể trong UI?

• Cách chạy test:
Lệnh/Thao tác: Chạy bài test TEST-C07 (Hiển thị 20 máy trạm; mô phỏng đổi trạng thái 1 máy sang Warning và 1 máy sang Offline)
Kết quả ĐÚNG: Màu sắc và icon cập nhật tức thì; không phụ thuộc duy nhất vào màu sắc (hỗ trợ người mù màu với nhãn trạng thái rõ ràng)

• Nếu gặp lỗi (3 bước kiểm tra cụ thể):
1. Kiểm tra hiển thị lưới máy trạm: mỗi ô máy tính phải có tên máy, IP, màu nền và icon trạng thái tương ứng. 2. Đảm bảo hỗ trợ người dùng mù màu: không chỉ dựa vào màu sắc mà phải kèm chữ viết (ONLINE, WARNING, OFFLINE). 3. Nhấp đúp vào ô máy tính để mở cửa sổ chi tiết (Device Detail) hiển thị CPU, RAM và các dịch vụ cổng đang chạy.

• Minh chứng & Vấn đáp:
Tệp minh chứng: Ảnh chụp màn hình Lab View 20 máy trạm ở các trạng thái khác nhau
Câu hỏi thi chính: Thiết kế giao diện hỗ trợ tiếp cận (Accessibility) trong phần mềm quản trị? Không chỉ dùng màu sắc đơn thuần để chỉ báo trạng thái; luôn đi kèm biểu tượng hình học và nhãn chữ để đảm bảo khả năng tiếp cận

Thẻ #08: [LG-C-008] — INCIDENT LIST/DETAIL VÀ ACTION BINDING (Tuần: W3 | P0)

• **Mục tiêu cụ thể:** Hoàn thành IncidentListView.cs & IncidentActionPanel.cs đáp ứng tiêu chuẩn INCIDENT_ACK, INCIDENT_START, INCIDENT_RESOLVE (incidentId, expectedRowVersion, commandId)

• **Trực giác đời thường:** *Giống như bảng phiếu điều phối sửa chữa: Bấm nút Tiếp nhận hoặc Bắt đầu sửa, có dán tem phiên bản để tránh hai người cùng sửa một việc (Incident action binding & expectedRowVersion).*

• **Điều kiện:** Cần xong trước: LG-C-006 | Phối hợp: LG-B-009

• **4 Việc cần làm:**
1. Thiết kế bảng danh sách sự cố với các cột: Mã máy, Tên sự cố, Mức độ nghiêm trọng, Trạng thái, Thời gian phát sinh, Người phụ trách.
2. Tạo các nút hành động: Tiếp nhận (ACK), Bắt đầu xử lý (START), Giải quyết xong (RESOLVE).
3. Khi bấm nút, trích xuất incidentId và expectedRowVersion hiện tại, đóng gói vào bản tin tương ứng gửi lên Central Server.
4. Nếu nhận được phản hồi ERR_CONCURRENCY_VIOLATION từ Server, hiển thị thông báo: 'Sự cố đã được người khác cập nhật!' và tự động tải lại dữ liệu mới nhất.

📌 📋 PROMPT COPY NHANH CHO AI (LG-C-008)

Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai C).
Tôi đang làm nhiệm vụ LG-C-008 của LabGuard: 'Incident list/detail và action binding'.
Mục tiêu cốt lõi: IncidentListView.cs & IncidentActionPanel.cs.
Ví dụ trực giác: Giống như bảng phiếu điều phối sửa chữa: Bấm nút Tiếp nhận hoặc Bắt đầu sửa, có dán tem phiên bản để tránh hai người cùng sửa một việc (Incident action binding & expectedRowVersion).
ĐẶC BIỆT LƯU Ý: Vòng đời sự cố hoàn chỉnh gồm 5 trạng thái WorkflowState: OPEN -> ACKNOWLEDGED -> IN_PROGRESS -> RECOVERED -> RESOLVED (trong đó RECOVERED là trạng thái hệ thống đã phục hồi kỹ thuật, làm điều kiện cần trước khi Admin/Operator bấm RESOLVE để đóng sự cố). Tuân thủ nghiêm ngặt 5 trạng thái chuẩn của hệ thống. Mã lỗi xung đột OCC là ERR_CONCURRENCY_VIOLATION.
Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục UI.
3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
5. Hướng dẫn tôi cách chạy test 'TEST-C08' để tự mắt thấy kết quả đúng.
6. Nhắc tôi lưu tệp minh chứng: Incident view screenshot, command request trace log.
7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

📌 🤔 PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)

- Bản chất kỹ thuật của INCIDENT_ACK, INCIDENT_START, INCIDENT_RESOLVE (incidentId, expectedRowVersion, commandId) là gì?

> - Tại sao cần xử lý tình huống lỗi: Người dùng bấm đúp nút gửi 2 lệnh liên tiếp; vô hiệu hóa nút ngay khi bấm và sử dụng commandId duy nhất?
> - Vị trí tệp và dòng code minh chứng cụ thể trong UI?

• **Cách chạy test:**
Lệnh/Thao tác: Chạy bài test TEST-C08 (Bấm nút tiếp nhận (ACK) và bắt đầu xử lý (START); đối soát command gửi trên mạng)
Kết quả ĐÚNG: Gửi đúng format command với expectedRowVersion hiện tại; vô hiệu hóa nút trong khi chờ phản hồi để tránh double click

• **Nếu gặp lỗi (3 bước kiểm tra cụ thể):**
1. Nạp danh sách sự cố từ INCIDENT_PAGE, hiển thị rõ mã sự cố, loại sự cố, thời gian và trạng thái hiện tại. 2. Khi người dùng bấm nút [TIẾP NHẬN] (INCIDENT_ACK), gửi kèm expectedRowVersion lấy từ dữ liệu hiện hành. 3. Vô hiệu hóa nút bấm ngay sau khi click để ngăn người dùng bấm đúp gửi 2 lệnh liên tiếp.

• **Minh chứng & Vấn đáp:**
Tệp minh chứng: Incident view screenshot, command request trace log
Câu hỏi thi chính: Gắn kết thao tác người dùng với cơ chế chống tranh chấp OCC? Lấy rowversion hiện có của dòng sự cố đưa vào bản tin lệnh; nếu bản ghi đã bị người khác sửa đổi thì server sẽ báo lỗi tranh chấp

Thẻ #09: [LG-C-009] — RBAC UI UX VÀ SESSION LIFECYCLE CLIENT-SIDE (Tuần: W3 | P0)

• **Mục tiêu cụ thể:** Hoàn thành RbacUiManager.cs đáp ứng tiêu chuẩn Role-based UI visibility: Viewer (chỉ xem), Operator (ACK, START, RESOLVE), Admin (toàn quyền kèm ASSIGN, User/Invite)

• **Trực giác đời thường:** *Giống như thẻ ra vào phân quyền: Khách xem chỉ được nhìn qua kính, kỹ thuật viên trực ban được bấm nút sửa chữa, quản trị viên có thêm chìa khóa phân công và tạo mã mời (RBAC UI UX & session lifecycle).*

• **Điều kiện:** Cần xong trước: LG-C-003, LG-C-008 | Phối hợp: LG-B-004

• **4 Việc cần làm:**
1. Xây dựng phương thức ApplyRbacPermissions(UserRole role) duyệt qua các thành phần trên màn hình.
2. Nếu vai trò là Viewer: Ẩn hoặc vô hiệu hóa các nút ACK, START, RESOLVE, ASSIGN và menu cấu hình.
3. Nếu vai trò là Operator: Kích hoạt các nút xử lý sự cố (ACK, START, RESOLVE), ẩn menu tạo tài khoản và cấu hình hệ thống.
4. Nếu vai trò là Admin: Hiển thị đầy đủ mọi chức năng bao gồm nút ASSIGN và quản lý mã mời (Invite Code).

> 📌 📋 **PROMPT COPY NHANH CHO AI (LG-C-009)**

Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai C).
Tôi đang làm nhiệm vụ LG-C-009 của LabGuard: 'RBAC UI UX và session lifecycle client-side'.
Mục tiêu cốt lõi: RbacUiManager.cs.
Ví dụ trực giác: Giống như thẻ ra vào phân quyền: Khách xem chỉ được nhìn qua kính, kỹ thuật viên trực ban được bấm nút sửa chữa, quản trị viên có thêm chìa khóa phân công và tạo mã mời (RBAC UI UX & session lifecycle).
Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục UI.
3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
5. Hướng dẫn tôi cách chạy test 'TEST-C09' để tự mắt thấy kết quả đúng.
6. Nhắc tôi lưu tệp minh chứng: Ảnh chụp giao diện của 3 vai trò người dùng khác nhau.
7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

📌 🧠 PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)

- Bản chất kỹ thuật của Role-based UI visibility: Viewer (chỉ xem), Operator (ACK, START, RESOLVE), Admin (toàn quyền kèm ASSIGN, User/Invite) là gì?
- Tại sao cần xử lý tình huống lỗi: Người dùng nhầm lẫn quyền trên UI; ghi rõ vai trò hiện tại trên thanh trạng thái của ứng dụng?
- Vị trí tệp và dòng code minh chứng cụ thể trong UI?

• Cách chạy test:

Lệnh/Thao tác: Chạy bài test TEST-C09 (Đăng nhập lần lượt bằng 3 tài khoản với 3 vai trò khác nhau; kiểm tra hiển thị nút)
Kết quả ĐÚNG: Viewer không nhìn thấy các nút thay đổi trạng thái; Operator thấy nút ACK/START/RESOLVE; Admin thấy thêm nút ASSIGN và Invite/User Management

• Nếu gặp lỗi (3 bước kiểm tra cụ thể):

1. Đọc vai trò người dùng từ session: Viewer, Operator hoặc Admin; hiển thị vai trò rõ ràng trên thanh trạng thái. 2. Ẩn hoặc disable các nút mutation (Tiếp nhận, Bắt đầu, Giải quyết) đối với tài khoản vai trò Viewer. 3. Xử lý bản tin LOGOUT khi người dùng bấm Đăng xuất: xóa sessionToken và quay về màn hình LoginForm.

• Minh chứng & Vấn đáp:

Tệp minh chứng: Ảnh chụp giao diện của 3 vai trò người dùng khác nhau
Câu hỏi thi chính: Phân định giữa UX Optimization trên Client và Server Authorization? Ẩn nút trên UI chỉ giúp người dùng không thao tác nhầm; việc xác thực quyền thực sự bắt buộc phải được thực thi tại Central Server

Thẻ #10: [LG-C-010] — HISTORY/REPORT/HEALTH VIEWS (Tuần: W3 | P0)

• **Mục tiêu cụ thể:** Hoàn thành ReportViewerForm.cs & HealthViewDialog.cs đáp ứng tiêu chuẩn GET_HISTORY, GET_METRICS, CSV Export utility

• **Trực giác đời thường:** *Giống như xuất sổ cái nhật ký công tác ra bảng tính Excel CSV có dấu tiếng Việt chuẩn để in nộp báo cáo ca trực (History/Report/Health views, UTF-8 BOM).*

• **Điều kiện:** Cần xong trước: LG-C-004 | Phối hợp: LG-B-011

• **4 Việc cần làm:**

1. Thiết kế ReportViewerForm cho phép lọc sự cố theo khoảng ngày, theo máy trạm và theo trạng thái.
2. Gửi bản tin GET_HISTORY lên máy chủ và hiển thị kết quả phân trang trên DataGridView.
3. Viết tính năng xuất danh sách sự cố ra file .csv với định dạng mã hóa UTF-8 with BOM (để mở bằng Excel không lỗi tiếng Việt).
4. Thiết kế HealthViewDialog hiển thị các thông số nhịp thở của Central Server: Số kết nối, Bộ đệm, Thời gian chạy liên tục (Uptime).

📌 📋 PROMPT COPY NHANH CHO AI (LG-C-010)

Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai C).
Tôi đang làm nhiệm vụ LG-C-010 của LabGuard: 'History/Report/Health views'.
Mục tiêu cốt lõi: ReportViewerForm.cs & HealthViewDialog.cs.
Ví dụ trực giác: Giống như xuất sổ cái nhật ký công tác ra bảng tính Excel CSV có dấu tiếng Việt chuẩn để in nộp báo cáo ca trực (History/Report/Health views, UTF-8 BOM).
Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục UI.
3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
5. Hướng dẫn tôi cách chạy test 'TEST-C10' để tự mắt thấy kết quả đúng.
6. Nhắc tôi lưu tệp minh chứng: File CSV báo cáo mẫu, ảnh chụp màn hình form báo cáo.
7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

📌 🧠 PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)

- Bản chất kỹ thuật của GET_HISTORY, GET_METRICS, CSV Export utility là gì?
- Tại sao cần xử lý tình huống lỗi: Xuất file CSV bị lỗi font chữ tiếng Việt; ghi kèm UTF-8 BOM byte (0xEF, 0xBB, 0xBF) ở đầu tệp?
- Vị trí tệp và dòng code minh chứng cụ thể trong UI?

• **Cách chạy test:**

Lệnh/Thao tác: Chạy bài test TEST-C10 (Thực hiện truy vấn sự cố 24h gần nhất và bấm xuất file CSV)
Kết quả ĐÚNG: Dữ liệu phân trang hiển thị đầy đủ; file CSV xuất ra mở được trên Excel với đúng bảng mã tiếng Việt UTF-8 BOM

• **Nếu gặp lỗi (3 bước kiểm tra cụ thể):**
1. Xử lý hiển thị màn hình báo cáo: gửi GET_REPORT và nạp số liệu từ REPORT_RESULT vào biểu đồ. 2. Khi xuất file CSV lịch sử sự cố: ghi 3 byte UTF-8 BOM (0xEF, 0xBB, 0xBF) ở đầu tệp để Excel mở đúng tiếng Việt. 3. Mở file CSV vừa xuất bằng Microsoft Excel kiểm tra dấu tiếng Việt không bị biến thành ký tự lạ.

• **Minh chứng & Vấn đáp:**
Tệp minh chứng: File CSV báo cáo mẫu, ảnh chụp màn hình form báo cáo
Câu hỏi thi chính: Kỹ thuật xuất báo cáo dữ liệu định dạng CSV hỗ trợ tiếng Việt? Bổ sung UTF-8 BOM để các ứng dụng văn phòng tự động nhận diện đúng bảng mã Unicode, tránh lỗi vỡ ký tự

Thẻ #11: [LG-C-011] — MVP UI INTEGRATION VÀ ACCESSIBILITY (Tuần: W3 | P0)

• **Mục tiêu cụ thể:** Hoàn thành UiIntegrationReport.md đáp ứng tiêu chuẩn Toàn bộ giao diện người dùng WinForms MVP

• **Trực giác đời thường:** *Giống như kiểm tra công thái học: Các nút bấm nhạy, phím Tab nhảy đúng thứ tự, phóng to chữ không bị tràn khung hình (MVP UI integration & accessibility).*

• **Điều kiện:** Cần xong trước: LG-C-007, LG-C-010 | Phối hợp: LG-A-012, LG-B-012

• **4 Việc cần làm:**
1. Rà soát thứ tự TabIndex trên tất cả các màn hình, đảm bảo người dùng có thể điều hướng bằng phím Tab thuận tiện.
2. Gắn các phím tắt tiện dụng (ví dụ F5 để làm mới, Enter để đăng nhập, Esc để đóng hộp thoại).
3. Kiểm tra độ tương phản màu sắc và kích cỡ font chữ, đảm bảo dễ đọc trong môi trường ánh sáng phòng máy.
4. Chạy thử luồng nghiệp vụ hoàn chỉnh: Đăng nhập -> Nhìn máy trạm đổi màu -> Bấm xử lý sự cố -> Xem báo cáo.

> 📌 📋 **PROMPT COPY NHANH CHO AI (LG-C-011)**
> Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai C).
> Tôi đang làm nhiệm vụ LG-C-011 của LabGuard: 'MVP UI integration và accessibility'.
> Mục tiêu cốt lõi: UiIntegrationReport.md.
> Ví dụ trực giác: Giống như kiểm tra công thái học: Các nút bấm nhạy, phím Tab nhảy đúng thứ tự, phóng to chữ không bị tràn khung hình (MVP UI integration & accessibility).
> Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
> 1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
> 2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục Integration.
> 3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
> 4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
> 5. Hướng dẫn tôi cách chạy test 'TEST-C11' để tự mắt thấy kết quả đúng.
> 6. Nhắc tôi lưu tệp minh chứng: Checklist kiểm tra TabIndex và phím tắt, ảnh chụp giao diện.

> 7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

> 📌 🤔 **PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)**
> - Bản chất kỹ thuật của Toàn bộ giao diện người dùng WinForms MVP là gì?
> - Tại sao cần xử lý tình huống lỗi: Con trỏ nhảy lộn xộn khi bấm Tab; chuẩn hóa lại thuộc tính TabIndex trên từng control trong form designer?
> - Vị trí tệp và dòng code minh chứng cụ thể trong Integration?

• **Cách chạy test:**
Lệnh/Thao tác: Chạy bài test TEST-C11 (Điều hướng toàn bộ ứng dụng chỉ bằng bàn phím (Tab, Enter, Escape, phím mũi tên))
Kết quả ĐÚNG: Thứ tự TabIndex tuần tự hợp lý; không có trường dữ liệu nào bị kẹt phím; các nút chính có phím nóng

• **Nếu gặp lỗi (3 bước kiểm tra cụ thể):**
1. Nhấn phím Tab liên tục trên giao diện để kiểm tra thứ tự con trỏ nhảy giữa các ô nhập liệu và nút bấm (TabIndex). 2. Chạy ứng dụng trên màn hình có tỷ lệ phóng đại (DPI) 125% hoặc 150%: kiểm tra không bị cắt cụt chữ hoặc nút bấm. 3. Kiểm tra các hộp thoại thông báo lỗi: dùng ngôn ngữ rõ ràng, dễ hiểu, tránh hiện exception thô cho người dùng.

• **Minh chứng & Vấn đáp:**
Tệp minh chứng: Checklist kiểm tra TabIndex và phím tắt, ảnh chụp giao diện
Câu hỏi thi chính: Tiêu chuẩn điều hướng bàn phím trong ứng dụng Desktop chuyên nghiệp? Giúp người vận hành thao tác nhanh chóng trong phòng máy mà không cần phụ thuộc hoàn toàn vào chuột

Thẻ #12: [LG-C-012] — MVP REVIEW VÀ P1 GO/DEFER GATE (Tuần: W3 | P0/Gate)

• **Mục tiêu cụ thể:** Hoàn thành M4_Gate_Review.md đáp ứng tiêu chuẩn Tiêu chí nghiệm thu cổng chất lượng M4

• **Trực giác đời thường:** *Giống như trạm cân kiểm định trước khi lên đèo: Quyết định đi tiếp cung đường mở rộng hay dừng lại trau chuốt hệ thống cốt lõi để bảo đảm an toàn về đích (MVP review & P1 GO/DEFER gate).*

• **Điều kiện:** Cần xong trước: LG-C-011 | Phối hợp: LG-A-012, LG-B-012

• **4 Việc cần làm:**
1. Tổ chức cuộc họp đánh giá kỹ thuật 3 bên giữa A, B và C vào cuối Tuần 3.
2. Kiểm tra 3 tiêu chí bắt buộc để được phép làm P1: Không có lỗi P0 tồn đọng, kiểm thử đồng thời đạt 100%, mỗi thành viên còn ít nhất 15 giờ dự phòng.

3. Nếu không thỏa mãn cả 3 điều kiện: Kích hoạt ngay quyết định DEFER P1 theo đúng kế hoạch dự phòng đã cam kết.
4. Ghi nhận phân công tuần 4 chuyển sang tập trung vào: Tinh chỉnh hiệu năng, kiểm thử ổn định mạng LAN và chuẩn bị hồ sơ minh chứng.

📌 📋 PROMPT COPY NHANH CHO AI (LG-C-012)

Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai C).
Tôi đang làm nhiệm vụ LG-C-012 của LabGuard: 'MVP review và P1 GO/DEFER gate'.
Mục tiêu cốt lõi: M4_Gate_Review.md.
Ví dụ trực giác: Giống như trạm cân kiểm định trước khi lên đèo: Quyết định đi tiếp cung đường mở rộng hay dừng lại trau chuốt hệ thống cốt lõi để bảo đảm an toàn về đích (MVP review & P1 GO/DEFER gate).
Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục Integration.
3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
5. Hướng dẫn tôi cách chạy test 'TEST-C12' để tự mắt thấy kết quả đúng.
6. Nhắc tôi lưu tệp minh chứng: M4_Gate_Review.md có chữ ký xác nhận của 3 thành viên.
7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

📌 🧠 PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)

- Bản chất kỹ thuật của Tiêu chí nghiệm thu cổng chất lượng M4 là gì?
- Tại sao cần xử lý tình huống lỗi: Còn lỗi Sev1 chưa giải quyết; tự động kích hoạt DEFER P1 để dành toàn bộ thời gian tuần 4 ổn định core?
- Vị trí tệp và dòng code minh chứng cụ thể trong Integration?

• Cách chạy test:
Lệnh/Thao tác: Chạy bài test TEST-C12 (Đối chiếu tình trạng thực tế của MVP với checklist M4 Gate)
Kết quả ĐÚNG: 0 lỗi Sev1/Sev2; cả 3 ứng dụng P0 chạy ổn định trên mạng LAN thật; có biên bản ký nhận thống nhất của cả 3 thành viên

• Nếu gặp lỗi (3 bước kiểm tra cụ thể):
1. Họp nhóm kiểm kê tiến độ cuối Tuần 3: đánh giá danh sách lỗi Sev1 (lỗi nghiêm trọng) còn tồn đọng. 2. Nếu có lỗi Sev1 chưa giải quyết hoặc tính năng cốt lõi chưa mượt: kích hoạt ngay quyết định DEFER P1. 3. Lập biên bản cuộc họp M4 Gate Review ghi rõ quyết định của cả nhóm và ký xác nhận.

• Minh chứng & Vấn đáp:
Tệp minh chứng: M4_Gate_Review.md có chữ ký xác nhận của 3 thành viên
Câu hỏi thi chính: Quy trình quản lý chất lượng phần mềm và cổng quyết định (Stage-Gate)? Thiết lập các mốc kiểm soát rõ ràng; chỉ mở rộng tính năng mới khi nền tảng cốt lõi đã đạt độ tin cậy tuyệt đối

Thẻ #13: [LG-C-013] — COLLECTOR AGENT-FACING HOST (BINARY DÙNG CHUNG) (Tuần: W4 | P1)

• **Mục tiêu cụ thể:** Hoàn thành LabGuard.Collector.exe & AgentListener.cs đáp ứng tiêu chuẩn Collector TCP server socket, MaxConcurrentConnections = 100

• **Trực giác đời thường:** *Giống như đặt một trạm bảo vệ trung gian ở cửa phòng máy để gom các báo cáo của học sinh trước khi gửi về văn phòng trường (Collector Agent-facing host binary dùng chung [P1]).*

• **Điều kiện:** Cần xong trước: M4 Gate GO, LG-C-012 | Phối hợp: LG-A-013

• **4 Việc cần làm:**
1. Tạo dự án mới LabGuard.Collector dạng Console Service trên .NET 9.
2. Khởi tạo TcpListener lắng nghe kết nối từ các máy trạm trong dải mạng nội bộ phòng máy.
3. Giới hạn số lượng kết nối tối đa MaxConcurrentConnections = 100 để bảo vệ tài nguyên.
4. Bàn giao từng kết nối máy trạm cho bộ đọc FrameCodec để bóc tách gói tin an toàn.

📌 📋 PROMPT COPY NHANH CHO AI (LG-C-013)

Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai C).
Tôi đang làm nhiệm vụ LG-C-013 của LabGuard: 'Collector Agent-facing host (binary dùng chung)'.
Mục tiêu cốt lõi: LabGuard.Collector.exe & AgentListener.cs.
Ví dụ trực giác: Giống như đặt một trạm bảo vệ trung gian ở cửa phòng máy để gom các báo cáo của học sinh trước khi gửi về văn phòng trường (Collector Agent-facing host binary dùng chung [P1]).
Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục Networking.
3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
5. Hướng dẫn tôi cách chạy test 'TEST-C13' để tự mắt thấy kết quả đúng.
6. Nhắc tôi lưu tệp minh chứng: Collector startup console log, cấu hình appsettings mẫu.
7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

📌 🧠 PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)

- Bản chất kỹ thuật của Collector TCP server socket, MaxConcurrentConnections = 100 là gì?
- Tại sao cần xử lý tình huống lỗi: Quá tải số lượng kết nối; từ chối bớt kết nối khi vượt ngưỡng và báo NotReady về Gateway?
- Vị trí tệp và dòng code minh chứng cụ thể trong Networking?

• **Cách chạy test:**
Lệnh/Thao tác: Chạy bài test TEST-C13 (Khởi chạy ứng dụng Collector trên máy trạm; cho Agent kết nối thử nghiệm)
Kết quả ĐÚNG: Chấp nhận kết nối từ Agent, đọc và phân khung FrameCodec đúng chuẩn giao thức

• **Nếu gặp lỗi (3 bước kiểm tra cụ thể):**
1. [P1] Đóng gói Collector thành ứng dụng chạy độc lập (LabGuard.Collector.exe) dùng chung thư viện Shared.Protocol. 2. Kiểm tra cổng lắng nghe của Collector: cấu hình lắng nghe trên cổng 5002 để nhận kết nối từ các Agent phòng máy. 3. Bắt SocketException khi cổng đã bị ứng dụng khác chiếm dụng và hiển thị thông báo hướng dẫn đổi cổng.

• **Minh chứng & Vấn đáp:**
Tệp minh chứng: Collector startup console log, cấu hình appsettings mẫu
Câu hỏi thi chính: Kiến trúc ứng dụng Collector dùng chung cho nhiều phiên bản triển khai? Một file thực thi duy nhất nhận tham số cấu hình (Port, UpstreamHost, NodeId) qua command line hoặc appsettings để chạy thành nhiều instance độc lập

Thẻ #14: [LG-C-014] — COLLECTOR UPSTREAM MULTIPLEX VÀ REPLY ROUTER (Tuần: W4 | P1)

• **Mục tiêu cụ thể:** Hoàn thành UpstreamMultiplexer.cs & ReplyRouter.cs đáp ứng tiêu chuẩn Single upstream TCP socket to Central, routing table clientConnectionId <-> socket

• **Trực giác đời thường:** *Giống như tổng đài điện thoại nội bộ: Gom 20 cuộc gọi của học sinh thành 1 đường dây về trường, khi trường trả lời thì chuyển đúng về máy nhánh (Collector upstream multiplex & reply router [P1]).*

• **Điều kiện:** Cần xong trước: LG-C-013 | Phối hợp: LG-B-013

• **4 Việc cần làm:**
1. Duy trì kết nối TCP/TLS duy nhất từ Collector tới cổng nội bộ của Central Server.
2. Gán một mã định danh kết nối (clientConnectionId kiểu số nguyên) duy nhất cho mỗi máy trạm kết nối vào.
3. Đóng gói các gói tin của Agent vào bản tin gom FORWARD_DATA mang kèm clientConnectionId gửi lên Central.
4. Khi Central gửi phản hồi về qua kết nối upstream, đọc clientConnectionId và tra bảng ánh xạ để chuyển tiếp đúng socket của Agent tương ứng.

> 📌 📋 **PROMPT COPY NHANH CHO AI (LG-C-014)**
> Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai C).
> Tôi đang làm nhiệm vụ LG-C-014 của LabGuard: 'Collector upstream multiplex và reply router'.
> Mục tiêu cốt lõi: UpstreamMultiplexer.cs & ReplyRouter.cs.
> Ví dụ trực giác: Giống như tổng đài điện thoại nội bộ: Gom 20 cuộc gọi của học sinh thành 1 đường dây về trường, khi trường trả lời thì chuyển đúng về máy nhánh (Collector upstream multiplex & reply router [P1]).
> Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
> 1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
> 2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục Networking.
> 3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.

4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
5. Hướng dẫn tôi cách chạy test 'TEST-C14' để tự mắt thấy kết quả đúng.
6. Nhắc tôi lưu tệp minh chứng: Multiplexing trace log, bảng định tuyến router.
7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

📌 🤔 **PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)**
- Bản chất kỹ thuật của Single upstream TCP socket to Central, routing table clientConnectionId <-> socket là gì?
- Tại sao cần xử lý tình huống lỗi: Một Agent bị đứt kết nối trong khi ACK đang trên đường về; kiểm tra socket còn sống trước khi ghi trả ACK?
- Vị trí tệp và dòng code minh chứng cụ thể trong Networking?

• **Cách chạy test:**
Lệnh/Thao tác: Chạy bài test TEST-C14 (Kết nối 10 Agent vào Collector; kiểm tra luồng tin gửi về Central và ACK trả ngược lại)
Kết quả ĐÚNG: Dữ liệu được dồn kênh chính xác; 10 Agent nhận đúng bản tin ACK tương ứng mà không bị nhầm lẫn

• **Nếu gặp lỗi (3 bước kiểm tra cụ thể):**
1. [P1] Kiểm tra cơ chế ghép kênh (Multiplexing): nhiều kết nối Agent từ phòng máy được gom qua một kênh duy nhất về Central. 2. Quản lý bảng định tuyến phản hồi (ReplyRouter): khi nhận BATCH_ACK từ Central, chuyển đúng về socket của Agent gửi ban đầu. 3. Xử lý trường hợp Agent ngắt kết nối trước khi nhận được ACK: dọn dẹp bộ nhớ định tuyến an toàn.

• **Minh chứng & Vấn đáp:**
Tệp minh chứng: Multiplexing trace log, bảng định tuyến router
Câu hỏi thi chính: Kỹ thuật dồn luồng và bảng định tuyến phản hồi trong Proxy trung chuyển? Gắn nhãn định danh kết nối (ConnectionId) vào gói tin gửi upstream; khi nhận ACK sẽ tra bảng định tuyến để trả lời đúng socket client

Thẻ #15: [LG-C-015] — COLLECTOR READINESS/HEALTH VÀ BOUNDED BUFFERS (Tuần: W4 | P1)

• **Mục tiêu cụ thể:** Hoàn thành CollectorHealthService.cs & BoundedForwardBuffer.cs đáp ứng tiêu chuẩn TCP Health Port cho Gateway probe, Bounded Channel buffer

• **Trực giác đời thường:** *Giống như bể điều hòa áp lực: Nếu đường truyền về trường bị nghẽn thì chặn bớt học sinh gửi bài để không làm tràn vỡ bể chứa trung gian (Collector readiness/health & bounded buffers [P1]).*

• **Điều kiện:** Cần xong trước: LG-C-014 | Phối hợp: LG-A-014

• 4 Việc cần làm:

1. Khởi tạo Bounded Channel với dung lượng 2000 phần tử để đệm các gói tin chờ gửi về Central.
2. Cấu hình chính sách khi đầy đệm: Chặn tạm thời luồng đọc của Agent (áp lực ngược - Backpressure) để Agent tự đệm ở local.
3. Mở một cổng TCP phụ phục vụ kiểm tra sức khỏe (Health Probe): trả về chuỗi 'OK' kèm số lượng kết nối hiện hữu.
4. Ghi nhận nhật ký cảnh báo khi hàng đợi vượt quá 80% dung lượng.

📌 📋 PROMPT COPY NHANH CHO AI (LG-C-015)

Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai C).
Tôi đang làm nhiệm vụ LG-C-015 của LabGuard: 'Collector readiness/health và bounded buffers'.
Mục tiêu cốt lõi: CollectorHealthService.cs & BoundedForwardBuffer.cs.
Ví dụ trực giác: Giống như bể điều hòa áp lực: Nếu đường truyền về trường bị nghẽn thì chặn bớt học sinh gửi bài để không làm tràn vỡ bể chứa trung gian (Collector readiness/health & bounded buffers [P1]).
Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục Networking.
3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
5. Hướng dẫn tôi cách chạy test 'TEST-C15' để tự mắt thấy kết quả đúng.
6. Nhắc tôi lưu tệp minh chứng: Collector health status log, buffer memory usage graph.
7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

📌 🧠 PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)

- Bản chất kỹ thuật của TCP Health Port cho Gateway probe, Bounded Channel buffer là gì?
- Tại sao cần xử lý tình huống lỗi: Nghẽn mạng upstream làm tràn RAM máy chủ trung chuyển; kích hoạt backpressure làm chậm tốc độ đọc từ Agent?
- Vị trí tệp và dòng code minh chứng cụ thể trong Networking?

• Cách chạy test:

Lệnh/Thao tác: Chạy bài test TEST-C15 (Mô phỏng ngắt kết nối upstream tới Central; kiểm tra phản ứng của buffer và health probe)
Kết quả ĐÚNG: Hàng đợi buffer không vượt ngưỡng dung lượng; cổng health báo NotReady tới Gateway để tạm dừng tiếp nhận

• Nếu gặp lỗi (3 bước kiểm tra cụ thể):

1. [P1] Định kỳ gửi COLLECTOR_HEARTBEAT về Central Server báo cáo số lượng client đang kết nối và tình trạng tải. 2. Kiểm tra hàng đợi đệm forward (BoundedForwardBuffer): nếu upstream bị chậm, áp dụng backpressure làm chậm việc nhận tin từ Agent. 3. Phản hồi bản tin HEALTH_RESULT khi nhận được gói HEALTH_PROBE từ Gateway.

• **Minh chứng & Vấn đáp:**
Tệp minh chứng: Collector health status log, buffer memory usage graph
Câu hỏi thi chính: Cơ chế kiểm tra độ sẵn sàng (Readiness Probe) và điều tiết áp lực ngược (Backpressure)? Chỉ nhận tải khi downstream và upstream đều sẵn sàng; tự động kích hoạt hàng rào bảo vệ khi bộ đệm đạt ngưỡng giới hạn

Thẻ #16: [LG-C-016] — MULTI-ADMIN CONCURRENCY VÀ RACE CONDITION TESTS (Tuần: W4 | P0/Stability)

• **Mục tiêu cụ thể:** Hoàn thành MultiAdminTestReport.md đáp ứng tiêu chuẩn 2 WinForms Admin clients, cùng thao tác trên 1 Ticket sự cố

• **Trực giác đời thường:** *Giống như thử nghiệm hai người cùng bấm mua chiếc vé xe cuối cùng trên mạng: Người đến sau được thông báo lịch sự rằng vé vừa được mua và giao diện tự làm mới (Multi-admin concurrency test & ERR_CONCURRENCY_VIOLATION handling).*

• **Điều kiện:** Cần xong trước: LG-C-008 | Phối hợp: LG-B-009

• **4 Việc cần làm:**
1. Khởi chạy 2 ứng dụng WinForms Admin trên 2 máy tính trong mạng LAN.
2. Kích hoạt một sự cố cảnh báo trên hệ thống để cả 2 màn hình cùng nhìn thấy.
3. Cán bộ 1 bấm nút ACK để tiếp nhận sự cố (thao tác thành công).
4. Cán bộ 2 ngay sau đó bấm nút ACK trên màn hình chưa kịp làm mới của mình.

> 📌 📋 **PROMPT COPY NHANH CHO AI (LG-C-016)**
> Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai C).
> Tôi đang làm nhiệm vụ LG-C-016 của LabGuard: 'Multi-admin concurrency và race condition tests'.
> Mục tiêu cốt lõi: MultiAdminTestReport.md.
> Ví dụ trực giác: Giống như thử nghiệm hai người cùng bấm mua chiếc vé xe cuối cùng trên mạng: Người đến sau được thông báo lịch sự rằng vé vừa được mua và giao diện tự làm mới (Multi-admin concurrency test & ERR_CONCURRENCY_VIOLATION handling).
> ĐẶC BIỆT LƯU Ý: Vòng đời sự cố hoàn chỉnh gồm 5 trạng thái WorkflowState: OPEN -> ACKNOWLEDGED -> IN_PROGRESS -> RECOVERED -> RESOLVED (trong đó RECOVERED là trạng thái hệ thống đã phục hồi kỹ thuật, làm điều kiện cần trước khi Admin/Operator bấm RESOLVE để đóng sự cố). Tuân thủ nghiêm ngặt 5 trạng thái chuẩn của hệ thống. Mã lỗi xung đột OCC là ERR_CONCURRENCY_VIOLATION.
> Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
> 1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
> 2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục Testing.
> 3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
> 4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
> 5. Hướng dẫn tôi cách chạy test 'TEST-C16' để tự mắt thấy kết quả đúng.
> 6. Nhắc tôi lưu tệp minh chứng: MultiAdminTestReport.md, video ghi lại thao tác xung đột.
> 7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

> 📌 🧠 **PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)**
> - Bản chất kỹ thuật của 2 WinForms Admin clients, cùng thao tác trên 1 Ticket sự cố là gì?
> - Tại sao cần xử lý tình huống lỗi: Ứng dụng bị crash hoặc hiện mã lỗi thô; hiển thị thông báo nghiệp vụ rõ ràng hướng dẫn người dùng làm mới dữ liệu?
> - Vị trí tệp và dòng code minh chứng cụ thể trong Testing?

• **Cách chạy test:**
Lệnh/Thao tác: Chạy bài test TEST-C16 (2 người cùng bấm nút Resolve trong cùng 1 giây)
Kết quả ĐÚNG: Người bấm trước thành công; người bấm sau nhận hộp thoại thông báo xung đột dữ liệu thân thiện và màn hình tự làm mới

• **Nếu gặp lỗi (3 bước kiểm tra cụ thể):**
1. Bật đồng thời 2 cửa sổ Admin Client trên 2 máy khác nhau, cùng mở xem một sự cố đang ở trạng thái OPEN. 2. Cả 2 người dùng cùng bấm nút [TIẾP NHẬN] trên cùng một sự cố: quan sát người thứ nhất thành công, người thứ hai nhận lỗi. 3. Kiểm tra thông báo trên máy thứ hai: ứng dụng phải bẫy lỗi ERR_CONCURRENCY_VIOLATION và tự tải lại trạng thái mới nhất.

• **Minh chứng & Vấn đáp:**
Tệp minh chứng: MultiAdminTestReport.md, video ghi lại thao tác xung đột
Câu hỏi thi chính: Trải nghiệm người dùng khi xảy ra xung đột tương tranh dữ liệu? Bắt mã lỗi xung đột OCC từ server và hiển thị hộp thoại thông báo dễ hiểu, tự động tải lại dữ liệu mới nhất cho người dùng

Thẻ #17: [LG-C-017] — UI RELEASE REGRESSION VÀ SETTINGS CLEANUP (Tuần: W5 | P0/Release)

• **Mục tiêu cụ thể:** Hoàn thành UiRegressionChecklist.md đáp ứng tiêu chuẩn High-DPI Per-Monitor V2 support trong WinForms

• **Trực giác đời thường:** *Giống như kiểm tra máy chiếu hội trường: Chữ và biểu đồ hiển thị sắc nét ở mọi độ phân giải màn hình trước ngày báo cáo chính thức (UI release regression & High-DPI settings).*

• **Điều kiện:** Cần xong trước: LG-C-011 | Phối hợp: None

• **4 Việc cần làm:**
1. Kiểm tra cấu hình PerMonitorV2 trong tệp manifest và mã khởi tạo Program.cs.
2. Thử nghiệm thay đổi độ thu phóng màn hình Windows từ 100% lên 125% và 150%, kiểm tra các icon và chữ viết vẫn sắc nét.
3. Rà soát lại toàn bộ thông báo tiếng Việt trên giao diện, đảm bảo chuẩn chính tả và đồng nhất thuật ngữ.
4. Xóa bỏ toàn bộ các chuỗi kết nối cứng (hard-coded connection strings) hoặc tài khoản thử nghiệm trong mã nguồn.

📌 📋 PROMPT COPY NHANH CHO AI (LG-C-017)

Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai C).
Tôi đang làm nhiệm vụ LG-C-017 của LabGuard: 'UI release regression và settings cleanup'.
Mục tiêu cốt lõi: UiRegressionChecklist.md.
Ví dụ trực giác: Giống như kiểm tra máy chiếu hội trường: Chữ và biểu đồ hiển thị sắc nét ở mọi độ phân giải màn hình trước ngày báo cáo chính thức (UI release regression & High-DPI settings).
Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục Testing.
3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
5. Hướng dẫn tôi cách chạy test 'TEST-C17' để tự mắt thấy kết quả đúng.
6. Nhắc tôi lưu tệp minh chứng: Ảnh chụp màn hình đối chiếu ở 3 tỷ lệ DPI khác nhau.
7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

📌 🤔 PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)

- Bản chất kỹ thuật của High-DPI Per-Monitor V2 support trong WinForms là gì?
- Tại sao cần xử lý tình huống lỗi: Lỗi co giãn DPI làm vỡ form giao diện; bật cấu hình HighDpiMode = SystemAware hoặc PerMonitorV2 trong Program.cs?
- Vị trí tệp và dòng code minh chứng cụ thể trong Testing?

• Cách chạy test:

Lệnh/Thao tác: Chạy bài test TEST-C17 (Chạy WinForms trên màn hình Full HD ở các mức scale 100%, 125% và 150%)
Kết quả ĐÚNG: Giao diện không bị mờ nhạt (blurry), chữ không bị cắt cụt, các icon tự co giãn sắc nét

• Nếu gặp lỗi (3 bước kiểm tra cụ thể):

1. Kiểm tra tệp Program.cs: bảo đảm đã gọi Application.SetHighDpiMode(HighDpiMode.PerMonitorV2) trước khi tạo form. 2. Xóa bỏ toàn bộ chuỗi kết nối và thông tin thử nghiệm hardcode trong mã nguồn, chuyển ra appsettings.json. 3. Kiểm tra bản build Release: đảm bảo không sinh ra các tệp pdb thừa và ứng dụng khởi động trong vòng dưới 3 giây.

• Minh chứng & Vấn đáp:

Tệp minh chứng: Ảnh chụp màn hình đối chiếu ở 3 tỷ lệ DPI khác nhau
Câu hỏi thi chính: Hỗ trợ màn hình độ phân giải cao (High DPI Scaling) trong WinForms? Cấu hình ứng dụng nhận biết DPI để hệ điều hành không tự động kéo dãn pixel làm mờ giao diện; thiết kế layout co giãn linh hoạt

Thẻ #18: [LG-C-018] — DEMO REHEARSAL VÀ BACKUP ASSETS (Tuần: W5 | P0/Demo)

• **Mục tiêu cụ thể:** Hoàn thành DemoScript.md & DemoBackup.mp4 đáp ứng tiêu chuẩn Kịch bản demo chuẩn gồm 6 bước nghiệp vụ xuyên suốt

• **Trực giác đời thường:** *Giống như diễn tập chương trình biểu diễn có bấm giờ từng phút, và chuẩn bị sẵn đĩa ghi âm dự phòng nếu micro bị trục trặc (12-minute demo script & DemoBackup.mp4).*

• **Điều kiện:** Cần xong trước: LG-C-017 | Phối hợp: LG-A-018

• **4 Việc cần làm:**
1. Soạn thảo kịch bản Demo 6 bước: Khởi động hệ thống -> Đăng nhập an toàn -> Giám sát phòng máy thời gian thực -> Giả lập sự cố quá tải CPU -> Tiếp nhận xử lý sự cố OCC -> Báo cáo thống kê & Khôi phục.
2. Phân vai rõ ràng: A phụ trách thao tác máy trạm và rút dây mạng; B giải thích cơ chế Server/CSDL; C điều khiển Dashboard và thuyết minh.
3. Quay video sắc nét toàn bộ quá trình demo từ đầu đến cuối, lưu vào USB và laptop cá nhân làm tài sản dự phòng.
4. Bấm giờ diễn tập thực tế đúng 12 phút, không để cháy giáo án.

📌 📋 PROMPT COPY NHANH CHO AI (LG-C-018)

Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai C).
Tôi đang làm nhiệm vụ LG-C-018 của LabGuard: 'Demo rehearsal và backup assets'.
Mục tiêu cốt lõi: DemoScript.md & DemoBackup.mp4.
Ví dụ trực giác: Giống như diễn tập chương trình biểu diễn có bấm giờ từng phút, và chuẩn bị sẵn đĩa ghi âm dự phòng nếu micro bị trục trặc (12-minute demo script & DemoBackup.mp4).
Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục UI.
3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
5. Hướng dẫn tôi cách chạy test 'TEST-C18' để tự mắt thấy kết quả đúng.
6. Nhắc tôi lưu tệp minh chứng: DemoScript.md, video demo backup chất lượng 1080p.
7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

📌 🤔 PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)

- Bản chất kỹ thuật của Kịch bản demo chuẩn gồm 6 bước nghiệp vụ xuyên suốt là gì?
- Tại sao cần xử lý tình huống lỗi: Mạng phòng thi gặp sự cố trong buổi báo cáo; lập tức chuyển sang video backup chất lượng cao để trình bày không bị gián đoạn?
- Vị trí tệp và dòng code minh chứng cụ thể trong UI?

• **Cách chạy test:**
Lệnh/Thao tác: Chạy bài test TEST-C18 (Chạy thử toàn bộ kịch bản demo theo đúng thời lượng quy định với 3 máy thật)
Kết quả ĐÚNG: Kịch bản demo trơn tru, làm nổi bật đầy đủ các yêu cầu môn học NT106; có video backup sẵn sàng

• **Nếu gặp lỗi (3 bước kiểm tra cụ thể):**
1. Bấm đồng hồ chạy thử kịch bản demo 12 phút đủ 6 bước: khởi động, liveness, quá tải CPU, xung đột OCC, rút dây mạng, báo cáo. 2. Kiểm tra tệp video dự phòng DemoBackup.mp4: độ phân giải 1080p, âm thanh thuyết minh rõ ràng, có phụ đề các mốc kỹ thuật. 3. Phân công rõ người thuyết minh và người thao tác bàn phím chuột để không bị lúng túng trong buổi báo cáo.

• **Minh chứng & Vấn đáp:**
Tệp minh chứng: DemoScript.md, video demo backup chất lượng 1080p
Câu hỏi thi chính: Kỹ năng lập kế hoạch demo và quản trị rủi ro khi báo cáo sản phẩm? Luôn chuẩn bị phương án dự phòng (video demo, dữ liệu mẫu, cấu hình offline) để đối phó với sự cố môi trường ngoài dự kiến

Thẻ #19: [LG-C-019] — RELEASE REVIEW CHECKLIST VÀ README (Tuần: W5 | P0/Release)

• **Mục tiêu cụ thể:** Hoàn thành README.md & RELEASE_CHECKLIST.md đáp ứng tiêu chuẩn Tài liệu hướng dẫn cài đặt chuẩn, yêu cầu môi trường .NET 9

• **Trực giác đời thường:** *Giống như tờ hướng dẫn lắp ráp đồ chơi đính kèm trong hộp: Người mua mở ra đọc 5 phút là biết cắm dây và vận hành ngay (Release review checklist & README).*

• **Điều kiện:** Cần xong trước: LG-C-018 | Phối hợp: LG-A-019, LG-B-019

• **4 Việc cần làm:**
1. Soạn thảo README.md gồm: Tổng quan sản phẩm, Yêu cầu cấu hình tối thiểu, Hướng dẫn cài đặt CSDL, Hướng dẫn chạy Central, Agent và Dashboard.
2. Đính kèm các lệnh chạy dòng lệnh mẫu cho cả Windows Terminal và PowerShell.
3. Kiểm tra danh mục phát hành: Mã nguồn không chứa tệp rác (.vs, bin, obj), các chứng chỉ mẫu và file cấu hình đã có sẵn.
4. Đóng gói bộ cài đặt hoặc tệp nén ZIP hoàn chỉnh theo đúng cấu trúc tiêu chuẩn UIT.

> 📌 📋 **PROMPT COPY NHANH CHO AI (LG-C-019)**
> Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai C).
> Tôi đang làm nhiệm vụ LG-C-019 của LabGuard: 'Release review checklist và README'.
> Mục tiêu cốt lõi: README.md & RELEASE_CHECKLIST.md.
> Ví dụ trực giác: Giống như tờ hướng dẫn lắp ráp đồ chơi đính kèm trong hộp: Người mua mở ra đọc 5 phút là biết cắm dây và vận hành ngay (Release review checklist & README).
> Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:

1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục System.
3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
5. Hướng dẫn tôi cách chạy test 'TEST-C19' để tự mắt thấy kết quả đúng.
6. Nhắc tôi lưu tệp minh chứng: README.md, RELEASE_CHECKLIST.md.
7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

📌 🧠 PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)
- Bản chất kỹ thuật của Tài liệu hướng dẫn cài đặt chuẩn, yêu cầu môi trường .NET 9 là gì?
- Tại sao cần xử lý tình huống lỗi: Thiếu các bước cấu hình kết nối CSDL hoặc chứng chỉ; bổ sung chi tiết từng bước có ảnh chụp minh họa?
- Vị trí tệp và dòng code minh chứng cụ thể trong System?

• **Cách chạy test:**
Lệnh/Thao tác: Chạy bài test TEST-C19 (Người ngoài nhóm làm theo hướng dẫn trong README để cài đặt và chạy thử)
Kết quả ĐÚNG: Cài đặt thành công trên máy tính mới trong vòng 10 phút; tài liệu rõ ràng, dễ hiểu

• **Nếu gặp lỗi (3 bước kiểm tra cụ thể):**
1. Đọc lại tệp README.md: tự đóng vai người ngoài làm theo từng bước cài đặt xem có chạy được ứng dụng không. 2. Kiểm tra các yêu cầu môi trường: .NET 9 SDK / Runtime, SQL Server 2022 / Express, quyền mở cổng tường lửa. 3. Rà soát checklist bàn giao sản phẩm: bảo đảm đủ bộ cài Agent, Server, Admin và cơ sở dữ liệu mẫu.

• **Minh chứng & Vấn đáp:**
Tệp minh chứng: README.md, RELEASE_CHECKLIST.md
Câu hỏi thi chính: Chuẩn hóa tài liệu chuyển giao và hướng dẫn triển khai sản phẩm? Tài liệu rõ ràng giúp người chấm bài hoặc kỹ sư tiếp quản có thể dễ dàng dựng môi trường và kiểm chứng tính đúng đắn

Thẻ #20: [LG-C-020] — VIVA C VÀ BÀN GIAO BACKUP (Tuần: W5 | P0/Viva)

• **Mục tiêu cụ thể:** Hoàn thành HANDBOOK_C.md & Viva rehearsal log đáp ứng tiêu chuẩn 30 câu hỏi viva chuyên sâu phần WinForms, Cache & Collector App

• **Trực giác đời thường:** *Giống như diễn tập bảo vệ đồ án: Nắm chắc nguyên lý điều phối luồng UI Invoke, cơ chế Snapshot và mã băm tương quan requestId để tự tin trả lời vấn đáp.*

• **Điều kiện:** Cần xong trước: LG-C-019 | Phối hợp: None

• **4 Việc cần làm:**
1. Ôn tập kỹ nguyên lý luồng STA trong WinForms, giải thích tại sao phải dùng Control.InvokeRequired và

hậu quả nếu vi phạm.
2. Nắm chắc cơ chế khớp cặp yêu cầu/phản hồi bằng TaskCompletionSource và ConcurrentDictionary.
3. Chuẩn bị đoạn code minh chứng cho việc bắt mã lỗi ERR_CONCURRENCY_VIOLATION và cập nhật lại giao diện người dùng.
4. Giải thích kiến trúc dồn kênh (Multiplexing) của Collector và cách định tuyến gói tin qua clientConnectionId.

> 📌 📋 **PROMPT COPY NHANH CHO AI (LG-C-020)**
> Tôi là sinh viên năm 2 đang làm đồ án NT106 bằng C# .NET 9 (Vai C).
> Tôi đang làm nhiệm vụ LG-C-020 của LabGuard: 'Viva C và bàn giao backup'.
> Mục tiêu cốt lõi: HANDBOOK_C.md & Viva rehearsal log.
> Ví dụ trực giác: Giống như diễn tập bảo vệ đồ án: Nắm chắc nguyên lý điều phối luồng UI Invoke, cơ chế Snapshot và mã băm tương quan requestId để tự tin trả lời vấn đáp.
> Hãy hướng dẫn tôi theo phong cách sinh viên năm 2:
> 1. Giải thích bản chất kỹ thuật một cách dễ hiểu nhất, dùng ví dụ đời thường.
> 2. Cho tôi biết cần tạo mới hoặc sửa file nào trong thư mục Viva.
> 3. Viết từng bước ngắn gọn, mỗi lần chỉ đưa tối đa 1 class hoặc 1 hàm.
> 4. Giải thích rõ ràng các từ khóa C# quan trọng (async, await, Task, using...).
> 5. Hướng dẫn tôi cách chạy test 'TEST-C20' để tự mắt thấy kết quả đúng.
> 6. Nhắc tôi lưu tệp minh chứng: HANDBOOK_C.md, biên bản diễn tập vấn đáp nội bộ.
> 7. Cuối cùng, hãy hỏi tôi 3 câu vấn đáp then chốt để chắc chắn tôi đã hiểu bản chất.

> 📌 🧠 **PHẦN BẠN PHẢI TỰ HIỂU (BẮT BUỘC ĐỂ THI VẤN ĐÁP)**
> - Bản chất kỹ thuật của 30 câu hỏi viva chuyên sâu phần WinForms, Cache & Collector App là gì?
> - Tại sao cần xử lý tình huống lỗi: Nhầm lẫn giữa luồng giao diện và luồng mạng; ôn tập kỹ sơ đồ marshaling trong handbook?
> - Vị trí tệp và dòng code minh chứng cụ thể trong Viva?

• Cách chạy test:
Lệnh/Thao tác: Chạy bài test TEST-C20 (Vấn đáp chéo giả định với thành viên A và B chấm điểm độc lập)
Kết quả ĐÚNG: Giải thích thành thạo các cơ chế UI thread, RPC correlation, snapshot cache và kiến trúc Collector

• Nếu gặp lỗi (3 bước kiểm tra cụ thể):
1. Ôn tập kỹ sơ đồ chuyển luồng giao diện: giải thích rõ tại sao không được truy cập WinForms control từ luồng mạng nền. 2. Mở file MainForm.cs chỉ đúng khối code sử dụng Control.Invoke / BeginInvoke để cập nhật giao diện. 3. Chuẩn bị trả lời câu hỏi: 'Snapshot Barrier hoạt động như thế nào và tại sao cần thiết trong hệ thống giám sát?'.

• Minh chứng & Vấn đáp:
Tệp minh chứng: HANDBOOK_C.md, biên bản diễn tập vấn đáp nội bộ
Câu hỏi thi chính: Tổng quan kiến trúc ứng dụng quản trị và điều phối giao diện của Lane C? Trình bày rõ

ràng cấu trúc ứng dụng WinForms, cơ chế nhận snapshot và delta event để duy trì tính nhất quán thời gian thực

PHẦN 6: KỊCH BẢN TEST MẠNG LAN THỰC TẾ & BẰNG CHỨNG NỘP BÀI

6.1. Hướng dẫn cắm dây và cấu hình IP phòng thi: Đồ án Lập trình mạng bắt buộc phải chạy trên ít nhất 2 đến 3 máy tính vật lý thật qua dây mạng LAN hoặc router Wi-Fi độc lập:

3 bước thiết lập nhanh: 1. Máy B (Máy chủ Central): Cắm dây mạng, mở CMD gõ ipconfig lấy địa chỉ IPv4 (ví dụ: 192.168.1.100). Bật Windows Defender Firewall cho phép cổng TCP 5000 và 5001.
2. Máy A (Agent) & Máy C (WinForms): Mở file appsettings.json, sửa dòng ServerIp thành 192.168.1.100.
3. Kiểm tra thông mạng: Đứng ở máy A/C mở CMD gõ ping 192.168.1.100. Nếu Reply from ... là thông mạng thành công!

6.2. 3 bài test LAN bắt buộc phải chụp ảnh làm minh chứng:

• Bài test 1 (Mã hóa TLS): Mở Wireshark trên máy A, bắt gói tin cổng 5000. Chụp ảnh màn hình thấy dòng TLSv1.3 Application Data (không thấy chuỗi JSON trần trụi) chứng minh dữ liệu được bảo mật.
• Bài test 2 (Rút dây mạng liveness 20s): Rút trực tiếp dây mạng máy A. Nhìn đồng hồ đúng 20 giây (> 20s không nhận heartbeat hợp lệ), ô vuông của máy A trên màn hình C tự động chuyển sang màu Xám (OFFLINE).
• Bài test 3 (Tranh chấp 2 Admin OCC): Bật 2 máy Admin cùng đăng nhập. Cả 2 cùng nhìn thấy 1 sự cố màu Đỏ. Cả 2 cùng bấm nút [TIẾP NHẬN] (gửi INCIDENT_ACK kèm expectedRowVersion). Người thứ hai nhận phản hồi mã lỗi ERR_CONCURRENCY_VIOLATION kèm thông báo: 'Sự cố đã được người khác xử lý!' chứng minh khóa lạc quan OCC hoạt động chuẩn xác!

PHẦN 7: 90 CÂU HỎI VẤN ĐÁP & CÔNG THỨC 3 BƯỚC ĐẠT KẾT QUẢ XUẤT SẮC

> 📌 **CÔNG THỨC 3 BƯỚC TRẢ LỜI VẤN ĐÁP ĐẠT KẾT QUẢ TỐT**
> • BƯỚC 1: Nêu bản chất lý thuyết ngắn gọn trong 1 câu (Tại sao cần nó, ví dụ đời thường).
> • BƯỚC 2: Nêu quyết định thiết kế của nhóm trong đồ án LabGuard (Dùng công nghệ gì, tham số bao nhiêu).
> • BƯỚC 3: Mở ngay Visual Studio, chỉ đúng tên file, tên hàm và dòng code hiện thực (Dẫn chứng thực tế).
> 👉 Trả lời đủ 3 bước này, hội đồng sẽ nhận thấy bạn tự tay viết code và hiểu sâu kiến trúc hệ thống!

30 câu hỏi trọng tâm — SINH VIÊN A (AGENT & MẠNG)

Câu #01: Tại sao TCP được gọi là giao thức hướng dòng byte (Byte Stream)?

• **Trả lời cốt lõi:** TCP truyền một chuỗi byte liên tục và bảo đảm tin cậy, có thứ tự; tuy nhiên TCP hoàn toàn không giữ ranh giới thông điệp (Message Boundary) của tầng ứng dụng. Một lần gửi có thể bị chia nhỏ hoặc dính liền khi nhận.

• **Vị trí code minh chứng:**

Câu #02: Hiện tượng phân mảnh (Partial Read) và dính gói (Frame Coalescing) trong TCP là gì?

• **Trả lời cốt lõi:** Partial Read là hiện tượng hàm `ReadAsync` chỉ đọc được một phần thông điệp do gói tin bị phân mảnh trên mạng hoặc bộ đệm socket chưa đầy; Frame Coalescing là hiện tượng nhiều thông điệp được gửi liên tiếp bị dính chùm vào cùng một bộ đệm đọc.

• **Vị trí code minh chứng:**

Câu #03: Tại sao giải pháp phân khung (Framing) bằng Length-Prefix 4 byte lại tối ưu hơn ký tự kết thúc dòng (Newline Framing)?

• **Trả lời cốt lõi:** Vì dữ liệu trao đổi là JSON (RFC 8259) có thể chứa ký tự xuống dòng `\n` bên trong chuỗi string đã escape; Length-Prefix cho biết chính xác số byte cần đọc trước, giúp cấp phát bộ đệm chính xác và tránh duyệt quét từng ký tự tốn CPU.

• **Vị trí code minh chứng:**

Câu #04: Quy ước thứ tự byte Big-Endian (Network Byte Order) là gì và tại sao bắt buộc phải dùng?

• **Trả lời cốt lõi:** Big-Endian đặt byte có trọng số lớn nhất ở địa chỉ bộ nhớ thấp nhất (đầu tiên trên đường truyền). Vì các kiến trúc CPU khác nhau (x86 dùng Little-Endian, ARM có thể dùng Big/Little) biểu diễn số nguyên khác nhau, bắt buộc phải dùng trật tự chuẩn mạng để giải mã độ dài nhất quán.

• **Vị trí code minh chứng:**

Câu #05: Phương thức `ReadExactlyBytesAsync` hoạt động như thế nào khi gặp ngắt kết nối giữa chừng (EOF)?

• **Trả lời cốt lõi:** Phương thức đọc lặp trong vòng lặp cho đến khi đủ số byte yêu cầu. Nếu socket bị đóng sớm (`ReadAsync` trả về 0 byte trước khi đủ độ dài), phương thức ném `EndOfStreamException` và hủy stream ngay lập tức, ngăn ngừa việc chuyển mẩu byte rác sang bộ giải mã JSON.

• **Vị trí code minh chứng:**

Câu #06: Tại sao lại giới hạn kích thước khung truyền tối đa `MaxFrameSize = 256 KiB`?

• **Trả lời cốt lõi:** Để phòng chống tấn công từ chối dịch vụ (DoS / OOM attack). Nếu kẻ xấu gửi 4 byte độ dài giả mạo là 2 GB, server không bị lừa cấp phát mảng byte khổng lồ làm sập bộ nhớ.

• **Vị trí code minh chứng:**

Câu #07: Tại sao cờ `Socket.Connected` không phản ánh chính xác trạng thái kết nối mạng thời gian thực?

• **Trả lời cốt lõi:** Cờ `Connected` chỉ phản ánh trạng thái sau lần thao tác I/O gần nhất. Nếu client bị mất điện đột ngột hoặc rút dây mạng mà server chưa gọi lệnh đọc/ghi, cờ này vẫn trả về `true` cho đến khi TCP Keep-Alive timeout hoặc I/O thất bại.

• **Vị trí code minh chứng:**

Câu #08: Tại sao hệ thống phải cài đặt cơ chế Heartbeat ở tầng ứng dụng?

• **Trả lời cốt lõi:** Vì TCP Keep-Alive mặc định của hệ điều hành có thể mất tới 2 giờ để phát hiện sự cố; Heartbeat tầng ứng dụng chu kỳ 5s giúp Central Server xác định chính xác liveness của máy trạm và kích hoạt cảnh báo trong vòng 20 giây.

• **Vị trí code minh chứng:**

Câu #09: Tại sao việc đo lường liveness phải dùng đồng hồ đơn điệu (Monotonic Clock) tại server?

• **Trả lời cốt lõi:** Dùng `Stopwatch.GetTimestamp()` tại server để đo khoảng cách thời gian trôi qua độc lập với đồng hồ hệ thống (Wall-clock), loại trừ hoàn toàn nguy cơ sai lệch hoặc nhảy lùi thời gian do đồng bộ NTP của client.

• **Vị trí code minh chứng:**

Câu #10: Single Writer Pattern trên socket được cài đặt như thế nào và tại sao cần thiết?

• **Trả lời cốt lõi:** Tài liệu chính thức từ Microsoft ghi rõ: lớp `NetworkStream` trong .NET chỉ hỗ trợ an toàn cho 1 thao tác đọc và 1 thao tác ghi diễn ra đồng thời. `NetworkStream` KHÔNG hỗ trợ nhiều tác vụ ghi (`WriteAsync`) đồng thời; nếu nhiều luồng cùng gọi `WriteAsync` mà không tự đồng bộ sẽ gây đan xen byte và làm hỏng frame trên wire. Do đó LabGuard sử dụng `Channel<byte[]>` (`System.Threading.Channels`) làm hàng đợi ghi đơn luồng (Single-Writer Queue), bảo đảm chỉ có đúng một Task nền duy nhất đọc từ Channel và ghi tuần tự các frame vào socket.

• **Vị trí code minh chứng:**

Câu #11: Sự khác biệt giữa `Channel<T>` có giới hạn (Bounded) và không giới hạn (Unbounded)?

• **Trả lời cốt lõi:** Bounded Channel giới hạn số phần tử trong hàng đợi (ví dụ: 100 mẩu tin); khi đầy, luồng ghi sẽ tạm hoãn bất đồng bộ (Backpressure), ngăn chặn việc ứng dụng tiêu tốn RAM vô hạn khi mạng bị nghẽn.

• **Vị trí code minh chứng:**

Câu #12: Cơ chế Reconnect Exponential Backoff kết hợp Jitter hoạt động ra sao?

• **Trả lời cốt lõi:** Khi mất kết nối, Agent chờ một khoảng thời gian tăng theo cấp số nhân ($1\text{s}, 2\text{s}, 4\text{s}, 8\text{s}... \max 30\text{s}$) cộng thêm một khoảng ngẫu nhiên (jitter), tránh hiện tượng hàng trăm máy trạm kết nối lại cùng một tích tắc gây bão kết nối (reconnect storm).

• **Vị trí code minh chứng:**

Câu #13: Tại sao cờ `isReplay` được đặt bên trong `payload` thay vì ở envelope root?

• **Trả lời cốt lõi:** Vì cờ này chỉ có ý nghĩa nghiệp vụ đối với dữ liệu đo lường (`METRIC_BATCH`) và trạng thái dịch vụ (`SERVICE_STATUS`) khi phát lại từ bộ nhớ đệm; các bản tin điều khiển hoặc đăng nhập không bao giờ có ngữ cảnh replay.

• **Vị trí code minh chứng:**

Câu #14: Khóa khử trùng lặp (Deduplication Composite Key) của telemetry gồm những trường nào?

• **Trả lời cốt lõi:** Bộ ba `DeviceId + BootId + BatchId`. `BootId` (UUID) sinh mới mỗi lần máy khởi động để tránh trùng số thứ tự `BatchId` khi bộ đếm bắt đầu lại từ 1.

• **Vị trí code minh chứng:**

Câu #15: Phương thức thu thập chỉ số CPU trên Windows bằng `Pdh` hoặc `PerformanceCounter` cần chú ý điều gì?

• **Trả lời cốt lõi:** Việc đọc lần đầu tiên cần khởi tạo phiên và mẫu thứ hai sau một khoảng thời gian mới tính được phần trăm CPU; tác vụ này cần chạy trên luồng nền riêng để không làm trễ chu kỳ gửi nhịp tim mạng.

• **Vị trí code minh chứng:**

Câu #16: Cơ chế thăm dò dịch vụ cục bộ (Local TCP Service Probe) có thể gây blocking không?

• **Trả lời cốt lõi:** Có thể gây block nếu dùng phương thức đồng bộ hoặc timeout mặc định của hệ điều hành (20s). Do đó bắt buộc dùng `ConnectAsync` với `CancellationTokenSource` hủy sau đúng 2 giây.

• **Vị trí code minh chứng:**

Câu #17: Sự khác biệt cốt lõi giữa Agent thật và Agent Simulator là gì?

• **Trả lời cốt lõi:** Simulator thật 100% về socket TCP, mã hóa TLS, phân khung Length-Prefix và định dạng bản tin gửi đi; điểm giả lập duy nhất là chỉ số CPU/RAM được sinh ngẫu nhiên theo kịch bản thay vì đọc từ phần cứng thật.

• **Vị trí code minh chứng:**

Câu #18: Làm thế nào để một tiến trình Simulator tạo được 50 kết nối socket đồng thời mà không nghẽn tài nguyên?

• **Trả lời cốt lõi:** Sử dụng các phương thức bất đồng bộ `async/await` trên ThreadPool của .NET, các socket dùng chung luồng I/O thay vì tạo 50 Thread riêng của hệ điều hành.

• **Vị trí code minh chứng:**

Câu #19: Bộ cân bằng tải Gateway L4 khác gì so với Gateway L7 (Reverse Proxy)?

• **Trả lời cốt lõi:** Gateway L4 chỉ chuyển tiếp luồng byte nhị phân thô (TCP byte relay) dựa trên địa chỉ IP và port mà không giải mã TLS hay đọc nội dung JSON bên trong; Gateway L7 phải giải mã giao thức ứng dụng, tốn nhiều CPU hơn.

• **Vị trí code minh chứng:**

Câu #20: Thuật toán Least-Connections trong Gateway L4 được cài đặt như thế nào?

• **Trả lời cốt lõi:** Gateway duy trì một biến đếm số kết nối TCP đang mở (Active Connections) tới từng Collector; khi có client mới kết nối, Gateway chọn Collector có biến đếm nhỏ nhất và đang có trạng thái Health Check là `Up`.

• **Vị trí code minh chứng:**

Câu #21: Tại sao hệ thống không yêu cầu di chuyển socket đang mở (Socket Migration) khi Collector gặp sự cố?

• **Trả lời cốt lõi:** Việc di chuyển socket TCP đang mở giữa các server yêu cầu hạ tầng mạng chuyên dụng rất phức tạp và vượt quá phạm vi đồ án NT106; đồ án áp dụng mô hình chuẩn công nghiệp: cho phép Agent ngắt kết nối và tự động kết nối lại sang node khác.

• **Vị trí code minh chứng:**

Câu #22: Mục tiêu thiết kế thời gian failover p95 ≤ 45 giây được kiểm chứng như thế nào?

• **Trả lời cốt lõi:** Đo khoảng thời gian từ lúc tắt tiến trình Collector 01 cho đến khi Agent nhận ra lỗi kết nối, áp dụng backoff, kết nối lại thành công qua Gateway tới Collector 02 và gửi mẻ tin telemetry tiếp theo.

• **Vị trí code minh chứng:**

Câu #23: Tại sao log hệ thống phải được cài đặt cơ chế xoay vòng (Log Rotation)?

• **Trả lời cốt lõi:** Tránh việc file log phình to vô hạn làm cạn kiệt dung lượng ổ đĩa của máy trạm; hệ thống quy định MaxFileSize = 10 MB, MaxFiles = 5, tổng dung lượng log <= 50 MB [CP2 TO VERIFY].

• **Vị trí code minh chứng:**

Câu #24: Khi kết nối qua mạng Internet bằng Port Forwarding hoặc Tunnel, Agent cần cấu hình gì?

• **Trả lời cốt lõi:** Cần cấu hình tên miền hoặc địa chỉ IP công khai kèm cổng NAT; đồng thời thiết lập tên máy chủ (Server Name) trong `SslStream.AuthenticateAsClient` khớp với Common Name (CN) hoặc SAN của chứng thư số TLS.

• **Vị trí code minh chứng:**

Câu #25: Làm thế nào để phân biệt giữa lỗi mạng tạm thời (Transient Network Error) và máy trạm bị tắt nguồn thật sự?

• **Trả lời cốt lõi:** Lỗi mạng tạm thời sẽ khiến heartbeat bị trễ nhưng khi có mạng lại Agent sẽ gửi kèm mẻ tin replay; nếu quá 20s không có bất kỳ tín hiệu nào và probe TCP thất bại, Central mới xác định máy trạm Offline.

• **Vị trí code minh chứng:**

Câu #26: Vai trò của việc tạo mã băm SHA-256 cho gói bàn giao (Release Package) là gì?

• **Trả lời cốt lõi:** Bảo đảm tính toàn vẹn của mã nguồn và tệp thực thi, chứng minh tệp giao nộp cho giảng viên không bị chỉnh sửa, thiếu sót hoặc nhiễm mã độc sau khi hoàn tất kiểm thử.

• **Vị trí code minh chứng:**

Câu #27: Tại sao không nên sử dụng giao thức UDP cho việc truyền dữ liệu giám sát phòng máy?

• **Trả lời cốt lõi:** UDP không có cơ chế bảo đảm thứ tự và không tin cậy; dữ liệu sự cố và lệnh quản trị cần tính chính xác tuyệt đối, việc tự cài đặt cơ chế ACK và retransmit trên UDP sẽ làm phức tạp hóa đồ án không cần thiết.

• **Vị trí code minh chứng:**

Câu #28: Kỹ thuật Bounded Channel giải quyết vấn đề gì trong Writer Loop của Agent?

• **Trả lời cốt lõi:** Khi mạng bị chập chờn, các mẻ tin thu thập được đẩy vào Channel; nếu Channel đạt giới hạn 100 mẻ tin, tác vụ đẩy tin sẽ tạm dừng (Backpressure) để kích hoạt cơ chế gộp mẻ hoặc lưu bộ nhớ đệm phụ thay vì làm tràn RAM.

• **Vị trí code minh chứng:**

Câu #29: Nếu một máy trạm Agent bị đổi địa chỉ IP động qua DHCP, hệ thống nhận diện máy trạm ra sao?

• **Trả lời cốt lõi:** Hệ thống định danh thiết bị duy nhất dựa trên trường `deviceId` trong Envelope và Session Token sau khi xác thực, hoàn toàn độc lập với địa chỉ IP tầng mạng.

• **Vị trí code minh chứng:**

Câu #30: Khi demo trước hội đồng, nếu mạng Internet tại trường bị chập chờn, phương án dự phòng là gì?

• **Trả lời cốt lõi:** Đồ án chuẩn bị sẵn mạng LAN nội bộ phòng máy (hoặc phát qua Router Wi-Fi riêng / Switch cơ học) kết nối 3 laptop; kịch bản demo trên mạng LAN độc lập 100% với đường truyền Internet bên ngoài.

• **Vị trí code minh chứng:**

30 câu hỏi trọng tâm — SINH VIÊN B (SERVER & CSDL)

Câu #01: Kiến trúc luồng (Threading Model) của Central Server được thiết kế như thế nào?

• **Trả lời cốt lõi:** Central Server sử dụng mô hình Async I/O dựa trên ThreadPool của .NET (`SocketAsyncEventArgs` / `ReadAsync`), không tạo 1 Thread riêng của hệ điều hành cho mỗi kết nối socket. Khi socket nhàn rỗi, luồng được hoàn trả cho ThreadPool để phục vụ tác vụ khác.

• **Vị trí code minh chứng:**

Câu #02: Tại sao không nên tạo một Thread riêng (`new Thread()`) cho mỗi client kết nối?

• **Trả lời cốt lõi:** Mỗi OS Thread trong Windows tiêu tốn mặc định khoảng 1 MB bộ nhớ stack và gây áp lực lớn cho CPU khi chuyển ngữ cảnh luồng (Context Switching). Với hàng trăm client, mô hình 1-thread-per-client sẽ làm cạn kiệt tài nguyên hệ thống.

• **Vị trí code minh chứng:**

Câu #03: Kỹ thuật Ingest Ring Buffer sử dụng `Channel<T>` có giới hạn nhằm giải quyết vấn đề gì?

• **Trả lời cốt lõi:** Đóng vai trò bộ đệm điều tiết tốc độ (Backpressure Buffer) giữa luồng nhận mạng cực nhanh và luồng ghi CSDL có độ trễ I/O đĩa. Khi bộ đệm đầy (ví dụ 500 mẻ tin), luồng đọc socket tạm hoãn để bảo vệ server không bị cạn kiệt RAM.

• **Vị trí code minh chứng:**

Câu #04: Tại sao Ingest Ring Buffer không được coi là nơi chống trùng lặp bền vững?

• **Trả lời cốt lõi:** Vì Ring Buffer nằm trên bộ nhớ RAM; nếu Central Server bị mất điện hoặc khởi động lại đột ngột, toàn bộ dữ liệu trong RAM sẽ bị mất. Nơi chống trùng bền vững bắt buộc phải nằm trên đĩa CSDL (bảng `ProcessedBatches`).

• **Vị trí code minh chứng:**

Câu #05: Cấu trúc bảng `ProcessedBatches` và chính sách dọn dẹp (TTL Policy) là gì?

• **Trả lời cốt lõi:** Bảng có khóa chính `(DeviceId, BootId, BatchId)` và cột `ReceivedUtc`. Dữ liệu được lưu vết trong 24 giờ để lọc trùng lặp khi Agent gửi lại; một job nền định kỳ xóa các bản ghi cũ hơn 24 giờ.

• **Vị trí code minh chứng:**

Câu #06: Khi phát hiện một mẻ tin đã tồn tại trong `ProcessedBatches`, server xử lý như thế nào?

• **Trả lời cốt lõi:** Server bỏ qua việc phân tích và ghi telemetry mới vào CSDL, nhưng **bắt buộc phải gửi lại `BATCH_ACK`** về cho Agent để Agent biết mẻ tin đã tới đích an toàn và xóa khỏi hàng đợi replay.

• **Vị trí code minh chứng:**

Câu #07: Cấu trúc bảng `ProcessedCommands` bảo đảm tính lũy thừa (Idempotency) của lệnh Admin ra sao?

• **Trả lời cốt lõi:** Bảng lưu khóa `(UserId, CommandId)` kèm mã băm `RequestDigest` (SHA-256) và kết quả đã trả `ResponsePayload`. Nếu Admin bấm lặp lại lệnh cùng `CommandId`, server trả ngay kết quả cũ đã commit mà không thực thi lại mutation.

• **Vị trí code minh chứng:**

Câu #08: Tại sao không dùng bảng `AuditLogs` hoặc `IncidentHistory` làm bảng chống trùng lệnh?

• **Trả lời cốt lõi:** `AuditLogs` và `IncidentHistory` phục vụ nhật ký lịch sử cho người quản trị theo dõi (mỗi lần bấm là một dòng log mới); bảng chống trùng lệnh (`ProcessedCommands`) có cấu trúc khóa duy nhất trên `CommandId` và TTL 7 ngày dành riêng cho logic máy.

• **Vị trí code minh chứng:**

Câu #09: Mã băm `RequestDigest` trong bảng `ProcessedCommands` ngăn chặn rủi ro gì?

• **Trả lời cốt lõi:** Ngăn chặn rủi ro tấn công hoặc nhầm lẫn tham số: nếu client gửi cùng một `CommandId` nhưng thay đổi nội dung lệnh (ví dụ đổi action từ ACK sang RESOLVE), server phát hiện mã băm khác nhau và ném lỗi `ERR_INVALID_COMMAND_DIGEST`.

• **Vị trí code minh chứng:**

Câu #10: Liveness Checker của Central Server phát hiện máy trạm Offline dựa trên nguyên lý nào?

• **Trả lời cốt lõi:** Duy trì bảng đăng ký nhịp tim trong bộ nhớ kèm mốc thời gian nhận gần nhất đo bằng `Stopwatch.GetTimestamp()`. Định kỳ mỗi giây, một tác vụ nền quét kiểm tra: nếu thời gian trôi qua vượt quá 20 giây, server tự động kích hoạt sự cố `DEVICE_OFFLINE`.

• **Vị trí code minh chứng:**

Câu #11: Quy tắc phát hiện lỗi CPU (`HIGH_CPU`) với Hysteresis và Streak được cài đặt như thế nào?

• **Trả lời cốt lõi:** Yêu cầu 3 mẫu liên tiếp có CPU $\ge 90\%$ (ở chu kỳ 5s, khoảng thời gian giữa mẫu 1 và 3 là đúng 10 giây). Khi đã mở sự cố, chỉ đóng sự cố khi nhận được 3 mẫu liên tiếp có CPU $\le 75\%$. Dải 15% Deadband ngăn hiện tượng flapping.

• **Vị trí code minh chứng:**

Câu #12: Quy tắc phát hiện dịch vụ chết (`SERVICE_DOWN`) quy định số lần thất bại và phục hồi ra sao?

• **Trả lời cốt lõi:** Cảnh báo kích hoạt khi probe TCP thất bại 2 lần liên tiếp (ở chu kỳ 5s); sự cố tự động chuyển trạng thái phục hồi kỹ thuật khi probe TCP thành công trở lại 2 lần liên tiếp.

• **Vị trí code minh chứng:**

Câu #13: Tại sao kiến trúc hệ thống phải phân tách thành `ConditionState` và `WorkflowState`?

• **Trả lời cốt lõi:** `ConditionState` (`ACTIVE`, `RECOVERED`, `UNKNOWN`) phản ánh hiện trạng kỹ thuật khách quan do Rule Engine đo lường từ Agent; `WorkflowState` (`OPEN`, `ACKNOWLEDGED`, `IN_PROGRESS`, `RECOVERED`, `RESOLVED`) quản lý tiến trình xử lý nghiệp vụ của con người.

• **Vị trí code minh chứng:**

Câu #14: Tình huống ``Tái phát sự cố'' (Relapse) xảy ra khi nào và state machine chuyển đổi ra sao?

• **Trả lời cốt lõi:** Khi sự cố đang ở `WorkflowState = RECOVERED` (lỗi kỹ thuật đã hết nhưng Admin chưa kịp đóng phiếu `RESOLVED`), nếu Agent lại báo lỗi trở lại (`ConditionState = ACTIVE`), sự cố sẽ tự động chuyển ngược về `WorkflowState = IN_PROGRESS` để tiếp tục theo dõi.

• **Vị trí code minh chứng:**

Câu #15: Ràng buộc duy nhất một Incident hoạt động (Unique Active Incident Constraint) được đảm bảo ra sao?

• **Trả lời cốt lõi:** Bằng Unique Filtered Index trên SQL Server: `CREATE UNIQUE INDEX UX_ActiveIncident ON Incidents(DeviceId, RuleType) WHERE WorkflowState <> 'RESOLVED'`, bảo đảm một máy không bao giờ bị mở nhiều phiếu cho cùng một lỗi.

• **Vị trí code minh chứng:**

Câu #16: Cột `rowversion` trong SQL Server là gì và hoạt động ra sao?

• **Trả lời cốt lõi:** `rowversion` là datatype riêng của SQL Server. Giá trị lưu trữ có kích thước 8 byte và do SQL Server tự sinh khi rowversion row được insert/update. Kiểu `binary(8)` thông thường KHÔNG có semantics tự cập nhật này. `rowversion` KHÔNG phải date/time. Nó đóng vai trò như một opaque version token phục vụ kiểm tra Optimistic Concurrency Control (OCC).

• **Vị trí code minh chứng:**

Câu #17: Optimistic Concurrency Control (OCC) giải quyết bài toán hai Admin cùng bấm ACK một sự cố như thế nào?

• **Trả lời cốt lõi:** Admin gửi lệnh kèm `expectedRowVersion` (coi như một opaque 8-byte token đặc thù của SQL Server). Lệnh cập nhật SQL kiểm tra: `UPDATE Incidents SET WorkflowState='ACKNOWLEDGED' WHERE IncidentId=@Id AND RowVersion=@expected`. Người đầu tiên cập nhật thành công và đổi `rowversion`; người thứ hai có điều kiện WHERE không khớp, số dòng bị ảnh hưởng là 0 $\rightarrow$ Server ném lỗi `409 Conflict`.

• **Vị trí code minh chứng:**

Câu #18: Tại sao nên dùng Optimistic Concurrency thay vì Pessimistic Locking (`SELECT FOR UPDATE`)?

• **Trả lời cốt lõi:** Pessimistic Locking giữ khóa bảng hoặc khóa dòng CSDL trong suốt thời gian người dùng mở form, làm nghẽn toàn bộ các kết nối khác. OCC không giữ khóa, cho phép đọc tự do và chỉ kiểm tra xung đột tại đúng thời điểm commit.

• **Vị trí code minh chứng:**

Câu #19: Giao dịch Incident được bao bọc trong Transaction Boundary như thế nào?

• **Trả lời cốt lõi:** Khi chuyển trạng thái sự cố, server thực thi đồng thời trong một giao dịch ACID: Cập nhật dòng bảng `Incidents`, thêm một dòng ghi nhật ký vào `IncidentHistory`, và cập nhật bảng chống trùng `ProcessedCommands`. Nếu có bất kỳ lỗi nào, toàn bộ giao dịch bị rollback.

• **Vị trí code minh chứng:**

Câu #20: Tại sao mật khẩu người dùng bắt buộc phải được băm kèm Salt bằng thuật toán PBKDF2?

• **Trả lời cốt lõi:** Salt (chuỗi ngẫu nhiên 16 byte / 128-bit) bảo đảm hai người dùng có cùng mật khẩu sẽ sinh ra hai chuỗi băm hoàn toàn khác nhau, vô hiệu hóa bảng tra cứu tính sẵn (Rainbow Table); kết hợp so sánh bằng `CryptographicOperations.FixedTimeEquals` để chống timing attack. Về mặt chuẩn mực: OWASP recommends Argon2id for new systems generally. If FIPS-140 compliance is required, OWASP recommends PBKDF2-HMAC-SHA256 with a work factor of 600,000 or more. LabGuard deliberately chooses PBKDF2 for BCL availability and course-scope simplicity; 600,000 is therefore the project baseline referenced from OWASP and must still be measured on the actual lab hardware.

• **Vị trí code minh chứng:**

Câu #21: Tại sao việc kiểm tra quyền hạn (Authorization / RBAC) bắt buộc phải thực hiện ở phía server?

• **Trả lời cốt lõi:** Ẩn nút trên giao diện WinForms chỉ là hỗ trợ trải nghiệm người dùng; kẻ xấu có thể can thiệp socket hoặc dùng tool gửi bản tin wire trực tiếp lên server. Server bắt buộc phải kiểm tra role trong Session Token trước khi thực thi lệnh.

• **Vị trí code minh chứng:**

Câu #22: Tại sao bản tin Snapshot gửi cho Admin phải được phân trang (`SNAPSHOT_PAGE`)?

• **Trả lời cốt lõi:** Nếu phòng máy có hàng trăm thiết bị và hàng nghìn lịch sử sự cố, gửi toàn bộ trong một bản tin JSON khổng lồ sẽ vượt trần `MaxFrameSize = 256 KiB` và làm nghẽn bộ đệm socket; phân trang giúp chia nhỏ dữ liệu và giải phóng luồng I/O.

• **Vị trí code minh chứng:**

Câu #23: Cơ chế Realtime Event Fanout được cài đặt như thế nào trong Central Server?

• **Trả lời cốt lõi:** Central Server duy trì danh sách các kết nối Admin đang mở (Subscriber Registry); khi có biến động từ Rule Engine, máy chủ đóng gói bản tin delta event và gửi tuần tự qua Channel Single Writer của từng Admin connection.

• **Vị trí code minh chứng:**

Câu #24: Kỹ thuật Slow-Client Isolation bảo vệ hệ thống như thế nào?

• **Trả lời cốt lõi:** Nếu một Admin Client có kết nối mạng quá chậm hoặc bị treo máy, hàng đợi gửi của Admin đó sẽ bị đầy; hệ thống kích hoạt chính sách drop event hoặc ngắt kết nối client đó sau một khoảng timeout, tuyệt đối không làm block các Admin Client khác.

• **Vị trí code minh chứng:**

Câu #25: Chính sách lưu trữ dữ liệu (Data Retention) được cài đặt và vận hành ra sao?

• **Trả lời cốt lõi:** Telemetry thô (`Metrics`) được lưu trong 48 giờ; một Background Worker định kỳ chạy vào ban đêm thực hiện câu lệnh xóa theo mẻ (Batch Delete) dựa trên cột `ReceivedUtc` để không làm khóa bảng lâu dài.

• **Vị trí code minh chứng:**

Câu #26: Kịch bản phục hồi thảm họa CSDL (Crash Recovery Proof) được kiểm chứng thế nào?

• **Trả lời cốt lõi:** Tiến hành sao lưu CSDL bằng lệnh `BACKUP DATABASE`, cố tình tắt đột ngột tiến trình SQL Server khi đang có tải ghi dồn dập, sau đó khởi động lại và chạy lệnh `RESTORE VERIFYONLY` để chứng minh CSDL tự phục hồi và dữ liệu không bị hỏng (corrupted).

• **Vị trí code minh chứng:**

Câu #27: Trong kiến trúc P1, cơ chế Fencing và Session Generation giải quyết vấn đề gì?

• **Trả lời cốt lõi:** Khi Agent chuyển từ Collector 01 sang Collector 02, Central Server tăng chỉ số `SessionGeneration`; nếu các gói tin chậm trễ từ Collector 01 cũ vẫn còn bay trên mạng tới sau, Central sẽ loại bỏ các gói tin này vì có chỉ số generation cũ hơn, ngăn chặn ghi đè dữ liệu cũ (split-brain).

• **Vị trí code minh chứng:**

Câu #28: Tại sao Central Server không lưu trạng thái phiên làm việc (Session) trong CSDL mà lưu trong RAM Registry?

• **Trả lời cốt lõi:** Kiểm tra token xác thực diễn ra ở mọi request với tần suất cực lớn; lưu Session Registry trong RAM (sử dụng `ConcurrentDictionary`) giúp truy xuất với độ trễ nano-giây, giảm tải triệt để cho CSDL SQL Server.

• **Vị trí code minh chứng:**

Câu #29: Tại sao kiểu dữ liệu `rowversion` không thể chuyển đổi tương đương sang SQLite?

• **Trả lời cốt lõi:** SQLite không có kiểu dữ liệu nhị phân 8 byte tự tăng nguyên bản ở mức engine như SQL Server; việc giả lập bằng trigger hoặc timestamp trong SQLite dễ bị sai lệch concurrency, do đó việc khóa chặt SQL Server 2022 Express là quyết định kiến trúc đúng đắn.

• **Vị trí code minh chứng:**

Câu #30: Khi Central Server bị khởi động lại, các Agent kết nối lại có làm sập server không?

• **Trả lời cốt lõi:** Nhờ thuật toán Exponential Backoff và Jitter phía Agent, các kết nối không ùa vào cùng một lúc; tại Central Server, hàng đợi Ingress Bounded Channel chỉ nhận số lượng kết nối tối đa cho phép, loại bỏ rủi ro nghẽn tài nguyên.

• **Vị trí code minh chứng:**

30 câu hỏi trọng tâm — SINH VIÊN C (DASHBOARD & UI)

Câu #01: Kiến trúc luồng của Windows Forms được thiết kế theo nguyên lý nào?

• **Trả lời cốt lõi:** Windows Forms hoạt động theo mô hình Single-Threaded Apartment (STA), trong đó chỉ có đúng một luồng duy nhất (UI Thread) được sở hữu vòng lặp thông điệp Windows (Message Loop) và được phép truy cập, sửa đổi các thành phần giao diện Win32.

• **Vị trí code minh chứng:**

Câu #02: Tại sao việc thao tác control WinForms trực tiếp từ luồng mạng (Background Thread) lại gây lỗi?

• **Trả lời cốt lõi:** Vì vi phạm tính an toàn đa luồng của hệ thống cửa sổ Windows; truy cập đồng thời từ luồng khác sẽ gây xung đột bộ nhớ đồ họa GDI, làm sai lệch trạng thái hiển thị và ném ngoại lệ `InvalidOperationException: Cross-thread operation not valid`.

• **Vị trí code minh chứng:**

Câu #03: Sự khác biệt kỹ thuật giữa `Control.Invoke` và `Control.BeginInvoke` là gì?

• **Trả lời cốt lõi:** `Control.Invoke` là cuộc gọi đồng bộ, luồng mạng sẽ bị block chờ cho đến khi UI Thread thực thi xong delegate; `Control.BeginInvoke` là cuộc gọi bất đồng bộ, luồng mạng đẩy delegate vào Message Queue của UI rồi tiếp tục chạy ngay mà không chờ đợi.

• **Vị trí code minh chứng:**

Câu #04: Khi nào nên dùng `Control.BeginInvoke` thay vì `Control.Invoke` trong ứng dụng realtime?

• **Trả lời cốt lõi:** Khi nhận các sự kiện delta thời gian thực liên tục từ mạng (`DEVICE_UPDATE`, `INCIDENT_OPEN`), nên dùng `BeginInvoke` để luồng mạng không bị chặn nếu UI Thread đang bận vẽ đồ thị, tránh làm nghẽn bộ đệm socket.

• **Vị trí code minh chứng:**

Câu #05: Tại sao thuộc tính `CheckForIllegalCrossThreadCalls = false` bị nghiêm cấm trong sản phẩm chuẩn?

• **Trả lời cốt lõi:** Vì câu lệnh này chỉ tắt cờ kiểm tra cảnh báo của .NET Framework chứ hoàn toàn không làm cho mã nguồn an toàn đa luồng; ứng dụng vẫn tiềm ẩn nguy cơ race condition nghiêm trọng, vẽ lỗi form hoặc tự sập ngẫu nhiên.

• **Vị trí code minh chứng:**

Câu #06: Phương thức `InvokeAsync` trong .NET 9+ WinForms mang lại lợi ích gì?

• **Trả lời cốt lõi:** Cho phép await trực tiếp một thao tác điều phối giao diện từ bất kỳ Task nền nào mà không cần tạo delegate thủ công, giúp mã nguồn bất đồng bộ sạch sẽ và tận dụng triệt để `IProgress<T>` hoặc `SynchronizationContext`.

• **Vị trí code minh chứng:**

Câu #07: Cơ chế RPC bất đồng bộ trên một kết nối TCP duy nhất được cài đặt như thế nào?

• **Trả lời cốt lõi:** Khi gửi một request, client tạo một `TaskCompletionSource<TResponse>`, lưu vào `ConcurrentDictionary` với key là `requestId` (UUID v4) rồi gửi frame. Luồng đọc socket khi nhận được response sẽ bóc tách `requestId`, tìm `TaskCompletionSource` tương ứng và gọi `SetResult()` để hoàn tất Task.

• **Vị trí code minh chứng:**

Câu #08: Làm thế nào để xử lý Timeout cho các yêu cầu RPC khi mạng bị ngắt hoặc server không phản hồi?

• **Trả lời cốt lõi:** Sử dụng `CancellationTokenSource` với thời gian chờ (ví dụ 5 giây): `cts.CancelAfter(5000)`. Đăng ký callback khi token bị hủy sẽ gọi `TrySetCanceled()` trên `TaskCompletionSource` và gỡ bản ghi khỏi Dictionary.

• **Vị trí code minh chứng:**

Câu #09: Tại sao ứng dụng Admin không gửi truy vấn định kỳ (Polling) mà sử dụng Snapshot + Realtime Delta?

• **Trả lời cốt lõi:** Polling liên tục tạo tải CPU và băng thông mạng rất lớn cho Central Server, đồng thời gây trễ thông tin. Snapshot + Delta chỉ nạp dữ liệu đầy đủ 1 lần lúc mở máy, sau đó server chủ động đẩy đúng các thay đổi nhỏ, giảm 90% lưu lượng mạng.

• **Vị trí code minh chứng:**

Câu #10: Quá trình nạp dữ liệu ban đầu bằng Snapshot phân trang (`SNAPSHOT_BEGIN/PAGE/END`) diễn ra ra sao?

• **Trả lời cốt lõi:** Client nhận `SNAPSHOT_BEGIN` để biết tổng số thiết bị và khởi tạo bộ nhớ tạm; nhận tuần tự các trang `SNAPSHOT_PAGE` để nạp danh sách thiết bị; khi nhận `SNAPSHOT_END`, client khóa bộ nhớ đệm `LocalCache`, cập nhật một lần lên UI và bắt đầu tiêu thụ các delta event.

• **Vị trí code minh chứng:**

Câu #11: Nếu một delta event đến trong khi ứng dụng đang nạp Snapshot phân trang, client xử lý thế nào?

• **Trả lời cốt lõi:** Client đưa delta event vào một hàng đợi chờ (Pending Event Queue); sau khi xử lý xong `SNAPSHOT_END` và xác định chỉ số `initialSequence`, client mới tuần tự áp dụng các delta event có sequence lớn hơn để không làm sai lệch trạng thái.

• **Vị trí code minh chứng:**

Câu #12: Màn hình sơ đồ phòng máy Lab View được thiết kế trực quan như thế nào?

• **Trả lời cốt lõi:** Thiết kế dạng lưới các ô máy tính (Grid Layout). Mỗi ô hiển thị tên máy (ví dụ `PC-07`), địa chỉ IP, trạng thái dạng chữ (`ONLINE`, `OFFLINE`, `WARNING`), kèm icon sinh động và viền màu tương ứng; không bao giờ chỉ dựa vào một màu nền đơn điệu.

• **Vị trí code minh chứng:**

Câu #13: Tại sao trạng thái trên Lab View phải xuất phát từ sự kiện mạng thật thay vì timer giả lập?

• **Trả lời cốt lõi:** Để bảo đảm tính trung thực của một đồ án mạng máy tính. Khi máy tính phòng máy bị rút dây mạng, ô hiển thị chuyển sang `OFFLINE` là do Central phát sự kiện `DEVICE_UPDATE` sau khi hết deadline heartbeat 20s, không phải do hàm timer trên WinForms tự đếm giờ.

• **Vị trí code minh chứng:**

Câu #14: Optimistic Concurrency Control (OCC) được thể hiện trên giao diện Admin như thế nào khi gặp lỗi 409 Conflict?

• **Trả lời cốt lõi:** Khi Admin bấm nút Resolve nhưng sự cố đã bị người khác thao tác trước đó, server trả về `status = "CONFLICT"`. UI bắt mã lỗi này, hiển thị thông báo: "Sự cố đã được Admin khác cập nhật", đồng thời tự động cập nhật lại thông tin mới nhất trên form mà không làm mất dữ liệu của người dùng.

• **Vị trí code minh chứng:**

Câu #15: Tại sao client phải gửi kèm `expectedRowVersion` trong các lệnh thao tác sự cố (`INCIDENT_ACK`, `INCIDENT_RESOLVE`)?

• **Trả lời cốt lõi:** Để Central Server so sánh với giá trị `rowversion` hiện tại trong CSDL. Nếu hai giá trị khớp nhau, chứng tỏ dữ liệu Admin đang nhìn thấy là mới nhất; nếu khác nhau, chứng tỏ đã có người khác sửa đổi trước đó.

• **Vị trí code minh chứng:**

Câu #16: Phân quyền người dùng (RBAC) được thể hiện ở giao diện WinForms ra sao?

• **Trả lời cốt lõi:** Sau khi đăng nhập, căn cứ vào role trong `SessionToken`: Nếu là `Viewer` (chế độ chỉ xem), toàn bộ các nút mutation (ACK, Start, Resolve, Assign, Invite) bị ẩn hoặc disable; nếu là `Operator`, được hiển thị và thao tác các nút xử lý sự cố trực ban (ACK, Start, Resolve); nếu là `Admin`, có toàn quyền Operator kèm theo nút phân công (Assign) và quản trị tài khoản / lời mời (Invite / User Management). Mọi thao tác đều được Central Server kiểm tra quyền độc lập.

• **Vị trí code minh chứng:**

Câu #17: Tại sao việc ẩn nút trên WinForms không thay thế được việc kiểm tra bảo mật phía server?

• **Trả lời cốt lõi:** Vì kẻ xấu có thể can thiệp bộ nhớ ứng dụng hoặc viết một script TCP độc lập để gửi lệnh trực tiếp vào socket. Giao diện chỉ mang tính chất hỗ trợ trải nghiệm người dùng (UX), quyền lực bảo mật tối thượng bắt buộc thuộc về server.

• **Vị trí code minh chứng:**

Câu #18: Cơ chế Reconnect Backoff phía Admin Client hoạt động như thế nào khi mất kết nối tới Central?

• **Trả lời cốt lõi:** Client chuyển giao diện sang trạng thái vô hiệu hóa các nút bấm, hiện thanh thông báo màu vàng "Mất kết nối, đang thử lại sau Xs...". Thời gian thử lại tăng dần (1s, 2s, 4s... max 30s) và tự động yêu cầu Snapshot mới khi kết nối lại thành công.

• **Vị trí code minh chứng:**

Câu #19: Tại sao sau khi Reconnect thành công, Admin Client bắt buộc phải yêu cầu Snapshot mới thay vì dùng tiếp cache cũ?

• **Trả lời cốt lõi:** Trong khoảng thời gian mất mạng, client đã bỏ lỡ nhiều delta events; việc tiếp tục dùng cache cũ sẽ dẫn đến hiển thị sai lệch hoặc gửi sai `expectedRowVersion` gây lỗi xung đột liên tục.

• **Vị trí code minh chứng:**

Câu #20: Bộ nhớ đệm cục bộ `LocalCache` trong WinForms được quản lý bằng cấu trúc dữ liệu nào?

• **Trả lời cốt lõi:** Sử dụng `ConcurrentDictionary<string, DeviceViewModel>` và `ConcurrentDictionary<string, IncidentViewModel>`, bảo đảm các thao tác đọc từ UI Thread và ghi từ luồng mạng diễn ra an toàn mà không gây deadlock.

• **Vị trí code minh chứng:**

Câu #21: Kỹ thuật Virtual Mode trên `DataGridView` hoặc `ListView` giải quyết vấn đề gì khi hiển thị hàng nghìn bản ghi?

• **Trả lời cốt lõi:** Không nạp sẵn hàng nghìn object đồ họa vào bộ nhớ; điều khiển chỉ yêu cầu dữ liệu cho đúng số dòng đang hiển thị trên màn hình qua sự kiện `CellValueNeeded`, giúp ứng dụng mở tức thì và tiêu tốn cực ít RAM.

• **Vị trí code minh chứng:**

Câu #22: Làm thế nào để xuất dữ liệu báo cáo lịch sử sự cố ra file CSV chuẩn UTF-8?

• **Trả lời cốt lõi:** Sử dụng `StreamWriter` với encoding `new UTF8Encoding(true)` (có ký tự BOM - Byte Order Mark) để Microsoft Excel trên Windows có thể mở file CSV và hiển thị Tiếng Việt có dấu chính xác mà không bị lỗi font.

• **Vị trí code minh chứng:**

Câu #23: Tại sao Sinh viên C chỉ phát triển một ứng dụng Collector dùng chung cho cả 2 instance ở P1?

• **Trả lời cốt lõi:** Để bảo đảm tính nhất quán của giao thức và mã nguồn. Nếu hai thành viên viết hai bản Collector riêng sẽ dễ dẫn đến lệch pha xử lý khung truyền; việc viết một bản chuẩn và chạy thành 2 instance là mô hình chuẩn mực công nghiệp.

• **Vị trí code minh chứng:**

Câu #24: Ứng dụng Collector trung gian (`LabGuard.Collector.exe`) thực hiện nhiệm vụ gì?

• **Trả lời cốt lõi:** Đóng vai trò proxy gom mẻ: Lắng nghe kết nối TCP từ Agent, phân khung Length-Prefix, xác thực token ban đầu, sau đó ghép kênh (Multiplexing) chuyển tiếp dữ liệu lên Central Server qua một kết nối socket duy nhất.

• **Vị trí code minh chứng:**

Câu #25: Cơ chế Upstream Multiplexer trong Collector hoạt động ra sao?

• **Trả lời cốt lõi:** Hàng chục Agent gửi dữ liệu vào Collector; Collector gom các mẻ tin này vào một hàng đợi Channel duy nhất và gửi tuần tự lên Central Server, kèm theo cơ chế định tuyến (Reply Router) để trả đúng ACK về cho từng Agent tương ứng.

• **Vị trí code minh chứng:**

Câu #26: Collector báo cáo trạng thái sẵn sàng (Readiness / Health Check) tới Gateway L4 như thế nào?

• **Trả lời cốt lõi:** Collector mở một cổng riêng hoặc lắng nghe tín hiệu thăm dò từ Gateway; nếu bộ đệm Ingress hoặc kết nối Upstream tới Central gặp sự cố, Collector trả về trạng thái không sẵn sàng để Gateway không điều phối thêm kết nối mới.

• **Vị trí code minh chứng:**

Câu #27: Làm thế nào để kiểm thử kịch bản hai Admin cùng xử lý một sự cố (Multi-Admin Concurrency)?

• **Trả lời cốt lõi:** Khởi chạy đồng thời 2 tiến trình `LabGuard.Admin.exe` trên 2 màn hình, cùng mở một phiếu sự cố `INC-001`. Cả 2 người cùng bấm nút "Acknowledge" trong cùng 1 giây. Kết quả: 1 bên thành công, 1 bên hiện hộp thoại Conflict và tự làm mới dữ liệu.

• **Vị trí code minh chứng:**

Câu #28: Các biện pháp chống tràn giao diện (UI Responsiveness) khi có bão sự kiện gửi về là gì?

• **Trả lời cốt lõi:** Sử dụng kỹ thuật gom mẻ cập nhật (Batch UI Updates) hoặc Debounce/Throttle: Thay vì mỗi event gọi một lần `Invoke`, client gom các event trong khoảng 100ms và gọi một lần `BeginInvoke` để cập nhật toàn bộ bảng.

• **Vị trí code minh chứng:**

Câu #29: Tại sao cần chuẩn bị video demo dự phòng (Demo Backup Video) trước ngày vấn đáp?

• **Trả lời cốt lõi:** Phòng ngừa các rủi ro bất khả kháng tại phòng thi (mất điện, đứt cáp mạng LAN, lỗi xung đột IP phòng máy của trường); video quay sẵn rõ nét từng kịch bản giúp chứng minh sản phẩm đã chạy thực tế hoàn hảo.

• **Vị trí code minh chứng:**

Câu #30: Quy trình đóng gói và tài liệu hướng dẫn `README.md` cần cung cấp những thông tin gì?

• **Trả lời cốt lõi:** Cung cấp cấu hình phần cứng yêu cầu, chuỗi kết nối CSDL mẫu, tài khoản Admin/Operator mặc định, thứ tự khởi động các ứng dụng (Central Server trước $\rightarrow$ Agent $\rightarrow$ Admin Client) và danh mục các kịch bản demo mẫu.

• **Vị trí code minh chứng:**

PHẦN 8: XỬ LÝ SỰ CỐ NHANH & 14 RỦI RO THỰC TẾ

8.1. Cẩm nang cấp cứu 5 lỗi lập trình kinh điển (Tra cứu trong 10 giây)

• Lỗi 1: 'Cross-thread operation not valid' (WinForms bị đơ hoặc ném ngoại lệ):
Nguyên nhân: Luồng mạng nền tự ý thay đổi Text hoặc BackColor của control trên giao diện.
Cách sửa: Dùng if (InvokeRequired) BeginInvoke(new Action(() => { ... }));

• Lỗi 2: 'ERR_STREAM_TRUNCATED' hoặc 'EndOfStreamException' (Đứt kết nối mạng giữa chừng):
Nguyên nhân: stream.ReadAsync trả về 0 byte khi máy kia tắt socket hoặc rớt mạng.
Cách sửa: Bọc trong try/catch (EndOfStreamException), đóng socket và gọi hàm kết nối lại có backoff.

• Lỗi 3: 'ERR_CONCURRENCY_VIOLATION' (Xung đột phiên bản khi cập nhật sự cố):
Nguyên nhân: Quản trị viên khác đã thao tác trước làm thay đổi giá trị rowversion trong CSDL.
Cách sửa: Server trả về 409 Conflict kèm dữ liệu mới nhất; Client hiển thị thông báo và tự nạp lại trạng thái sự cố.

• Lỗi 4: 'AuthenticationException: Remote certificate is invalid':
Nguyên nhân: Chứng chỉ SSL tự ký chưa khớp địa chỉ IP máy chủ trong mạng LAN.
Cách sửa: Kiểm tra cấu hình IP trong chứng chỉ SAN hoặc cấu hình tin cậy chứng chỉ phòng lab.

• Lỗi 5: 'The transaction log for database is full':
Nguyên nhân: Chạy thử tải nạp hàng nghìn bản ghi làm đầy tệp LDF của SQL Server.
Cách sửa: Vào SSMS, chuột phải CSDL -> Properties -> Options -> Recovery model chọn 'Simple'.

8.2. Danh mục 14 rủi ro thực tế (Tóm tắt nhanh dấu hiệu và cách phòng ngừa)

• [RSK-01] Chệch hướng phạm vi (Scope Creep) / Làm Collector P1 quá sớm (Phụ trách: SV C) — Phòng ngừa: Tuân thủ nghiêm ngặt nguyên tắc: Chỉ kích hoạt P1 nếu M4 đạt 100% và còn dư ít nhất 15h dự phòng. | Dự phòng: Kích hoạt quyết định DEFER P1 ngay lập tức, cắt giảm 47.0h, dồn toàn lực vào ổn định và kiểm thử MVP.

• [RSK-02] Lệch pha giao thức mạng (Protocol Drift) giữa 3 thành viên A, B, C (Phụ trách: SV A) — Phòng ngừa: Dùng chung hợp đồng CONTRACT_MATRIX.md, 16 fixture JSON chuẩn và kiểm thử chéo bằng test-frame-codec. | Dự phòng: Dừng code ngay lập tức, họp khẩn 30 phút đối chiếu theo 03_PROTOCOL_SPEC.md để chốt lại định dạng.

• [RSK-03] Lỗi phân mảnh gói tin TCP (Partial Read / Frame Coalescing) (Phụ trách: SV A) — Phòng ngừa: Hiện thực FrameCodec chuẩn với 4 byte Length Big-Endian, vòng lặp ReadExactlyBytesAsync đọc tích lũy đủ byte. | Dự phòng: Chạy bộ test-frame-codec với 10,000 khung ngẫu nhiên để cô lập lỗi giải mã trước khi tích hợp vào server.

• [RSK-04] Central Server bị nghẽn hàng đợi / Tràn bộ nhớ RAM (OOM) (Phụ trách: SV B) — Phòng ngừa: Dùng BoundedChannel giới hạn 1000 phần tử, áp lực ngược (Backpressure) và nạp số liệu theo lô bất đồng bộ. | Dự phòng: Tạm dừng tiếp nhận kết nối mới, ghi log cảnh báo và điều chỉnh chu kỳ gửi số liệu của Agent lên 10 giây.

• [RSK-05] Bão kết nối lại đồng loạt (Reconnect Storm) làm sập Central (Phụ trách: SV A) — Phòng ngừa: Cài đặt thuật toán Exponential Backoff kèm độ trễ ngẫu nhiên (Full Jitter 1-5s) trên tất cả Agent. | Dự phòng: Cấu hình Server Socket Backlog = 100 và giới hạn tốc độ tiếp nhận kết nối mới trên giây.

• [RSK-06] Xung đột tranh chấp (Race Condition) khi 2 Admin cùng thao tác sự cố (Phụ trách: SV B/C) — Phòng ngừa: Dùng Optimistic Concurrency Control (OCC) với cột rowversion trong SQL Server và gửi expectedRowVersion từ UI. | Dự phòng: Server trả về lỗi ERR_CONCURRENCY_VIOLATION; WinForms tự động thông báo và tải lại trạng thái mới nhất.

• [RSK-07] Cơ sở dữ liệu SQL Server gặp sự cố / Khởi động lại đột ngột (Phụ trách: SV B) — Phòng ngừa: Bọc câu lệnh trong cơ chế Retry có backoff, duy trì cờ DbDegraded và bảo vệ hàng đợi đệm trong RAM. | Dự phòng: Giữ kết nối máy trạm, tiếp tục nhận số liệu vào bộ đệm bộ nhớ; tự động ghi nhận bù khi DB online lại.

• [RSK-08] Đơ luồng giao diện WinForms (UI Freeze) do gọi mạng hoặc vẽ lại (Phụ trách: SV C) — Phòng ngừa: Mọi tác vụ mạng chạy trên Task nền độc lập; dùng Control.Invoke/BeginInvoke marshal an toàn về luồng UI. | Dự phòng: Chỉ cập nhật ô dữ liệu thay đổi trên DataGridView/Grid thay vì gọi Invalidate() toàn bộ form.

• [RSK-09] Lỗi chứng chỉ số TLS / Bắt tay thất bại trên mạng LAN thật (Phụ trách: SV A/B) — Phòng ngừa: Tạo chứng chỉ phát triển tự ký với SAN chứa đúng địa chỉ IP máy chủ LAN; cài đặt CA vào máy kiểm thử. | Dự phòng: Có sẵn cờ cấu hình DevAllowUntrustedRoot trong môi trường nội bộ để gỡ rối nhanh khi cần.

• [RSK-10] Mạng Wi-Fi / LAN của trường đại học bị chặn cổng hoặc ngắt kết nối (Phụ trách: SV A) — Phòng ngừa: Mang theo Router Wi-Fi cá nhân độc lập và dây cáp mạng LAN chuẩn Cat6 để tự thiết lập mạng riêng. | Dự phòng: Chuyển toàn bộ 3 khối về chạy trên localhost (127.0.0.1) trên cùng 1 máy tính xách tay cấu hình cao.

• [RSK-11] Một thành viên bị kẹt nhiệm vụ (Blocker) quá 1 ngày làm việc (Phụ trách: SV A/B/C) — Phòng ngừa: Kích hoạt vai trò Backup Owner tương ứng nhảy vào pair programming hỗ trợ ngay trong ngày. | Dự phòng: Tổ chức họp khẩn, giảm độ phức tạp của giải pháp hoặc tách nhỏ phần việc để không cản trở 2 bạn còn lại.

• [RSK-12] Sử dụng AI sinh code nhưng thành viên không hiểu bản chất (Phụ trách: SV A/B/C) — Phòng ngừa: Buộc phải trả lời đạt bộ 30 câu hỏi Viva cá nhân; cam kết không commit bất kỳ dòng code nào chưa hiểu rõ. | Dự phòng: Diễn tập phản biện nội bộ hàng tuần, truy hỏi từng hàm cho đến khi thành viên tự tin giải thích mạch lạc.

• [RSK-13] Giao diện bị vỡ hạt / Mất nút bấm khi chiếu lên máy chiếu hội đồng (High-DPI) (Phụ trách: SV C) — Phòng ngừa: Khai báo PerMonitorV2 trong tệp manifest của WinForms và dùng layout TableLayoutPanel linh hoạt. | Dự phòng: Cài đặt độ phân giải màn hình về chuẩn 1920x1080 Scale 100% trước khi kết nối dây HDMI máy chiếu.

• [RSK-14] Tràn nhật ký giao dịch SQL Server (LDF Full) do kiểm thử số liệu kéo dài (Phụ trách: SV B) — Phòng ngừa: Cấu hình CSDL ở chế độ Simple Recovery Model cho môi trường thử nghiệm và chạy lệnh xóa lô nhỏ. | Dự phòng: Thực hiện lệnh DBCC SHRINKFILE để thu hẹp tệp log và chạy tác vụ RetentionWorker xóa dữ liệu cũ.

PHẦN 9: KỊCH BẢN DIỄN TẬP DEMO 12 PHÚT TRƯỚC HỘI ĐỒNG

Phân vai và bấm giờ: Hội đồng chỉ cho nhóm 12 phút để trình diễn sản phẩm. Nhóm hãy bấm đồng hồ và diễn tập đúng 6 bước sau:

• Phút 00-02 | BƯỚC 1: Khởi động & Bắt tay an toàn:
B bật Central Server. A bật 2 Agent. C mở WinForms đăng nhập. Cả 3 máy kết nối TLS thành công.

• Phút 02-04 | BƯỚC 2: Giám sát phòng máy thời gian thực:
Màn hình C hiển thị các ô vuông máy tính màu Xanh lá (ONLINE). Giải thích chu kỳ gửi nhịp tim 5s và mẻ đo lường METRIC_BATCH.

• Phút 04-06 | BƯỚC 3: Giả lập quá tải CPU:
A chạy script kích CPU >= 90% trên máy trạm. Sau 3 chu kỳ lấy mẫu (Hysteresis), máy chuyển màu Vàng, Server phát sóng bản tin INCIDENT_OPEN và tạo sự cố màu Đỏ trên màn hình C.

• Phút 06-08 | BƯỚC 4: Tranh chấp 2 Admin (OCC):
Mở 2 cửa sổ Admin cùng bấm nút [TIẾP NHẬN] trên 1 sự cố. Người sau nhận mã lỗi ERR_CONCURRENCY_VIOLATION chứng minh bẫy lỗi OCC thành công.

• Phút 08-10 | BƯỚC 5: Thử thách rút dây mạng:
A rút trực tiếp dây mạng máy 1. Đếm đúng 20 giây (> 20s không nhận heartbeat), máy 1 trên màn hình C chuyển màu Xám (OFFLINE). Cắm dây lại, tự hồi phục màu Xanh.

• Phút 10-12 | BƯỚC 6: Xuất báo cáo CSV & Chuyển sang Vấn đáp:
C mở màn hình Báo cáo, bấm nút Xuất file CSV mở bằng Excel xem lịch sử. Cả 3 bạn sẵn sàng trả lời vấn đáp cá nhân.

> 📌 **KẾ HOẠCH DỰ PHÒNG NẾU PHÒNG THI GẶP SỰ CỐ KỸ THUẬT**
> Nếu gặp sự cố mạng hoặc thiết bị tại phòng thi (router lab chập chờn, máy chiếu):
> 1. Lập tức mở video DemoBackup.mp4 quay sẵn độ phân giải 1080p trên máy tính.
> 2. Vừa chiếu video vừa thuyết minh theo đúng 6 bước nghiệp vụ trên.
> 3. Mở sẵn Visual Studio để hội đồng kiểm tra mã nguồn thực tế.
> 👉 Hội đồng đánh giá cao sự chuẩn bị chu đáo và khả năng dự phòng rủi ro của nhóm!

PHỤ LỤC: TỪ ĐIỂN THUẬT NGỮ 'NÓI TIẾNG NGƯỜI'

Bảng tra cứu nhanh 15 thuật ngữ cốt lõi được giải thích bằng ví dụ đời thường:

• Byte Stream (Dòng byte)

Nói đơn giản: Dữ liệu truyền qua mạng liên tục như dòng nước chảy trong ống, không có ranh giới từng gói.
Ví dụ đời thường: Giống như đổ một chậu nước vào đường ống: Nước chảy liên tục, không chia thành từng ca nước riêng biệt.
Áp dụng trong LabGuard: Khi Agent gửi 2 bản tin JSON, Server có thể đọc được cả 2 bản tin dính liền vào nhau.
Nhớ khi nào: Khi viết hàm đọc Socket (FrameCodec) ở Tuần 1.

• Framing (Đóng khung gói tin)

Nói đơn giản: Gắn thêm một nhãn ghi độ dài gói tin ở đầu để bên nhận biết đọc bao nhiêu byte là đủ một bức thư.
Ví dụ đời thường: Dán nhãn ghi 'Gói hàng nặng 5kg': Người nhận cân đủ 5kg là biết đã nhận trọn vẹn gói hàng.
Áp dụng trong LabGuard: Gắn 4 byte Big-Endian chỉ độ dài N của chuỗi UTF-8 JSON trước mỗi bản tin gửi đi.
Nhớ khi nào: Khi hiện thực LG-A-002 và LG-B-003.

• Partial Read (Đọc thiếu byte)

Nói đơn giản: Hàm đọc stream.ReadAsync chỉ đọc được một phần dữ liệu vì mạng bị trễ hoặc dữ liệu chưa kịp tới.
Ví dụ đời thường: Nhận một lá thư bị xé làm đôi: Mới nhận được nửa trang đầu, phải chờ bưu tá giao tiếp nửa trang sau.
Áp dụng trong LabGuard: Đọc header 4 byte nhưng lệnh ReadAsync đầu tiên chỉ trả về 2 byte, phải lặp lại để đọc nốt 2 byte.
Nhớ khi nào: Bắt buộc phải nhớ khi thầy cô hỏi vấn đáp về Socket.

• Heartbeat (Nhịp tim sống)

Nói đơn giản: Tín hiệu nhỏ gửi định kỳ để báo: 'Tôi vẫn đang sống, đừng coi tôi là đã sập!'.
Ví dụ đời thường: Tiếng tích tắc của đồng hồ hoặc bảo vệ bấm chuông điểm danh mỗi 15 phút.
Áp dụng trong LabGuard: Agent gửi bản tin HEARTBEAT mỗi 5 giây. Nếu quá 20 giây không thấy thì Server báo OFFLINE.
Nhớ khi nào: Khi làm LG-A-007 và LG-B-006 ở Tuần 2.

• Deduplication (Khử trùng lặp)

Nói đơn giản: Nhận lại cùng một lô dữ liệu cũ thì gửi xác nhận nhưng bỏ qua, không lưu lần thứ hai vào CSDL.
Ví dụ đời thường: Người giao hàng giao lại món quà bạn đã nhận hôm qua: Bạn ký nhận để họ về nhưng không cất thêm quà rác.
Áp dụng trong LabGuard: Server kiểm tra bảng ProcessedBatches bằng bộ khóa (DeviceId, BootId, BatchId) trước khi lưu.
Nhớ khi nào: Khi làm LG-B-005 và bảo vệ đồ án.

• Backpressure (Áp lực ngược)

Nói đơn giản: Bên gửi gửi quá nhanh trong khi bên nhận xử lý không kịp; phải có cơ chế chặn bớt lại kẻo tràn RAM.
Ví dụ đời thường: Đường cao tốc bị tắc: Trạm thu phí phải đóng bớt làn để xe không dồn vào làm vỡ trận bên trong thành phố.
Áp dụng trong LabGuard: Dùng BoundedChannel giới hạn 1000 phần tử. Nếu đầy thì tạm dừng đọc socket từ Agent.
Nhớ khi nào: Khi giải thích kiến trúc Server LG-B-003.

• OCC (Khóa lạc quan)

Nói đơn giản: Cho nhiều người cùng đọc dữ liệu, nhưng lúc bấm lưu phải kiểm tra xem có ai nhanh tay sửa trước mình chưa.
Ví dụ đời thường: Đặt vé máy bay: Hai người cùng xem chiếc ghế 12A, ai bấm thanh toán trước được ghế, người sau bị báo hết vé.
Áp dụng trong LabGuard: Hai Admin cùng bấm nhận sự cố: Người sau nhận mã ERR_CONCURRENCY_VIOLATION và màn hình tự tải lại.
Nhớ khi nào: Khi làm LG-B-009, LG-C-008 và demo перед hội đồng.

• rowversion / TIMESTAMP

Nói đơn giản: Một con số bí mật trong SQL Server tự động nhảy số mỗi khi dòng dữ liệu bị chỉnh sửa.
Ví dụ đời thường: Dấu mộc chứng thực có ghi số lần sửa: Nếu số lần sửa khác với lúc bạn cầm văn bản thì văn bản đã bị đổi.
Áp dụng trong LabGuard: Cột RowVersion trong bảng Incidents dùng để so sánh WHERE RowVersion = @expectedRowVersion.
Nhớ khi nào: Khi viết câu lệnh UPDATE kiểm soát tranh chấp sự cố.

• Control.Invoke (Chuyển luồng UI)

Nói đơn giản: Trong WinForms, chỉ có luồng giao diện chính mới được quyền vẽ nút bấm; luồng mạng muốn đổi màu phải nhờ thư ký Invoke.
Ví dụ đời thường: Khách lạ không được tự tiện viết lên bảng thông báo của công ty mà phải nhờ lễ tân

viết hộ.
Áp dụng trong LabGuard: Khi nhận gói tin mạng trên background thread, gọi Control.Invoke để đổi màu máy trạm sang Xanh/Vàng.
Nhớ khi nào: Khi gặp lỗi kinh điển 'Cross-thread operation not valid'.

• PBKDF2 & Salt

Nói đơn giản: Thuật toán băm mật khẩu cực mạnh kết hợp một chuỗi ngẫu nhiên (salt) và lặp 600,000 lần để chống hacker dò pass.
Ví dụ đời thường: Cho thêm muối vào bột mì rồi nhào nặn 600,000 lần: Không ai có thể tách lại hạt muối hay hạt bột ban đầu.
Áp dụng trong LabGuard: Mật khẩu Admin được băm bằng PBKDF2-HMAC-SHA256 lưu trong bảng Users.
Nhớ khi nào: Khi làm LG-B-004 và trả lời viva bảo mật.

• TLS & SslStream

Nói đơn giản: Mã hóa toàn bộ dữ liệu chạy trên dây mạng để hacker cắm Wireshark vào cũng chỉ thấy ký tự rác vô nghĩa.
Ví dụ đời thường: Nói chuyện bằng mật mã riêng trong phòng kín: Người đứng ngoài hành lang nghe lén không hiểu gì.
Áp dụng trong LabGuard: Bọc NetworkStream bằng SslStream để bảo vệ thông tin mật khẩu và dữ liệu phòng máy.
Nhớ khi nào: Khi chụp ảnh Wireshark làm bằng chứng nộp đồ án.

• Snapshot (Ảnh chụp toàn cảnh)

Nói đơn giản: Gói tin chụp lại toàn bộ tình trạng của tất cả các máy trong phòng gửi cho Admin vừa mới mở phần mềm.
Ví dụ đời thường: Chụp một bức ảnh toàn cảnh lớp học để giáo viên mới bước vào biết ngay bạn nào đang ngồi đâu.
Áp dụng trong LabGuard: Chuỗi SNAPSHOT_BEGIN -> SNAPSHOT_PAGE -> SNAPSHOT_END nạp đầy đủ 40 máy trạm lên lưới WinForms.
Nhớ khi nào: Khi làm LG-C-005 ở Tuần 2.

• CancellationToken

Nói đơn giản: Một cái cờ hiệu dùng để báo cho các luồng ngầm biết: 'Dừng lại ngay, người dùng vừa bấm tắt chương trình!'.
Ví dụ đời thường: Hiệu lệnh còi báo động: Khi nghe tiếng còi, mọi công nhân lập tức dừng máy móc và giải tán an toàn.
Áp dụng trong LabGuard: Truyền cancellationToken vào ReadAsync, Delay, Channel để tắt ứng dụng êm ru (Graceful Shutdown).
Nhớ khi nào: Khi viết các hàm async/await trong toàn bộ project.

• Exponential Backoff & Jitter

Nói đơn giản: Chờ đợi kết nối lại tăng dần theo cấp số nhân (1s, 2s, 4s...) kèm số lẻ ngẫu nhiên để không làm sập máy chủ.
Ví dụ đời thường: Cả lớp cùng ùa vào cửa một lúc sẽ gây kẹt cứng; nếu mỗi bạn tự động tản ra chờ vài giây ngẫu nhiên thì qua cửa êm.
Áp dụng trong LabGuard: Agent tự động thử lại khi mất kết nối Central, tránh gây ra hiện tượng Bão kết nối lại (Reconnect Storm).
Nhớ khi nào: Khi làm LG-A-009.

• Hysteresis (Độ trễ chống rung)

Nói đơn giản: Ngưỡng kích hoạt sự cố và ngưỡng tắt sự cố phải lệch nhau một khoảng để cảnh báo không bị nhấp nháy liên tục.
Ví dụ đời thường: Điều hòa: 30 độ C mới bật máy lạnh, giảm xuống 24 độ C mới tắt máy lạnh; chứ không bật tắt liên tục ở 27 độ.
Áp dụng trong LabGuard: CPU >= 90% liên tiếp 3 mẫu mới tạo sự cố; CPU <= 75% liên tiếp 3 mẫu mới cho hồi phục bình thường.
Nhớ khi nào: Khi làm bộ đánh giá quy tắc LG-B-007.